

UNIVERSIDAD DE VALLADOLID

ESCUELA TÉCNICA SUPERIOR DE INGENIEROS DE TELECOMUNICACIÓN

Despliegue de un Servicio de Infraestructura 4G para la Detección de Señales en Vehículo Conectado

MÁSTER EN INGENIERÍA DE TELECOMUNICACIÓN



Equipo **G2TP2**

Miguel Barrigón Gómez

Lucía López García

Víctor Sánchez-Valencia

Rodrigo Seco Dujo

29 de mayo de 2026

Índice general

Glosario de acrónimos y abreviaturas	9
1. Contexto y Motivación	14
1.1. El vehículo conectado y autónomo	14
1.1.1. Niveles de conducción autónoma según la SAE	14
1.1.2. Sistemas ADAS	15
1.2. Estadísticas de seguridad vial	15
1.3. Arquitecturas de procesamiento: nube, edge y local	15
1.4. El papel del 4G y del 5G	16
1.5. Estructura del informe	16
2. Objetivos	18
2.1. Objetivo general	18
2.2. Objetivos específicos	18
3. Metodología del proyecto	20
3.1. Herramientas de planificación y gestión	20
3.1.1. Jira	20
3.1.2. Confluence, Drive, GitHub y RoboFlow	20
3.1.3. Diagrama de Gantt	21
3.2. Análisis de riesgos	25
3.3. Estimación de horas trabajadas	25
4. Arquitectura de 4G	26
4.1. El organismo 3GPP y su modelo de <i>Releases</i>	26
4.2. El patrón arquitectónico común de los sistemas celulares	26
4.3. Particularización al ecosistema 3GPP	27
4.3.1. El equipo de usuario	27
4.3.2. La red de acceso	28
4.3.3. La red troncal	28
4.4. Terminología: LTE, EPS y 4G	28
4.5. La arquitectura de LTE	29
4.5.1. La red de acceso: E-UTRAN	29
4.5.2. El núcleo de red: EPC	30
4.5.3. Interfaces internas del EPC	30
4.6. La interfaz radio de LTE	31
4.6.1. Capa física: técnicas de acceso múltiple y modulación	31
4.6.2. Estructura temporal de la trama LTE	32
4.6.3. Estructura temporal de la trama LTE	32
4.6.4. MIMO	33
4.6.5. Adaptación al canal: CQI, MCS y TBS	34
4.7. Recapitulación	34
5. Infraestructura de red 4G	35
5.1. Componentes esenciales de una red 4G	36
5.2. Propuesta de infraestructura real	37
5.2.1. Estudio de cobertura	37
5.2.2. Propuesta de infraestructura	44
5.2.3. Estudio de viabilidad y capacidad de red	45

5.3.	Infraestructura de red 4G en el laboratorio	50
5.3.1.	Selección del software 4G: DAFO	50
5.3.2.	Componentes empleados	51
5.3.3.	Arquitectura software: srsRAN sobre Docker Engine nativo	53
5.3.4.	Comandos de arranque	59
5.3.5.	Esquema final del laboratorio	60
6.	Rendimiento de la red 4G	61
6.1.	Procedimiento de prueba	61
6.2.	Capacidad disponible: estimación teórica	61
6.3.	Mediciones de la capacidad de la red	62
6.3.1.	Funcionamiento y limitaciones de <i>iperf3</i>	62
6.3.2.	Mediciones con TCP	62
6.3.3.	Mediciones con UDP	64
6.4.	Mediciones de latencia y jitter	65
6.5.	Mediciones de rendimiento del ordenador del <i>core</i>	66
6.6.	Problemas observados y soluciones aplicadas	67
6.6.1.	Latencia elevada y cortes de vídeo	67
6.6.2.	Pérdida de conexión por interferencias	67
6.6.3.	Conflicto de IPs entre la Jetson y el core	67
6.6.4.	Visualización OpenCV y problema X11	68
6.7.	Conclusiones del rendimiento	68
7.	Inteligencia Artificial	69
7.1.	Conceptos teóricos y objetivos	69
7.1.1.	Redes neuronales convolucionales (CNN)	69
7.1.2.	Métricas de evaluación	70
7.1.3.	Objetivos de la parte de IA del proyecto	70
7.2.	Selección del modelo de detección	71
7.2.1.	DAFO de YOLOv11	72
7.2.2.	Comparativa cuantitativa	72
7.3.	Herramientas y entorno de trabajo	73
7.4.	Datasets y etiquetado	73
7.4.1.	Punto de partida: tres corpus heredados	73
7.4.2.	Limpieza y homogeneización	74
7.4.3.	Catálogo inicial de 22 clases	74
7.4.4.	Carencias identificadas	74
7.4.5.	Reducción del <i>dataset</i> : 11 clases finales	75
7.4.6.	Distribución del nuevo dataset	76
7.4.7.	Captura y etiquetado de imágenes propias con el DeepRacer	77
7.5.	Iteraciones del modelo	77
7.5.1.	Modelo V1 — primer entrenamiento	78
7.5.2.	Modelo V2 — reetiquetado y reducción de clases	79
7.5.3.	Modelo V3 — incorporación de fotografías propias	79
7.5.4.	Modelo V4 — augmentation intensiva	80
7.5.5.	Modelo V5 — sin augmentation	80
7.5.6.	Modelo V6 — augmentation selectiva	81
7.5.7.	Modelo V7 — versión final	81
7.5.8.	Conclusiones de la iteración del modelo	84
7.6.	Inferencia local en la NVIDIA Jetson Orin Nano	85
7.6.1.	Motivación del paso de Roboflow a inferencia local	85
7.6.2.	Despliegue del entorno: 9 fases	85

7.6.3.	Resumen del entorno final en la Jetson	88
7.6.4.	Pipeline de inferencia	88
7.6.5.	Conclusiones del despliegue en Jetson	89
7.7.	Integración con la red 4G y el DeepRacer	89
7.7.1.	Comparativa de tiempos: nube vs local	90
7.8.	Escalado IA al proyecto: vehículo conectado en la autovía Valladolid-Segovia	90
7.8.1.	Entrenamiento desde cero basado en la normativa nacional	90
7.8.2.	Transferencia de aprendizaje y repositorios públicos	91
7.8.3.	Optimización del algoritmo y viabilidad de infraestructura	91
7.8.4.	Consideraciones de la red y despliegue de mejoras relevantes para la A-601	92
8.	Valoración económica	93
8.1.	E-UTRAN — estaciones de nueva planta	93
8.2.	E-UTRAN — estaciones en alquiler	93
8.3.	Adquisición del espectro	94
8.4.	Resumen E-UTRAN total	94
8.5.	EPC y red de transporte	94
8.5.1.	Procesado de IA: servicio en la nube frente a infraestructura propia	94
8.6.	Coste de operación anual	95
8.7.	Coste total de despliegue	96
9.	Conclusiones y líneas futuras	97
9.1.	Conclusiones	97
9.2.	Líneas futuras	97
A.	Documentación legal y permisos	99
A.1.	Cesión de uso del espectro radioeléctrico	99
A.2.	Pago de tasas por uso del dominio público radioeléctrico	101
A.3.	Solicitud de autorización de puesta en servicio de la estación radioeléctrica	103
A.4.	Cumplimentación técnico y carácter experimental del sistema	107
B.	Configuración resumida del laboratorio	108
B.1.	Configuración de red	108
B.2.	Parámetros radio del eNodeB	108
B.3.	Resumen del entorno Jetson	108
C.	Diario de trabajo del proyecto	109
C.1.	Semana 1 (9/2 – 15/2)	109
C.2.	Semana 2 (16/2 – 22/2)	109
C.3.	Semana 3 (23/2 – 1/3)	110
C.4.	Semana 4 (2/3 – 8/3)	110
C.5.	Semana 5 (9/3 – 15/3)	110
C.6.	Semana 6 (16/3 – 22/3)	110
C.7.	Semana 7 (23/3 – 29/3)	111
C.8.	Semana 8 (13/4 – 19/4)	111
C.9.	Semana 9 (20/4 – 26/4)	111
C.10.	Semana 10 (27/4 – 3/5)	112
C.11.	Semana 11 (4/5 – 10/5)	112
C.12.	Semana 12 (11/5 – 17/5)	113
C.13.	Semana 13 (18/5 – 24/5)	113
C.14.	Semana 14 (25/5 – 31/5)	113
C.15.	Semana 15 (1/6 – 4/6) - final de proyecto	114

C.16. Recuento total de horas trabajadas en el laboratorio	114
D. Configuración y códigos	115
D.1. Configuración del DeepRacer (<code>vehicle.conf</code>)	115
D.2. Script para la conducción autónoma (<code>coche_autonomo.py</code>)	115
D.3. Script de inferencia local con API Roboflow	122
D.4. Script de inferencia en la Jetson	123
D.5. Script para la obtención de la matriz de confusión	125
D.6. Configuración del eNodeB (<code>enb.conf</code> , extracto)	125
D.7. Configuración del EPC (<code>epc.conf</code>)	134
D.8. Base de datos de usuarios (<code>user_db.csv</code>)	136
E. Guía de instalación y configuración de la red 4G privada	137
E.1. Instalación de Docker Engine nativo	137
E.1.1. Desinstalación de Docker Desktop	137
E.1.2. Limpieza de repositorios conflictivos	137
E.1.3. Instalación de Docker Engine	137
E.2. Construcción de la imagen con soporte BladeRF	138
E.2.1. Creación del contenedor base	138
E.2.2. Instalación de dependencias y clonación del repositorio	138
E.2.3. Compilación e instalación	138
E.2.4. Verificación	138
E.2.5. Carga de la FPGA (<i>bitstream</i> v0.15.3)	139
E.2.6. Persistencia de la imagen Docker	139
E.3. Instalación de srsRAN 4G	139
E.3.1. Dependencias	139
E.3.2. Clonación y compilación con soporte BladeRF	139
E.3.3. Verificación	139
E.3.4. Generación de los ficheros de configuración	140
E.4. Configuración de la red 4G	140
E.4.1. Configuración del EPC (<i>core</i>)	140
E.4.2. Registro de la SIM en el HSS	140
E.4.3. Configuración del eNodeB	140
E.4.4. Configuración del <i>scheduler</i> para estabilidad de vídeo	140
E.4.5. Configuración NAT para salida a Internet	141
E.4.6. Arquitectura de red resultante	141
E.5. Instalación y configuración de Mosquitto MQTT	141
E.5.1. Instalación	141
E.5.2. Configuración	141
E.5.3. Arranque del servicio	142
E.5.4. Comandos de control del vehículo por MQTT	142
E.6. Configuración del vehículo DeepRacer	142
E.6.1. Activación del módem USB	142
E.6.2. Configuración de <code>vehicle.conf</code>	142
E.6.3. Arranque del servicio ARTEMIS	143
E.6.4. Indicadores de estado LED	143
E.7. Control del vehículo desde el <i>core</i>	143
E.7.1. Dependencias en el contenedor del <i>core</i>	143
E.7.2. Configuración X11 para visualización	143
E.7.3. Parámetros del servidor de control	144
E.7.4. Arranque del servidor	144
E.7.5. Activación del control remoto	144

E.8. Secuencia completa de arranque del sistema	144
E.8.1. <i>Host</i> del <i>core</i>	144
E.8.2. Contenedor del <i>core</i> — Terminal 1 (EPC + Mosquitto)	144
E.8.3. Contenedor del <i>core</i> — Terminal 2 (servidor de control)	145
E.8.4. Contenedor del eNodeB	145
E.8.5. Vehículo DeepRacer	145
E.8.6. Activación del control (Terminal 3 del <i>core</i>)	145
E.8.7. Comandos útiles de operación	145
Bibliografía	146

Índice de figuras

3.1. Diagrama de Gantt realizado al comienzo del proyecto como estimación	22
3.2. Diagrama de Gantt final generado en Jira. Distribución por epics: PLA, INF, IA, DOC	24
4.1. Rejilla de recursos físicos de una subtrama LTE con prefijo cíclico normal. Cada celda es un <i>Resource Element</i> (RE); un bloque de 12 subportadoras $\times$ 1 <i>slot</i> constituye un <i>Resource Block</i> (RB), y el par de RBs que abarca la subtrama completa forma un PRB <i>pair</i> , que es la unidad efectiva de asignación del <i>scheduler</i> . Adaptado de [TelecomHall Forum, 2021].	33
5.1. Autovía A-601 de Pinares, Valladolid-Segovia.	37
5.2. Propiedades de la Banda de Frecuencias en el estudio de Xirio.	39
5.3. Propiedades del sector (transmisor).	40
5.4. Propiedades del rango de colores.	41
5.5. Estudio de Cobertura con las antenas de Vodafone.	42
5.6. Estudio de Cobertura con las antenas disponibles de todos los operadores.	43
5.7. Estudio Cobertura total con antenas de operadores y propias.	44
5.8. Numero de señales etiquetadas por cada fotograma.	46
5.9. Tráfico (IMD) en la Autovía A-601 de Pinares, Valladolid-Segovia.	49
5.10. Blade RF empleada en el entorno del laboratorio.	52
5.11. Módem USB de Huawei empleado en el entorno del laboratorio.	52
5.12. Coche DeepRacer de AWS empleado en el entorno del laboratorio.	53
5.13. NVIDIA Jetson Orin nano empleada en el entorno del laboratorio.	53
5.14. Esquema laboratorio	60
6.1. Prueba iperf3 para el protocolo TCP y con CQI 15.	63
6.2. Captura de las medidas de CQI al realizar las pruebas a una distancia realista.	63
6.3. Prueba iperf3 para el protocolo TCP y con CQI 11.	63
6.4. Prueba iperf3 para el protocolo UDP y una tasa de transmisión de 16 Mbit/s.	64
6.5. Prueba iperf3 para el protocolo UDP y una tasa de transmisión de 20 Mbit/s.	64
6.6. Segunda prueba iperf3 para el protocolo UDP y una tasa de transmisión de 16 Mbit/s.	65
6.7. Ejecución de <code>ping</code> en condiciones de uso de red ideales.	66
6.8. Ejecución de <code>ping</code> sobre una red saturada.	66
6.9. Carga de CPU y memoria durante el funcionamiento del control autónomo.	67
7.1. Subcategorías de la Inteligencia Artificial.	69
7.2. Matriz de confusión genérica.	70
7.3. Mapa de calor de las etiquetas iniciales del <i>dataset</i>	75
7.4. Matriz de confusión del modelo V3 con confusiones de etiquetas.	77
7.5. Precisiones del primer modelo entrenado de YoloV11 por clases.	78
7.6. Imagen tomada durante el proceso de capturas para su etiquetado.	79
7.7. Inferencia realizada con el modelo V3.	80
7.8. Inferencia realizada donde se produce error en las señales de giros.	80
7.9. Inferencia realizada con el modelo V6.	81
7.10. Matriz de confusión del modelo en su versión definitiva.	82
7.11. Curva Confianza-Precisión del modelo v7.	83
7.12. Curva Precisión- <i>Recall</i> del modelo v7.	83
7.13. Curva Confianza- <i>Recall</i> del modelo v7.	84

7.14. Tabla resumen de las prestaciones del modelo final para cada clase.	84
7.15. Carretera A-601 cubierta de nieve	91
8.1. Posible arquitectura de <i>Edge Computing</i> con alquiler de infraestructura.	95
A.1. Contrato de cesión del espectro radioeléctrico.	100
A.2. Justificante de pago de tasas (Modelo 790)	102
A.3. Modelo de solicitud APS.	106

Índice de tablas

3.1. Matriz de riesgos identificados al inicio del proyecto.	25
4.1. Principales interfaces del EPC y elementos que conectan.	30
5.1. Resumen de la infraestructura propuesta.	45
5.2. DAFO de los <i>softwares</i> 4G <i>open source</i> candidatos.	51
5.3. Comparativa cuantitativa de <i>softwares</i> 4G candidatos.	51
7.1. DAFO de YOLOv11 como modelo de detección.	72
7.2. Comparativa cuantitativa entre modelos candidatos. Puntuación: 0=no cumple, 1=cumple ligeramente, 2=cumple consistentemente, 3=excelente.	72
7.3. Catálogo inicial de 22 clases con su volumen de etiquetas.	74
7.4. Clases finales del modelo YOLOv11 entrenado por el equipo G2TP2.	76
7.5. Distribución de etiquetas del corpus depurado.	76
7.6. Iteraciones del modelo YOLOv11 entrenadas durante el proyecto.	78
7.7. Configuración final del entorno de inferencia en la Jetson Orin Nano.	88
7.8. Comparativa de tiempos de inferencia: nube (Roboflow) vs local (Jetson).	90
8.1. Presupuesto de una E-UTRAN completa (nueva planta).	93
8.2. Presupuesto de una E-UTRAN en alquiler sobre torre existente.	93
8.3. Coste total de despliegue de la E-UTRAN.	94
8.4. Presupuesto de despliegue del EPC y la red de transporte.	94
8.5. Coste de operación anual estimado.	95
B.1. Direccionamiento IP empleado en el laboratorio.	108
B.2. Parámetros de la celda LTE del laboratorio.	108
C.1. Horas trabajadas registradas en Jira por miembro del equipo en el laboratorio.	114
E.1. Resumen de parámetros de la red 4G privada del laboratorio.	137
E.2. Parámetros del <i>scheduler</i> del eNodeB.	141
E.3. Comandos MQTT del protocolo ARTEMIS.	142
E.4. Indicadores LED del DeepRacer.	143
E.5. Parámetros de red del servidor de control UDP.	144

Glosario de acrónimos y abreviaturas

Acrónimo	Significado
3GPP	<i>Third Generation Partnership Project</i> . Organismo de estandarización que especifica las tecnologías GSM, UMTS, LTE y 5G.
ETSI	<i>European Telecommunications Standards Institute</i> . Organismo europeo de normalización en telecomunicaciones.
IEEE	<i>Institute of Electrical and Electronics Engineers</i> . Asociación profesional y organismo de estandarización.
ITU	<i>International Telecommunication Union</i> . Agencia de las Naciones Unidas para las telecomunicaciones.
SAE	<i>Society of Automotive Engineers</i> . Organismo que define los niveles de automatización de conducción (J3016).
EPS	<i>Evolved Packet System</i> . Denominación oficial del sistema 4G completo (E-UTRAN + EPC).
LTE	<i>Long Term Evolution</i> . Tecnología de acceso radio de cuarta generación. Utilizado coloquialmente para designar el sistema EPS completo.
LTE-A	<i>LTE-Advanced</i> . Evolución de LTE introducida en la <i>Release</i> 10, conforme a los requisitos IMT-Advanced.
NR	<i>New Radio</i> . Interfaz radio de quinta generación (5G), introducida en la <i>Release</i> 15 de 3GPP.
AN	<i>Access Network</i> . Red de acceso radio.
E-UTRAN	<i>Evolved Universal Terrestrial Radio Access Network</i> . Red de acceso de LTE, compuesta por eNodeBs.
eNB / eNodeB	<i>Evolved NodeB</i> . Estación base de LTE que integra todas las funciones de la red de acceso.
GERAN	<i>GSM/EDGE Radio Access Network</i> . Red de acceso de 2G/2.5G.
RAN	<i>Radio Access Network</i> . Red de acceso radio (término genérico).
UTRAN	<i>UMTS Terrestrial Radio Access Network</i> . Red de acceso de 3G.
CN	<i>Core Network</i> . Red troncal.
EPC	<i>Evolved Packet Core</i> . Núcleo de red de LTE, orientado íntegramente a paquetes IP.
HSS	<i>Home Subscriber Server</i> . Base de datos central de suscriptores del EPC (perfiles, IMSI, claves).
MME	<i>Mobility Management Entity</i> . Nodo de control del EPC: gestión de registro, autenticación, movilidad y <i>bearers</i> .
P-GW	<i>Packet Data Network Gateway</i> . Pasarela del EPC hacia redes externas (Internet, IMS).
S-GW	<i>Serving Gateway</i> . Pasarela del EPC para el plano de usuario; actúa como anclaje de movilidad.
PCRF	<i>Policy and Charging Rules Function</i> . Entidad del EPC que gestiona las políticas de QoS y tarificación.
ANDSF	<i>Access Network Discovery and Selection Function</i> . Entidad que ayuda al UE a descubrir redes no-3GPP.
IMS	<i>IP Multimedia Subsystem</i> . Subsistema de control de servicios multimedia sobre IP (voz, vídeo, mensajería).
GTP / GTP-U	<i>GPRS Tunnelling Protocol (User plane)</i> . Protocolo de encapsulamiento del plano de usuario sobre S1-U y S5.

Continúa en la página siguiente

Acrónimo	Significado
NAS	<i>Non-Access Stratum</i> . Capa de señalización entre el UE y la MME, independiente de la red de acceso.
S1AP	<i>S1 Application Protocol</i> . Protocolo de señalización del plano de control sobre la interfaz S1-MME.
SCTP	<i>Stream Control Transmission Protocol</i> . Protocolo de transporte utilizado por S1AP.
SIP	<i>Session Initiation Protocol</i> . Protocolo de señalización para el establecimiento de sesiones multimedia en IMS.
RRC	<i>Radio Resource Control</i> . Protocolo de control de recursos radio entre el UE y el eNB.
MAC	<i>Medium Access Control</i> . Subcapa de enlace que gestiona el acceso al medio radio.
HARQ	<i>Hybrid Automatic Repeat Request</i> . Mecanismo de retransmisión que combina FEC con ARQ.
MQTT	<i>Message Queuing Telemetry Transport</i> . Protocolo de mensajería ligero utilizado en IoT y en la comunicación con el DeepRacer.
OFDM	<i>Orthogonal Frequency Division Multiplexing</i> . Multiplexación por división de frecuencias ortogonales.
OFDMA	<i>Orthogonal Frequency Division Multiple Access</i> . Técnica de acceso múltiple basada en OFDM, usada en el enlace descendente de LTE.
SC-FDMA	<i>Single-Carrier Frequency Division Multiple Access</i> . Técnica de acceso múltiple del enlace ascendente de LTE, con baja PAPR.
MIMO	<i>Multiple-Input Multiple-Output</i> . Uso de múltiples antenas en transmisión y recepción para aumentar la capacidad.
FDD	<i>Frequency Division Duplexing</i> . Duplexación por división de frecuencia (UL y DL en bandas separadas).
TDD	<i>Time Division Duplexing</i> . Duplexación por división de tiempo (UL y DL alternan en la misma banda).
TDMA	<i>Time Division Multiple Access</i> . Acceso múltiple por división en el tiempo (usado en GSM).
FDMA	<i>Frequency Division Multiple Access</i> . Acceso múltiple por división en frecuencia.
WCDMA	<i>Wideband Code Division Multiple Access</i> . Técnica de acceso de UMTS/3G.
QPSK	<i>Quadrature Phase-Shift Keying</i> . Modulación de 2 bits por símbolo.
QAM	<i>Quadrature Amplitude Modulation</i> . Modulación en amplitud y fase (16-QAM: 4 bits/símbolo; 64-QAM: 6 bits/símbolo).
PAPR	<i>Peak-to-Average Power Ratio</i> . Relación entre la potencia pico y la media de una forma de onda.
RB / PRB	<i>Resource Block / Physical Resource Block</i> . Unidad mínima de asignación: 12 subportadoras $\times$ 1 slot (180 kHz $\times$ 0,5 ms).
RE	<i>Resource Element</i> . Unidad elemental del <i>grid</i> tiempo-frecuencia: 1 subportadora $\times$ 1 símbolo OFDM.
TTI	<i>Transmission Time Interval</i> . Intervalo de tiempo de transmisión; en LTE, equivale a 1 ms (1 subtrama).
CQI	<i>Channel Quality Indicator</i> . Indicador de calidad de canal reportado por el UE (valores 0–15).
MCS	<i>Modulation and Coding Scheme</i> . Esquema de modulación y codificación seleccionado por el eNB.

Continúa en la página siguiente

Acrónimo	Significado
TBS	<i>Transport Block Size</i> . Número de bits útiles transportados en una subtrama para un MCS y número de RBs dados.
BLER	<i>Block Error Rate</i> . Tasa de error de bloque.
SNR	<i>Signal-to-Noise Ratio</i> . Relación señal a ruido.
RSRP	<i>Reference Signal Received Power</i> . Potencia de la señal de referencia recibida; indicador primario de cobertura LTE.
EARFCN	<i>E-UTRA Absolute Radio Frequency Channel Number</i> . Número de canal que identifica unívocamente la frecuencia central en LTE.
REFSENS	<i>Reference Sensitivity</i> . Nivel mínimo de señal garantizado para recepción correcta según 3GPP TS 36.101.
DL	<i>Downlink</i> . Enlace descendente (eNB $\rightarrow$ UE).
UL	<i>Uplink</i> . Enlace ascendente (UE $\rightarrow$ eNB).
DMRS	<i>Demodulation Reference Signal</i> . Señal de referencia para la demodulación del canal.
PDCCH	<i>Physical Downlink Control Channel</i> . Canal de control del enlace descendente.
PDSCH	<i>Physical Downlink Shared Channel</i> . Canal compartido de datos del enlace descendente.
PUCCH	<i>Physical Uplink Control Channel</i> . Canal de control del enlace ascendente.
PUSCH	<i>Physical Uplink Shared Channel</i> . Canal compartido de datos del enlace ascendente.
RACH	<i>Random Access Channel</i> . Canal de acceso aleatorio utilizado por el UE para contactar con el eNB.
UE	<i>User Equipment</i> . Equipo de usuario (terminal + tarjeta SIM).
ME	<i>Mobile Equipment</i> . Hardware del terminal móvil.
MT	<i>Mobile Termination</i> . Parte del ME responsable de la comunicación con la red.
TE	<i>Terminal Equipment</i> . Parte del ME que aloja las aplicaciones del usuario.
UICC	<i>Universal Integrated Circuit Card</i> . Tarjeta inteligente extraíble que aloja la identidad del suscriptor.
SIM	<i>Subscriber Identity Module</i> . Aplicación de identidad de suscriptor sobre UICC (GSM).
USIM	<i>Universal Subscriber Identity Module</i> . Aplicación de identidad para UMTS/LTE sobre UICC.
IMSI	<i>International Mobile Subscriber Identity</i> . Identificador único del suscriptor en la red.
MCC	<i>Mobile Country Code</i> . Código de país del PLMN (3 dígitos).
MNC	<i>Mobile Network Code</i> . Código de red del PLMN (2–3 dígitos).
PLMN	<i>Public Land Mobile Network</i> . Red móvil terrestre pública, identificada por MCC + MNC.
TAC	<i>Tracking Area Code</i> . Código del área de seguimiento; determina la zona de paginación del UE.
SUI	<i>Stanford University Interim</i> . Modelo empírico de pérdidas de propagación para entornos suburbanos/rurales.
IMD	<i>Intensidad Media Diaria</i> . Número medio de vehículos que circulan por un tramo en 24 horas.
IA	Inteligencia Artificial.
YOLO	<i>You Only Look Once</i> . Familia de modelos de detección de objetos en tiempo real.

Continúa en la página siguiente

Acrónimo	Significado
CNN	<i>Convolutional Neural Network</i> . Red neuronal convolucional.
mAP	<i>mean Average Precision</i> . Métrica principal de evaluación en detección de objetos.
FP / FN	<i>False Positive / False Negative</i> . Detección incorrecta / detección omitida.
IoU	<i>Intersection over Union</i> . Métrica de solapamiento entre el <i>bounding box</i> predicho y el real.
ADAS	<i>Advanced Driver Assistance Systems</i> . Sistemas avanzados de asistencia a la conducción.
AEB	<i>Autonomous Emergency Braking</i> . Frenada autónoma de emergencia.
ACC	<i>Adaptive Cruise Control</i> . Control de cruceo adaptativo.
LDW	<i>Lane Departure Warning</i> . Aviso de salida involuntaria de carril.
LKA	<i>Lane Keeping Assist</i> . Asistente de mantenimiento de carril.
TSR	<i>Traffic Sign Recognition</i> . Reconocimiento de señales de tráfico.
BSD	<i>Blind Spot Detection</i> . Detección de ángulo muerto.
C-ITS	<i>Cooperative Intelligent Transport Systems</i> . Sistemas de transporte inteligente cooperativo.
V2X	<i>Vehicle-to-Everything</i> . Comunicación del vehículo con su entorno (otros vehículos, infraestructura, red).
C-V2X	<i>Cellular Vehicle-to-Everything</i> . Variante de V2X basada en tecnología celular (3GPP).
DGT	Dirección General de Tráfico.
SDR	<i>Software-Defined Radio</i> . Radio definida por software.
FPGA	<i>Field-Programmable Gate Array</i> . Matriz de puertas lógicas programable.
GPU	<i>Graphics Processing Unit</i> . Unidad de procesamiento gráfico, utilizada para inferencia de IA.
CPU	<i>Central Processing Unit</i> . Unidad central de procesamiento.
RRU	<i>Remote Radio Unit</i> . Unidad radio remota de una estación base.
BBU	<i>Baseband Unit</i> . Unidad de banda base de una estación base.
OLT	<i>Optical Line Terminal</i> . Terminal de línea óptica en una red PON.
ONT	<i>Optical Network Terminal</i> . Terminal de red óptica del usuario en una red PON.
USB	<i>Universal Serial Bus</i> . Bus serie universal para conexión de periféricos.
SSD	<i>Solid-State Drive</i> . Unidad de almacenamiento de estado sólido.
IMU	<i>Inertial Measurement Unit</i> . Unidad de medida inercial (acelerómetro + giroscopio).
LED	<i>Light-Emitting Diode</i> . Diodo emisor de luz.
API	<i>Application Programming Interface</i> . Interfaz de programación de aplicaciones.
CLI	<i>Command-Line Interface</i> . Interfaz de línea de comandos.
NAT	<i>Network Address Translation</i> . Traducción de direcciones de red.
DHCP	<i>Dynamic Host Configuration Protocol</i> . Protocolo de asignación dinámica de direcciones IP.
DNS	<i>Domain Name System</i> . Sistema de nombres de dominio.
IP	<i>Internet Protocol</i> . Protocolo de Internet.
TCP	<i>Transmission Control Protocol</i> . Protocolo de control de transmisión.
UDP	<i>User Datagram Protocol</i> . Protocolo de datagramas de usuario.
CAPEX	<i>Capital Expenditure</i> . Gasto de capital (inversión inicial).
OPEX	<i>Operational Expenditure</i> . Gasto operativo (coste recurrente).

Continúa en la página siguiente

Acrónimo	Significado
DAFO	Debilidades, Amenazas, Fortalezas y Oportunidades. Herramienta de análisis estratégico.
QoS	<i>Quality of Service</i> . Calidad de servicio.
MEC	<i>Multi-access Edge Computing</i> . Computación en el borde de acceso múltiple (ETSI).

1 Contexto y Motivación

En los últimos años, la Inteligencia Artificial (IA) ha cobrado una relevancia creciente como herramienta para resolver problemas que, hasta hace poco, resultaban inabordables o requerían un esfuerzo de investigación desproporcionado. Una de las aplicaciones más directas en las que la IA ha demostrado un potencial extraordinario es la conducción autónoma y, en general, el ecosistema del vehículo conectado.

1.1. El vehículo conectado y autónomo

La conducción autónoma se sustenta en gran medida en el procesamiento del entorno percibido por una serie de sensores: cámaras, radares, LiDAR e IMUs. De toda esa información, las imágenes son particularmente valiosas, ya que permiten identificar carriles y señalización en el suelo, obstáculos fijos y móviles, señalización crítica como semáforos y señales verticales, y posibles peligros para la vida del conductor o de otras personas del entorno.

Para que un vehículo autónomo pueda reaccionar a tiempo frente a una señal de STOP, un peatón cruzando o una señal de obras, se requiere procesar un volumen de imágenes lo suficientemente grande en un tiempo lo bastante corto como para que el vehículo pueda actuar de forma inmediata. Este requisito de tiempo real implica disponer de hardware dedicado al procesamiento eficiente y en masa de imágenes, típicamente basado en GPU.

1.1.1. Niveles de conducción autónoma según la SAE

La Society of Automotive Engineers (SAE) ha definido una clasificación de seis niveles de automatización de la conducción en su estándar J3016 [SAE International, 2021], ampliamente aceptada en la industria y en la regulación:

Nivel 0 — Sin automatización

El conductor humano realiza todas las tareas de conducción. El vehículo puede emitir avisos (luces, alarmas), pero no toma decisiones.

Nivel 1 — Asistencia al conductor

El sistema asume el control de una tarea concreta (control de velocidad de cruce adaptativo, asistente de mantenimiento de carril). El conductor sigue siendo plenamente responsable.

Nivel 2 — Automatización parcial

El sistema controla simultáneamente dirección y velocidad en escenarios concretos, pero el conductor debe vigilar permanentemente y estar preparado para tomar el control. Tesla Autopilot y Mercedes Drive Pilot son ejemplos comerciales.

Nivel 3 — Automatización condicional

El sistema asume el control completo en condiciones específicas (autovía, atascos), pero debe poder transferir el control al conductor con un aviso previo razonable. La latencia y la fiabilidad de la red son críticas en este nivel.

Nivel 4 — Automatización alta

El vehículo es plenamente autónomo en escenarios bien definidos, sin necesidad de intervención humana. Si las condiciones se salen del escenario seguro, el vehículo se detiene de forma controlada.

Nivel 5 — Automatización completa

El vehículo es autónomo en cualquier condición, equivalente a un conductor humano. Aún no existe ningún producto comercial en este nivel.

La demostración desarrollada en este proyecto se sitúa entre los niveles 1 y 2, ya que el sistema asiste al conductor en el reconocimiento de señales y puede ejecutar acciones de control derivadas (frenar ante un STOP, ajustar velocidad ante un LIMITE-30), pero es necesaria la supervisión humana. El objetivo del estudio es validar la viabilidad de soportar este tipo de servicios sobre infraestructura LTE.

1.1.2. Sistemas ADAS

Los sistemas de asistencia avanzada al conductor (*Advanced Driver-Assistance Systems*, ADAS) son la base sobre la que se construyen los niveles 1, 2 y 3 de automatización. Incluyen:

- Frenada automática de emergencia (AEB).
- Aviso de salida de carril (LDW) y mantenimiento de carril (LKA).
- Control de crucero adaptativo (ACC).
- Reconocimiento de señales de tráfico (TSR). Objetivo principal de este proyecto.
- Detección de puntos ciegos (BSD).
- Detección de fatiga del conductor.

Desde julio de 2024, en aplicación del Reglamento (UE) 2019/2144 [Parlamento Europeo y Consejo de la Unión Europea, 2019], todos los vehículos nuevos vendidos en la Unión Europea deben incorporar varios de estos sistemas como dotación de serie, lo que ha democratizado tecnologías que hasta hace poco eran exclusivas de gama alta.

1.2. Estadísticas de seguridad vial

Según datos publicados por la Dirección General de Tráfico (DGT), en 2025 se registraron en España 1119 fallecidos en accidentes de tráfico en vías interurbanas [Dirección General de Tráfico, 2026], una cifra estable respecto a años recientes, pero todavía lejos del objetivo de cero víctimas planteado por la Unión Europea para 2050 (Visión Cero). El 45 % de los accidentes se produce en autovías y autopistas, y un porcentaje significativo de ellos se asocia a distracciones del conductor o a una percepción tardía de las señales de tráfico, dos áreas donde los sistemas ADAS demuestran un gran beneficio.

La integración de redes celulares en estos sistemas (V2X, *Vehicle-to-Everything*) podría aportar mejoras adicionales:

- Información en tiempo real sobre el estado del tráfico, accidentes y obras.
- Comunicación directa entre vehículos para anticipar maniobras.
- Actualización dinámica del estado de las señales (límites variables, obras temporales).
- Coordinación con la infraestructura (semáforos, pasos a nivel).

1.3. Arquitecturas de procesamiento: nube, edge y local

Llegados al punto del despliegue, existen tres aproximaciones posibles para soportar la inferencia de IA en un vehículo conectado [Rodríguez Cayetano, 2026]:

1. **Procesado local en el vehículo** (*on-board*): el algoritmo se ejecuta en hardware dedicado a bordo del coche. Es la aproximación con menor latencia y mayor independencia de la red, pero exige hardware potente, espacio físico y consumo eléctrico considerables, que encarecerían de manera considerable el vehículo.
2. **Procesado en la nube**: el coche envía las imágenes a un servidor remoto que devuelve los resultados de inferencia. Reduce el coste de hardware embarcado, pero introduce latencias variables y dependencia de la conectividad fluctuante de internet, sin garantías de QoS.
3. **Procesado en el borde de la red** (*edge computing*): el algoritmo se ejecuta en un servidor cercano al usuario, idealmente en la propia infraestructura del operador móvil (MEC, *Mobile Edge Computing*) [ETSI White Paper, 2023]. Combina la baja latencia del procesamiento local con la centralización del procesamiento en la nube y abarata de manera considerable el proyecto si se realiza a demanda.

1.4. El papel del 4G y del 5G

El despliegue masivo de redes 4G y, sobre todo, la llegada del 5G han abierto la puerta a las dos últimas opciones, que podrían suponer un ahorro importante en hardware embarcado. La latencia mínima asumible en LTE es de 30 ms a 50 ms, valor que puede no ser suficiente para conducción autónoma plena (nivel SAE 4–5) pero sí para escenarios de asistencia y de toma de decisiones que no requieran reacción inmediata. El 5G NR (*New Radio*) reduce la latencia teórica a < 5 ms en condiciones óptimas, lo que abriría la puerta al nivel SAE 4 sobre red móvil.

La madurez tecnológica del despliegue 4G y su presencia (cobertura prácticamente total del territorio nacional español) la convierten en un punto de partida natural para validar la viabilidad de estas arquitecturas. El presente informe explora precisamente esa posibilidad: utilizar una red LTE para soportar un servicio de detección de señales de tráfico desde un vehículo conectado, asumiendo todas las limitaciones del estándar y poniéndolas a prueba sobre una demostración funcional, complementado con un diseño teórico para una autovía real.

1.5. Estructura del informe

El presente informe está organizado de la siguiente forma:

- El **Capítulo 2** establece los objetivos generales y específicos del proyecto.
- El **Capítulo 3** describe la metodología de gestión empleada (Jira, Confluence, GitHub, Roboflow y Drive) y aporta una estimación de horas trabajadas y coste humano.
- El **Capítulo 4** presenta los conceptos teóricos de las redes 4G/LTE: 3GPP, arquitectura E-UTRAN/EPC, interfaces, capa física, MIMO.
- El **Capítulo 5** es el núcleo del proyecto en su parte de infraestructura: estudio de cobertura sobre Xirio, propuesta de despliegue, valoración de viabilidad y la demostración en el laboratorio en 13 fases.
- El **Capítulo 6** recoge las mediciones de rendimiento de la demostración.
- El **Capítulo 7** aborda el bloque de IA: selección del modelo, generación del *dataset*, las siete iteraciones del modelo y el despliegue en la Jetson Orin Nano.
- El **Capítulo 8** cuantifica el coste económico del despliegue propuesto.
- El **Capítulo 9** presenta las conclusiones y líneas futuras.

- Los **Anexos** recogen la documentación legal, la configuración del laboratorio, el diario completo del proyecto semana a semana y el código fuente principal.

2 Objetivos

En este informe se explora el despliegue de una red LTE para ser utilizada en la detección de señales de tráfico en un vehículo conectado. El procesamiento se realiza en la **Jetson Orin Nano**(paradigma *edge computing*), eliminando la dependencia de la nube y reduciendo la latencia de inferencia. La red LTE actúa como canal de control y telemetría entre el coche y el servidor, abriendo la puerta a una operación supervisada y remota.

Se pretende investigar las capacidades de las redes LTE actuales para sostener servicios tan exigentes como el vehículo conectado. La pregunta a responder es directa: **¿es viable utilizar LTE como soporte de un servicio de conducción asistida basado en visión por *Machine Learning*?**

Para cumplir con la cláusula tercera del pliego del proyecto, nuestro equipo ha asumido el papel de un grupo de ingenieros adjudicatarios de un contrato para desplegar un servicio de vehículo conectado sobre infraestructura propia e infraestructura ya instalada. El servicio consiste en la detección automática de señales de tráfico mediante técnicas de IA, con envío de telemetría y comandos a través de la red 4G. El despliegue se planifica sobre el tramo de autovía que une Valladolid y Segovia, de 104 km de longitud (km 0 al km 104) [Cláusula primera del pliego].

2.1. Objetivo general

Desarrollar y validar un futuro despliegue del servicio de vehículo conectado que integre una infraestructura de red 4G para la transmisión de vídeo y comandos de control en tiempo real, junto con un sistema de IA capaz de detectar señales de tráfico.

2.2. Objetivos específicos

Gestión

Coordinar el flujo de trabajo del equipo mediante **Jira** (planificación, sprints y backlog), **GitHub** (control de versiones de código), **Confluence**, y **Drive** (documentación). Tramitar los permisos legales y administrativos para el uso del espectro radioeléctrico. Elaborar una memoria técnico-económica detallada del despliegue propuesto.

Red 4G

Desplegar un núcleo (EPC) y una estación base (eNodeB) 4G completos sobre software *open source* (**srsRAN 4G**). Configurar la SDR **BladeRF 2.0 micro xA9** como frontal radio. Establecer comunicación inalámbrica estable en la **banda 7** con el módem Huawei 4G LTE. Realizar mediciones de rendimiento que aseguren la viabilidad del procesamiento de imágenes en tiempo real.

Inteligencia Artificial

Gestionar y etiquetar un *corpus* específico para el entrenamiento del modelo de *Machine Learning* usando la plataforma **Roboflow**. Entrenar el modelo **YOLOv11** capaz de identificar señales de tráfico con una adecuada precisión para la interpretación por parte del vehículo. Implementar el modelo en una **NVIDIA Jetson Orin Nano** para ejecución *edge*, eliminando la dependencia de Roboflow.

Integración

Integrar el hardware de captura y procesamiento en el vehículo **AWS DeepRacer**. Sincronizar el flujo de vídeo desde la cámara del vehículo hasta el módulo de IA a través de la red 4G mediante un broker MQTT (Mosquitto) y un servidor UDP de control.

Finalmente, todo el software empleado para crear la red de laboratorio, entrenar el modelo y operar el vehículo es de código abierto o, al menos, de uso gratuito. Los pesos del modelo y los *scripts* de control quedan disponibles en el repositorio interno del equipo: <https://github.com/g2tp2uva-lab/G2TP2>.

3 Metodología del proyecto

Para llevar a cabo el proyecto, se ha utilizado un conjunto de herramientas de planificación y colaboración que ha permitido tener un control claro del progreso y una trazabilidad completa de las tareas realizadas. En este capítulo se describen dichas herramientas y se aporta una estimación de las horas de trabajo dedicadas.

3.1. Herramientas de planificación y gestión

3.1.1. Jira

Como herramienta principal de gestión y planificación se ha empleado **Atlassian Jira** [Atlassian, 2026b], una plataforma de gestión de proyectos ampliamente utilizada en el ámbito del desarrollo ágil de software. Frente a la elección habitual de Trello en proyectos académicos, se optó por Jira por tres razones fundamentales:

1. **Gestión de *sprints* semanales.** Jira permite planificar el trabajo en bloques temporales bien definidos, alineados con las semanas lectivas del cuatrimestre. Lo que nos ha permitido llevar un control semanal de las tareas realizadas cada semana.
2. **Diagrama de Gantt integrado en la vista *Timeline*.** A diferencia de Trello, Jira ofrece una visualización temporal de las tareas con dependencias y *deadlines*, sin necesidad de recurrir a herramientas externas.
3. **Trazabilidad de tiempos.** La función de *aggregate time spent* permite registrar cuánto tiempo ha dedicado cada miembro a cada tarea, lo que ha resultado clave para el cálculo de horas totales del proyecto.

El espacio de trabajo del equipo se organizó en torno al proyecto **TP2CAR** (Taller de Proyectos II — Coche conectado), con el tablero accesible en <https://tp2uva.atlassian.net/>. Se definieron seis *epics* principales correspondientes a los grandes bloques del proyecto: **PLA** (Planificación y gestión), **DOC** (Documentación e Informe), **4G-DEMO** (Infraestructura 4G para la demostración), **4G-ESCALADO** (Infraestructura 4G escalado a la adjudicación), **IA-DEMO** (Inteligencia Artificial para la demostración) e **IA-ESCALADO** (Inteligencia Artificial escalado a la adjudicación). Cada *epic* agrupa las *stories* y *tasks* concretas asignadas a cada miembro.

3.1.2. Confluence, Drive, GitHub y Roboflow

Confluence

Integrado con Jira en la que se ha redactado la documentación viva del proyecto con páginas en las que se recogía todo el proceso seguido tanto en un “diario” como con páginas específicas de cada parte (infraestructura, modelo de IA, legislación...) en las que se recogían los pasos seguidos, los problemas detectados y sus soluciones, dudas y posibles mejoras... Cada página se enlaza desde el diario de trabajo en Jira, manteniendo un único punto donde se pueden encontrar todos los documentos [Atlassian, 2026a].

OneDrive

Almacenamiento en la nube para los ficheros pesados que no encajan en Confluence como capturas de Xirio, *datasheets*, copias de seguridad del Docker, etc.

GitHub

Control de versiones de todo el código fuente: ficheros de configuración de srsRAN (*epc.conf*, *enb.conf*, *user_db.csv*), *scripts* de inferencia del modelo de IA (local y

JETSON), servidor UDP de control y código del DeepRacer y almacenamiento de modelos entrenados (.pt) [GitHub, Inc., 2026].

Roboflow

Plataforma para la gestión de *datasheets*, etiquetados, división en datos de entrenamiento, evaluación y test. También ha permitido la selección del modelo, su entrenamiento y obtención de alguna métricas de rendimiento [Roboflow Inc., 2026].

3.1.3. Diagrama de Gantt

El diagrama de Gantt es una herramienta visual fundamental porque permite planificar, programar y hacer un seguimiento claro de las tareas, los hitos y las dependencias a lo largo de la vida de un proyecto, facilitando la coordinación del equipo y la comunicación del progreso a todos los involucrados. Elegir crear este diagrama directamente en **Jira** es la opción más estratégica si ya es nuestra herramienta principal de gestión, ya que elimina por completo la necesidad de introducir datos manualmente o usar herramientas externas. Al aprovechar funcionalidades como la vista de cronograma (*Timeline*) o los *Advanced Roadmaps*, Jira utiliza las tareas ya creadas para generar el diagrama y calcula de forma automática el impacto de cualquier modificación, ajustando en tiempo real las fechas y dependencias. Esto garantiza que nuestro cronograma esté siempre actualizado, centralizado y refleje fielmente la realidad del proyecto sin duplicar esfuerzos [Eddy Meardon, 2026].

A partir del backlog inicial de Jira se generó un primer diagrama de Gantt provisional, en el que se planificaron los grandes hitos del proyecto.

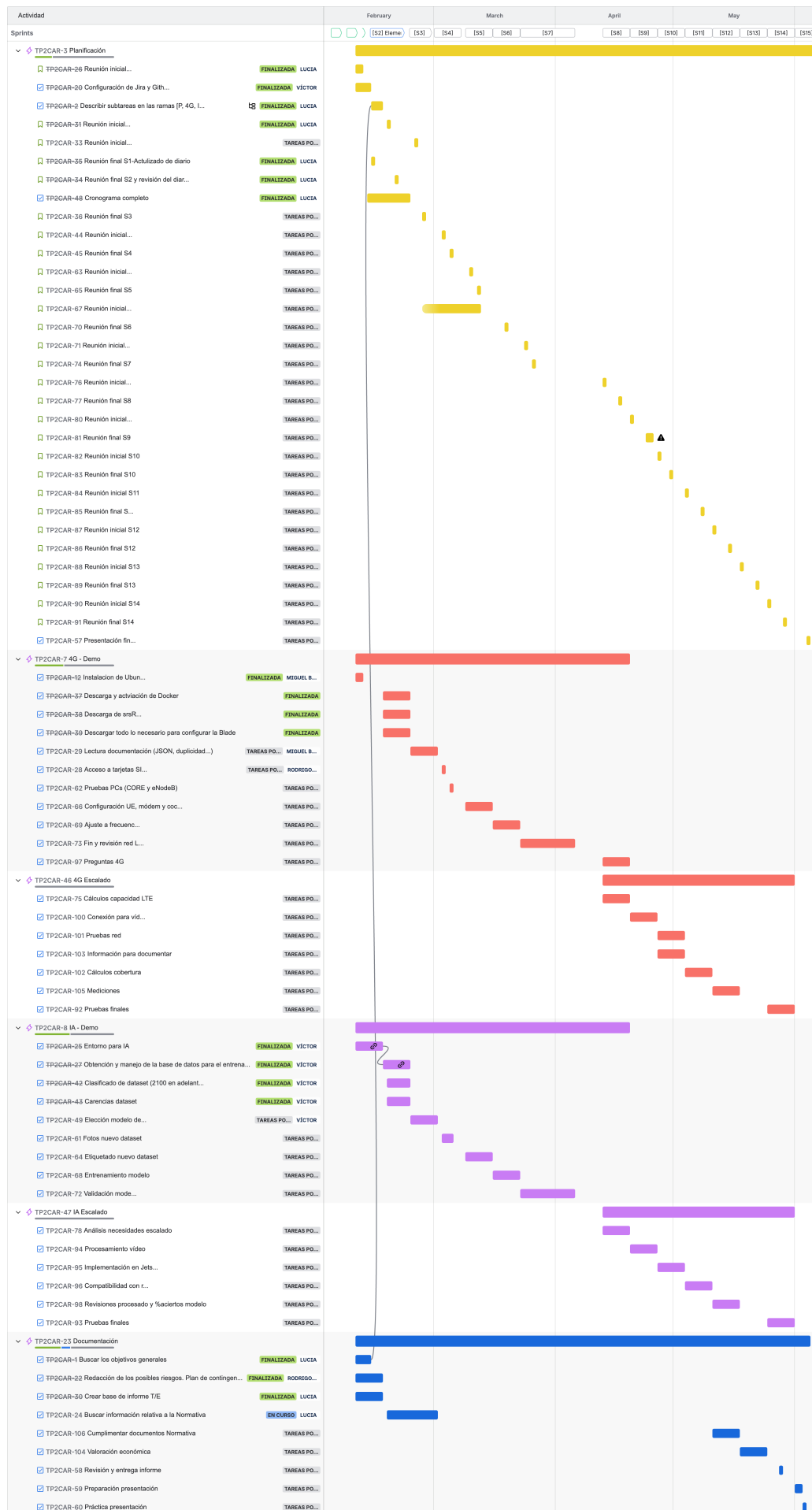


Figura 3.1: Diagrama de Gantt realizado al comienzo del proyecto como estimación

El cronograma final, generado de forma automática por Jira tras el cierre del último *sprint*, muestra una distribución del trabajo más concentrada en abril y mayo, fruto del incremento de horas de laboratorio en el segundo bimestre. En un primer vistazo, no parece haber grandes cambios organizativos, el esquema inicial parece mantenerse en el final. Sí es posible apreciar que algunas tareas se han alargado más de lo previsto, indicado con una barra horizontal más larga. En líneas generales, y a pesar de algún impedimento (también recogido en los posibles riesgos de la sección 3.2) ,se ha seguido con la planificación inicial y se ha logrado alcanzar el objetivo en los plazos marcados.

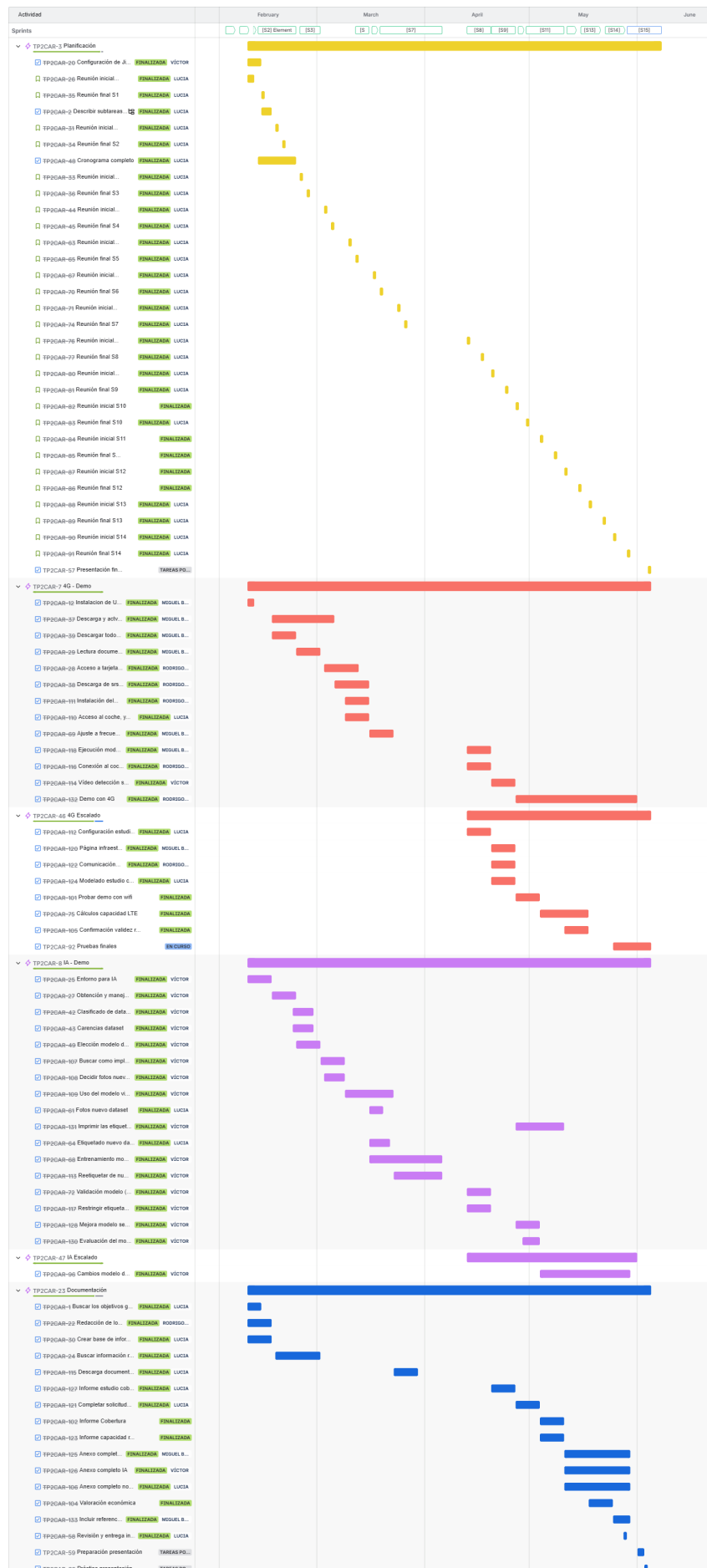


Figura 3.2: Diagrama de Gantt final generado en Jira. Distribución por epics: PLA, INF, IA, DOC

3.2. Análisis de riesgos

Antes del inicio del trabajo de laboratorio, y siguiendo las recomendaciones del seminario de planificación impartido por Renault, se elaboró una matriz de riesgos con su correspondiente plan de contingencia (Tabla 3.1).

Tabla 3.1: Matriz de riesgos identificados al inicio del proyecto.

Nivel	Riesgo	Plan de contingencia
ALTO	Daño en la SDR BladeRF por software	Bloquear actualizaciones de firmware; cargar bitstream solo en RAM (-1, no -L).
ALTO	Cuellos de botella en la retransmisión de vídeo	Priorizar UL en el <i>scheduler</i> de srsRAN; ajustar QoS y MCS mínimo.
ALTO	Fallos en el coche que haga que quede inutilizado	Uso de coches alternativos de compañeros; implementar protocolos de seguridad.
MEDIO	Incompatibilidades entre paquetes	Documentación detallada y compilación desde fuente cuando proceda.
MEDIO	Precisión IA inferior al 85 %	Ampliar <i>dataset</i> ; aplicar <i>augmentation</i> controlada; reentrenar.
MEDIO	Incompatibilidad Jetson-DeepRacer	Derivar parte del procesamiento al servidor a través de 4G.
MEDIO	Fallo en copias de los <i>dockers</i>	Mantener volcados <i>.tar.gz</i> en OneDrive y GitHub.
MEDIO	Falta de información de infraestructura	Búsqueda preventiva, asistencia a seminarios (Vodafone, Telefónica).
MEDIO	Saturación del espectro	Reparto del tiempo con los compañeros para realizar la demo.
MEDIO	Mala gestión del tiempo	Sesiones extra; reorientar tiempo de tareas opcionales hacia las críticas.

3.3. Estimación de horas trabajadas

El periodo de trabajo se ha dividido en dos bimestres. El primer bimestre comprende las semanas 1 a 7 (del 9 de febrero al 29 de marzo), durante las cuales cada miembro dedicó 4 h en laboratorio más una media de 2 h adicionales semanales. Esto suma:

$$(4 + 2) \times 7 = 42 \text{ horas/persona en el primer bimestre.}$$

El segundo bimestre se extiende de la semana 8 a la 13 (13 de abril a 22 de mayo), con un incremento sustancial de la dedicación: 12 h en laboratorio más una media de 6 h adicionales. Las dos últimas semanas (14 y 15) se reservaron para finalización, redacción de la memoria y preparación de la presentación, con dedicaciones de 12 h y 6 h adicionales en casa respectivamente.

$$(12 + 6) \times 6 + 24 + 18 = 150 \text{ horas/persona en el segundo bimestre.}$$

El total por miembro asciende a **192 horas**, lo que para un equipo de cuatro personas supone:

$$192 \times 4 = 728 \text{ horas.}$$

Suponiendo el salario medio de un ingeniero de telecomunicaciones recién titulado en España de 15,38 €/h¹, el coste humano del proyecto asciende a:

$$728 \times 13,85 \approx 10\,082 \text{ €}.$$

¹Referencia: <https://es.talent.com/salary?job=Ingeniero+telecomunicaciones>.

4 Arquitectura de 4G

El presente capítulo establece el marco conceptual sobre el que se refleja el resto del informe, partiendo de los principios de organización que comparten todos los sistemas de red móvil y descendiendo hasta los mecanismos concretos que distinguen a LTE de generaciones anteriores. El apartado se articula en cuatro bloques: primero se sitúa el papel del organismo 3GPP como agente normalizador; a continuación se describe el patrón arquitectónico común a cualquier red móvil; en tercer lugar se particulariza dicho patrón al caso de LTE, detallando E-UTRAN, EPC y las interfaces que los vertebran; y, por último, se examinan los mecanismos de la capa física que sustentan el enlace radio.

4.1. El organismo 3GPP y su modelo de *Releases*

Las redes móviles modernas no son producto del trabajo aislado de un fabricante, sino el resultado de un proceso de estandarización colaborativo coordinado por el *Third Generation Partnership Project* (3GPP), una iniciativa nacida en 1998 con el propósito original de definir una red móvil de tercera generación que fuese una evolución respecto a GSM. Lo que comenzó como un consorcio orientado a 3G se ha mantenido hasta nuestros días como el principal foro técnico donde se especifican las generaciones 4G y 5G, integrando a operadores, fabricantes de equipos y organismos nacionales de normalización.

El alcance de las especificaciones publicadas por 3GPP es amplio. Cubre tanto los aspectos arquitectónicos (definición de los elementos funcionales, de las interfaces lógicas que los unen y de las conexiones físicas que las materializan) como los del acceso radio propiamente dicho, incluyendo formas de onda, esquemas de modulación, codificación de canal y procedimientos de acceso al medio. A esto se suma la normalización de la pila de protocolos en sus dos planos (control y usuario) y la definición de los servicios soportados, con sus correspondientes garantías de calidad (QoS).

Los documentos resultantes se publican agrupados en *Releases* numeradas, cada una de las cuales cierra un conjunto coherente de especificaciones disponibles para implementación. Tres de estas *Releases* resultan especialmente relevantes para situar la tecnología LTE en su contexto:

- La *Release* 8, cerrada entre 2008 y 2009, contiene la primera definición formal del sistema LTE.
- La *Release* 10, finalizada en 2011, introduce LTE-Advanced y aporta las extensiones que permiten alcanzar los requisitos IMT-Advanced de la ITU.
- Posteriormente, la *Release* 15 (2018) marca la transición hacia 5G NR (*New Radio*).

De forma particular, este proyecto se apoya en la TS 36.101 perteneciente a la *Release* 10 [3GPP, 2014b] (concretamente se usará LTE de categoría 6), cuya tabla 7.3.1-1 fija dos parámetros que se reutilizarán en todo el dimensionado posterior: una sensibilidad de referencia (REFSENS) de -92 dBm para un canal de hasta 20 MHz en banda 7, y una potencia de transmisión media del UE de 23 dBm. Estos valores son los que se introducirán en la simulación de cobertura del Capítulo 5.

4.2. El patrón arquitectónico común de los sistemas celulares

A pesar de las diferencias notables entre generaciones tecnológicas, todos los sistemas celulares responden a un mismo esquema arquitectónico, organizado en tres dominios funcionales [Sesia et al., 2011]. Esta división refleja una decisión de diseño que persigue desacoplar las funciones de

radio de las funciones de gestión, de modo que la evolución de la interfaz aérea pueda producirse sin reescribir el núcleo de red, y viceversa.

El primer dominio lo conforma el equipo de usuario, que es el dispositivo a través del cual el suscriptor accede a los servicios. El segundo es la red de acceso, encargada de proporcionar el canal radio entre el terminal y la infraestructura fija. El tercer dominio, la red troncal, asume las funciones de mayor nivel: autenticación, control de movilidad, gestión del enrutamiento de datos y conexión con redes externas. Esta modularidad cumple dos funciones prácticas: por un lado, permite que cada bloque evolucione siguiendo su propio ciclo de innovación; por otro, habilita escenarios mixtos en los que distintas tecnologías de acceso conviven sobre una misma red troncal, o donde una única red de acceso da servicio a varios núcleos en paralelo.

4.3. Particularización al ecosistema 3GPP

Acorde a 3GPP [3GPP, 2014a, 2014d], los tres dominios mencionados reciben las siguientes denominaciones: el equipo de usuario se denomina *User Equipment* (UE), la red de acceso se denomina *Access Network* (AN) y la red troncal recibe el nombre de *Core Network* (CN). A continuación, se examina cada uno de ellos.

4.3.1. El equipo de usuario

En la nomenclatura 3GPP, el UE es la entidad que materializa el lado de cliente del sistema. Internamente, consta de dos componentes claramente diferenciados que cumplen funciones complementarias.

El primero, el terminal móvil o *Mobile Equipment* (ME), agrupa todo el hardware del dispositivo: procesador de banda base, transceptor, antenas, batería, pantalla y demás periféricos. Dentro del ME, las especificaciones distinguen a su vez entre la terminación móvil (MT), responsable de la comunicación con la red, y el equipo terminal (TE), que aloja las aplicaciones y la interfaz con el usuario.

El segundo componente es la *Universal Integrated Circuit Card* (UICC), una tarjeta que actúa como soporte de la identidad del suscriptor en la red. La UICC aloja una o varias aplicaciones de identidad según la tecnología de la red sobre la que se opere: SIM (*Subscriber Identity Module*) en GSM y USIM (*Universal SIM*) en UMTS y LTE. Esta tarjeta es portadora de información crítica para los procedimientos de seguridad: el identificador internacional del suscriptor (IMSI), la clave secreta compartida con el HSS (Ki), las funciones criptográficas MILENAGE que derivan las claves de sesión y, en el caso particular de la USIM, los mecanismos que permiten al UE autenticar a su vez a la red, dotando al sistema de autenticación mutua y bloqueando así los ataques basados en celdas falsas (*man-in-the-middle*).

La separación entre terminal y tarjeta produce que la identidad del suscriptor resida en la UICC, no en el dispositivo, por lo que un mismo usuario puede emplear distintos terminales sin perder su perfil de servicio. Es precisamente este desacople el que permite a las operadoras comercializar de forma independiente el hardware y la suscripción.

Para la desarrollada en este proyecto, el rol de UE se materializa mediante una combinación de tres elementos: un vehículo AWS DeepRacer que aporta el equipo terminal y la capacidad de procesamiento, un módem Huawei E3372h-320 USB Stick que proporciona la terminación móvil y la interfaz radio del lado del usuario, y una tarjeta SIM programable configurada con un IMSI y una clave Ki conocidos por el HSS desplegado en el núcleo de red. El transceptor radio del lado de la infraestructura es proporcionado por el dispositivo bladeRF 2.0 micro conectado al eNodeB.

4.3.2. La red de acceso

La red de acceso es el segmento que físicamente sostiene la conexión radio y que media entre el UE y la red troncal mediante los protocolos correspondientes.

A lo largo de las distintas generaciones, 3GPP ha definido tres redes de acceso sucesivas, que se distinguen sobre todo por la técnica de acceso múltiple empleada. La GERAN (*GSM/EDGE Radio Access Network*, 2.5G) combina TDMA con FDMA. La UTRAN (*UMTS Terrestrial RAN*, 3G) añade a esos mecanismos el WCDMA, que permite a múltiples usuarios compartir la misma frecuencia mediante códigos ortogonales. Finalmente, E-UTRAN (*Evolved UTRAN*, 4G) introduce un cambio de paradigma al adoptar OFDMA en el enlace descendente y SC-FDMA en el ascendente [3GPP, 2014c]. La adopción de SC-FDMA en uplink, frente a la simetría OFDMA/OFDMA que hubiera sido natural, responde a una preocupación práctica: el amplificador de potencia del terminal. Las formas de onda OFDMA presentan picos de potencia muy superiores a su valor medio (relación PAPR alta), lo que obligaría al amplificador del UE a operar lejos del punto de saturación, con la consiguiente pérdida de eficiencia energética y reducción de la autonomía. SC-FDMA reduce drásticamente esa PAPR, permitiendo al amplificador trabajar más cerca de su punto óptimo [Myung et al., 2006].

La conexión entre la red de acceso y la troncal se realiza siempre a través de una interfaz específica de cada generación. En el caso particular de E-UTRAN, a través de la interfaz S1, que se detallará más adelante.

4.3.3. La red troncal

La red troncal asume las funciones que requieren persistencia, conocimiento global del estado de la red y conexión hacia el exterior: autenticación, control de acceso, gestión de la movilidad, mantenimiento de las sesiones de datos, enrutamiento, conexión con redes ajenas (típicamente Internet) y monitorización del estado del sistema.

Históricamente, la red troncal se ha articulado en torno a tres dominios diferenciados según el tipo de servicio que sustentan. El dominio de circuitos (CS) ofrece servicios mediante conmutación de circuitos (esencialmente voz tradicional) y solo está presente en 2G y 3G; LTE prescinde de él por completo, delegando la voz en una solución sobre IP basada en VoLTE e IMS. El dominio de paquetes (PS), por su parte, ofrece conectividad mediante conmutación de paquetes y es el dominio que concentra todo el tráfico en LTE. Finalmente, el subsistema IP Multimedia (IMS) actúa como capa de control para los servicios multimedia sobre IP (voz, vídeo, mensajería), aportando la señalización SIP/Diameter y las funciones de tarificación que requieren estos servicios.

4.4. Terminología: LTE, EPS y 4G

Antes de entrar en la arquitectura concreta del sistema, conviene aclarar un detalle terminológico que suele resultar fuente de confusión. En las especificaciones oficiales de 3GPP, la denominación formal del sistema móvil de cuarta generación es EPS (*Evolved Packet System*), mientras que el acrónimo LTE (*Long Term Evolution*) hace referencia, en sentido estricto, únicamente a la evolución de la interfaz aérea. En la práctica, sin embargo, el término LTE ha terminado por designar al sistema completo, y la propia 3GPP lo emplea con frecuencia en ese sentido extendido. En este informe se utilizarán ambos términos de manera intercambiable.

La cuestión se complica cuando entra en juego la nomenclatura «4G». Este término procede originalmente de una iniciativa de la ITU (*International Telecommunication Union*) que en 2008 publicó los requisitos IMT-Advanced para los sistemas de cuarta generación. La primera versión de LTE no satisfacía estrictamente dichos requisitos (sí lo hace LTE-Advanced, introducido en la *Release 10*), pero la presión comercial y la evidente discontinuidad técnica con UMTS llevaron a la ITU a admitir LTE como tecnología 4G *de facto*. Hoy ambos términos se usan indistintamente,

con la advertencia conceptual de que uno describe un conjunto de requisitos y el otro un sistema concreto que los cumple (en su versión Advanced) o se aproxima a ellos (en su versión inicial).

4.5. La arquitectura de LTE

Bajo su denominación formal como EPS, LTE está integrado por tres grandes componentes: la red de acceso (E-UTRAN), el núcleo de red (EPC, *Evolved Packet Core*) y la evolución del subsistema IMS heredado del entorno UMTS. La interconexión entre los equipos físicos donde residen las funciones de E-UTRAN y EPC se apoya en su totalidad sobre tecnologías IP, complementadas con las interfaces lógicas que el propio estándar define. Esto implica que la red de transporte de una infraestructura LTE es, técnicamente, una red IP convencional, y que el conjunto de la infraestructura comprende, además de los elementos específicos del estándar 3GPP, todos los componentes habituales de una red IP: routers, servidores DHCP, servidores DNS y demás.

4.5.1. La red de acceso: E-UTRAN

Si hay un rasgo que distingue a E-UTRAN de las redes de acceso anteriores es su simplicidad estructural. Donde GSM y UMTS distribuían las funciones de control entre dos elementos (estación base y controlador: BTS/BSC y NodoB/RNC, respectivamente), E-UTRAN reduce todo el dominio radio a un único tipo de equipo: el *evolved NodeB* (eNB) [3GPP, 2014d; Sesia et al., 2011]. El eNB asume tanto la transmisión y recepción en la interfaz radio como el control de bajo nivel de los terminales conectados a su celda.

Esta concentración de funciones tiene cuatro consecuencias directas. En primer lugar, reduce la latencia, al eliminar el salto que en arquitecturas previas suponía el tránsito por un controlador externo. En segundo lugar, simplifica la operación y el despliegue, ya que la red de acceso queda formada por un solo tipo de elemento radio. En tercer lugar, incrementa la complejidad y el coste por unidad de eNB, dado que cada estación base debe incorporar internamente las funciones de control que antes residían en un nodo centralizado. Y en cuarto lugar, obliga a que los eNBs vecinos puedan comunicarse directamente entre sí para coordinar las decisiones de *handover*, requisito que se traduce en la aparición de la interfaz X2.

La E-UTRAN se conecta con el resto del sistema a través de tres interfaces lógicas, cada una con un cometido bien delimitado. La interfaz Uu, en ocasiones denominada «interfaz aérea» o LTE Uu, es la única estrictamente radio del sistema y transporta toda la información intercambiada entre el eNB y los terminales que tiene a su cargo; todas las demás interfaces son lógicas y discurren sobre IP. La interfaz S1, que conecta el eNB con el EPC [3GPP, 2014e], se subdivide a su vez en dos planos: S1-MME (también designada como S1-CP), que enlaza el eNB con la entidad MME para transportar señalización NAS y S1AP, y S1-U, que enlaza el eNB con el S-GW y transporta los paquetes de usuario encapsulados sobre GTP-U. La separación entre estos dos planos no es un detalle de imagen: permite que el plano de control y el plano de usuario terminen en entidades distintas del EPC, y que cada uno de ellos escale de forma independiente en función de la carga. Finalmente, la interfaz X2 conecta directamente eNBs vecinos y es opcional; cuando está presente, se utiliza para intercambiar mensajes de señalización y para reenviar los paquetes de usuario durante un *handover*, minimizando la pérdida de datos al cambiar de celda.

Estas interfaces no se materializan necesariamente como conexiones físicas dedicadas. Lo habitual es que el tráfico fluya enrutado sobre una red IP, lo que implica que cada elemento (eNB, MME, S-GW, etc.) disponga de su propia dirección IP. Las interfaces deben entenderse, en consecuencia, como relaciones lógicas a través de las cuales los nodos se reconocen mutuamente e intercambian información, no como cables específicos del estándar.

4.5.2. El núcleo de red: EPC

El EPC es el resultado de un rediseño de la red troncal centrado en ofrecer conectividad IP de forma eficiente y aprovechar al máximo las capacidades que E-UTRAN aporta en el lado radio [3GPP, 2014a]. Su arquitectura agrupa entidades que sustentan la conectividad IP, junto a otras dedicadas al control de la sesión y a la tarificación.

Tres entidades constituyen el núcleo funcional del EPC. La MME (*Mobility Management Entity*) es la responsable del plano de control y gestiona la totalidad del ciclo de vida del UE en la red: el registro de los terminales al conectarse, la autenticación contra el HSS, el establecimiento y mantenimiento de los *bearers* EPS, los procedimientos de movilidad (*handover* y paginación), así como la selección concreta del S-GW y del P-GW que servirán a cada UE. El S-GW (*Serving Gateway*), por su parte, actúa como punto de anclaje del plano de usuario en LTE: se encarga de reenviar los paquetes IP entre el eNB y el P-GW y, lo que resulta más importante, funciona como anclaje de movilidad tanto en los *handovers* entre eNBs como en las transiciones entre 3GPP y otras tecnologías. Por último, el P-GW (*Packet Data Network Gateway*) constituye la pasarela hacia las redes externas (Internet, IMS, redes corporativas) y concentra varias funciones críticas: asignación de direcciones IP a los UE (típicamente vía NAT), aplicación de las políticas de QoS y de filtrado de tráfico, e implementación de las funciones de tarificación basadas en consumo.

A estas tres entidades se suma el HSS (*Home Subscriber Server*), que actúa como base de datos central del sistema. En el HSS residen los perfiles de los suscriptores: IMSI, claves de autenticación (Ki y OPc), perfiles de QoS, APN suscritos, restricciones de *roaming* y demás atributos. Durante el procedimiento de *attach*, la MME consulta al HSS para validar la identidad del UE que solicita conectarse.

Esta combinación de cuatro entidades (MME, S-GW, P-GW y HSS) basta para sustentar las funciones esenciales del sistema: el registro y autenticación del usuario, el establecimiento de sesiones y la provisión de conectividad IP entre los UE y las redes externas. Conviene observar la distribución de roles entre planos: las funciones del plano de usuario se concentran en las pasarelas (S-GW y P-GW), mientras que la MME monopoliza el plano de control y la señalización asociada.

Junto a estas entidades básicas, el EPC contempla dos elementos adicionales de carácter opcional. El PCRF (*Policy and Charging Rules Function*) gestiona las políticas de QoS y las reglas de tarificación aplicadas al tráfico de datos, habilitando esquemas de cobro avanzados (por servicio, por volumen, por tiempo, etc.). El ANDSF (*Access Network Discovery and Selection Function*) ayuda al UE a descubrir redes no-3GPP en su entorno, típicamente Wi-Fi, y a decidir cuándo conviene cambiar entre estas y la red 3GPP.

4.5.3. Interfaces internas del EPC

La interconexión entre E-UTRAN y EPC se canaliza a través de la interfaz S1 ya descrita [3GPP, 2014a]. En concreto, el plano de control (S1-MME) termina en la MME y el plano de usuario (S1-U) en el S-GW. El resto de interfaces relevantes del EPC se sintetizan en la Tabla 4.1.

Tabla 4.1: Principales interfaces del EPC y elementos que conectan.

Interfaz	Elementos	Función
S1-MME	eNB ↔ MME	Plano de control: NAS y S1AP.
S1-U	eNB ↔ S-GW	Plano de usuario: GTP-U.
S5	S-GW ↔ P-GW	Plano de usuario, mismo operador.
S8	S-GW ↔ P-GW	Plano de usuario, <i>roaming</i> .
S6a	MME ↔ HSS	Autenticación y gestión del perfil.
S10	MME ↔ MME	Movilidad entre MMEs.
S11	MME ↔ S-GW	Control de <i>bearers</i> .
SGi	P-GW ↔ PDN	Salida hacia redes externas (Internet).
Gx	P-GW ↔ PCRF	Reglas de QoS y tarificación.

Para comprender el flujo real de tráfico y la señalización, conviene examinar la función específica de cada interfaz interna. La MME canaliza el plano de control de los UE conectados mediante los protocolos NAS (*Non-Access Stratum*) y, a través de la interfaz S11, comanda al S-GW las operaciones que afectan al plano de usuario. La conexión de la MME con el HSS, a través de S6a, es la que sustenta la autenticación y el acceso al perfil del suscriptor. Cuando un terminal se desplaza entre áreas de seguimiento (TA) gestionadas por distintas MMEs, estas se coordinan a través de la interfaz S10.

Por el lado del plano de usuario, el P-GW es el responsable de la conexión con redes externas o con plataformas de servicio como IMS, conexión que se materializa a través de la interfaz SGi. Es también el P-GW el encargado de asignar direcciones IP públicas a los UE (típicamente mediante NAT) y de aplicar los parámetros de calidad de servicio acordados sobre cada sesión de datos.

En la demostración del laboratorio, la implementación `srsepc` de srsRAN [Gomez-Miguel et al., 2016; Software Radio Systems, 2026] integra MME, S-GW, P-GW y HSS dentro de un único proceso. Las interfaces internas (S5, S6a, S11) no se materializan como tráfico real sobre IP, sino que se reducen a llamadas a función dentro de ese mismo proceso. La única interfaz que existe físicamente es la S1, que conecta `srsepc` con el contenedor del eNodeB que ejecuta `srsenb` en una máquina física distinta.

4.6. La interfaz radio de LTE

Llegados a este punto, conviene descender al nivel de la interfaz radio propiamente dicha. Esta interfaz, designada formalmente como Uu, es el enlace entre el UE y el eNB y constituye el corazón del sistema móvil [3GPP, 2014c]. La pila de protocolos radio sigue el modelo OSI y se ha diseñado para soportar una amplia variedad de servicios sin sacrificar la sencillez conceptual.

4.6.1. Capa física: técnicas de acceso múltiple y modulación

La capa física es la responsable última de mover bits a través del canal radio. Para gestionar el acceso múltiple, LTE combina dos esquemas asimétricos: OFDMA en el enlace descendente y SC-FDMA en el ascendente.

El OFDMA (*Orthogonal Frequency Division Multiple Access*) divide el espectro disponible en un conjunto de subportadoras ortogonales que el *scheduler* del eNB asigna dinámicamente a los usuarios en función de las condiciones instantáneas del canal. Esto permite que distintos usuarios transmitan simultáneamente sobre subportadoras distintas con eficiencia espectral elevada y aprovechando la diversidad multiusuario y la diversidad frecuencial. Adicionalmente, la estructura OFDM aporta robustez frente a los efectos del multitrayecto, gracias al prefijo cíclico, y permite una gestión del ancho de banda asignado.

El SC-FDMA (*Single-Carrier Frequency Division Multiple Access*) se reserva para el enlace ascendente y combina los beneficios de la transmisión de portadora única con las ventajas de la multiplexación por división de frecuencia. La razón de esta elección, ya apuntada, es minimizar la PAPR (*Peak-to-Average Power Ratio*) en la transmisión del terminal: una PAPR baja se traduce en un amplificador de potencia más eficiente y, por tanto, en mayor autonomía del dispositivo, sin sacrificar significativamente las prestaciones del enlace.

En cuanto al espectro, la capa física de LTE está diseñada para operar en las bandas altas de UHF, con un margen que se extiende desde los 450 MHz hasta los 3,5 GHz. El estándar define hasta 40 bandas de operación posibles que pueden emplearse en duplexación por división de frecuencia (FDD) o por división de tiempo (TDD). Este proyecto opera en la **banda 7** en modo FDD, con uplink en 2500 MHz a 2570 MHz y downlink en 2620 MHz a 2690 MHz.

Sobre la portadora se transportan datos modulados según uno de los esquemas previstos por el estándar. En el enlace descendente, las modulaciones admisibles son QPSK, 16-QAM y 64-

QAM, con 256-QAM disponible a partir de LTE-Advanced. En el enlace ascendente, el conjunto es más restrictivo: QPSK y 16-QAM como modulaciones obligatorias, y 64-QAM únicamente para las categorías de UE más avanzadas. Las eficiencias espectrales nominales son las esperables: 2 bits/símbolo para QPSK, 4 bits/símbolo para 16-QAM y 6 bits/símbolo para 64-QAM [Goldsmith, 2005; Sesia et al., 2011]. La eficiencia real, sin embargo, queda sensiblemente por debajo de estos valores una vez se descuentan los *overheads* de codificación de canal y la información de control: típicamente en torno al 70 % del valor nominal. Es precisamente esta cifra (16-QAM como esquema mínimo garantizado, con 4 bps/Hz nominales y un factor de utilización efectiva del 70 %) la que servirá de base para los cálculos de capacidad del Capítulo 5.

4.6.2. Estructura temporal de la trama LTE

El recurso radio en LTE se organiza no solo en frecuencia, sino también en el tiempo, siguiendo una jerarquía estrictamente definida [3GPP, 2014c]. La unidad temporal de mayor nivel es la trama radio, de 10 ms de duración. Cada trama se subdivide en 10 subtramas idénticas de 1 ms, y a su vez cada subtrama se compone de 2 *slots* de 0,5 ms. La cantidad de símbolos OFDM que caben en un *slot* depende de la longitud del prefijo cíclico empleado: 7 símbolos con prefijo cíclico normal y 6 con prefijo extendido (este último reservado a escenarios con multitrayecto severo, como entornos celulares de cobertura amplia).

4.6.3. Estructura temporal de la trama LTE

El recurso radio en LTE se organiza no solo en frecuencia, sino también en el tiempo, siguiendo una jerarquía estrictamente definida [3GPP, 2014c]. La unidad temporal de mayor nivel es la trama radio, de 10 ms de duración. Cada trama se subdivide en 10 subtramas idénticas de 1 ms, y a su vez cada subtrama se compone de 2 *slots* de 0,5 ms. La cantidad de símbolos OFDM que caben en un *slot* depende de la longitud del prefijo cíclico empleado: 7 símbolos con prefijo cíclico normal y 6 con prefijo extendido (este último reservado a escenarios con multitrayecto severo, como entornos celulares de cobertura amplia).

La Figura 4.1 ilustra la rejilla de recursos resultante para una subtrama con prefijo cíclico normal. El eje vertical representa el tiempo (14 símbolos OFDM repartidos en 2 *slots* de 0,5 ms), mientras que el eje horizontal representa la frecuencia (12 subportadoras separadas 15 kHz, que totalizan los 180 kHz de un *Resource Block*).

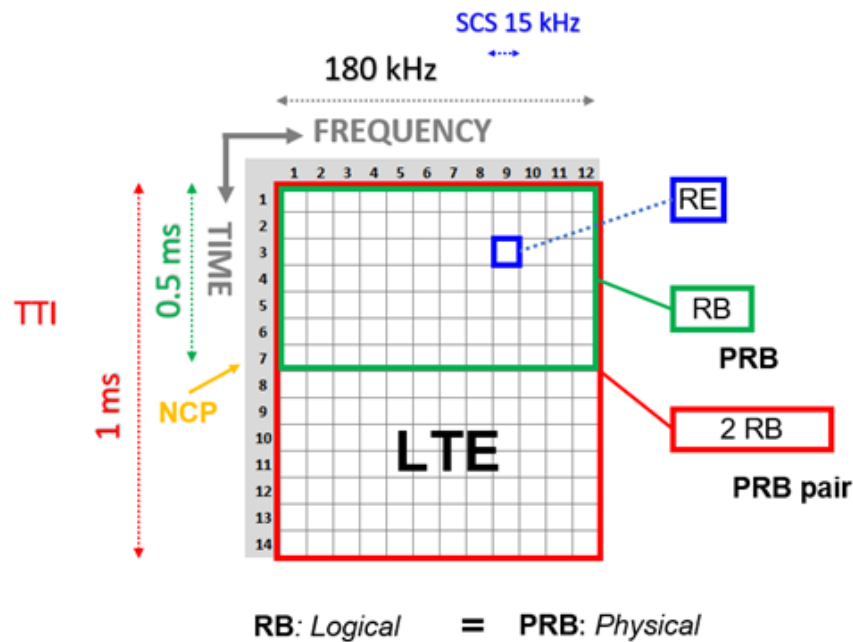


Figura 4.1: Rejilla de recursos físicos de una subtrama LTE con prefijo cíclico normal. Cada celda es un *Resource Element* (RE); un bloque de 12 subportadoras $\times$ 1 *slot* constituye un *Resource Block* (RB), y el par de RBs que abarca la subtrama completa forma un *PRB pair*, que es la unidad efectiva de asignación del *scheduler*. Adaptado de [TelecomHall Forum, 2021].

Sobre este eje temporal se superpone la asignación de frecuencias. La unidad mínima que el *scheduler* puede asignar a un usuario es el **Resource Block** (RB) [3GPP, 2014b, 2014c], formado por 12 subportadoras consecutivas (lo que totaliza 180 kHz) durante un *slot*. El número de RBs disponibles depende, naturalmente, del ancho de banda del canal: 50 RBs para un canal de 10 MHz y 100 RBs para uno de 20 MHz. Esta granularidad fina (en tiempo y en frecuencia) es la que permite al eNB ajustar la asignación de recursos vehículo a vehículo y subtrama a subtrama, adaptando la entrega de servicio a la calidad de canal instantánea y a la demanda real de cada UE.

4.6.4. MIMO

Junto al acceso múltiple y la asignación de recursos, una pieza clave de la interfaz radio de LTE es el uso de múltiples antenas tanto en la transmisión como en la recepción, es decir, MIMO (*Multiple-Input Multiple-Output*) [Foschini y Gans, 1998; Sesia et al., 2011]. Esta técnica representa, probablemente, el salto cualitativo más significativo respecto a las generaciones móviles anteriores.

El principio operativo es el siguiente: los datos se dividen en flujos paralelos que se modulan y se transmiten de forma simultánea por distintas antenas, todos ellos sobre la misma frecuencia. Las múltiples copias de cada flujo se combinan en el canal radio (a través de las reflexiones por multitrayecto, que en este contexto dejan de ser un problema y se convierten en un recurso aprovechable) y son separadas en el receptor mediante algoritmos que estiman los canales individuales entre cada pareja antena-Tx/antena-Rx. El resultado es un incremento de capacidad que no requiere espectro adicional: la mejora se obtiene únicamente añadiendo antenas.

La especificación inicial de LTE soporta MIMO 2×2 (dos antenas en transmisión y dos en recepción), mientras que LTE-Advanced extiende esa capacidad hasta MIMO 8×8 . El estudio teórico de capacidad se ha diseñado bajo el supuesto de MIMO 2×2 . Aunque las capacidades del frontal radio empleado (la BladeRF 2.0 micro xA9, que dispone de 2 cadenas de transmisión y 2 de recepción) lo permiten, dada la restricción de antenas que tenemos en el laboratorio, esto no

se ha usado .

4.6.5. Adaptación al canal: CQI, MCS y TBS

El último engranaje que conviene examinar es el mecanismo por el cual el sistema adapta la transmisión a las condiciones cambiantes del canal radio [Sesia et al., 2011]. Esa adaptación se sustenta en una cadena de tres parámetros relacionados.

En primer lugar, el UE estima periódicamente la calidad del canal que recibe y comunica esta información al eNB mediante un indicador denominado CQI (*Channel Quality Indicator*). Este indicador toma valores discretos entre 0 (sin servicio) y 15 (canal excelente), recogiendo el estado instantáneo del enlace en un único parámetro. A partir del CQI reportado, el eNB selecciona el MCS (*Modulation and Coding Scheme*) que considera más adecuado para esas condiciones; este esquema fija conjuntamente la modulación (QPSK, 16-QAM o 64-QAM) y la tasa de codificación de canal. Finalmente, combinando el MCS elegido con el número de RBs asignados al UE, se determina el TBS (*Transport Block Size*), es decir, el número exacto de bits que efectivamente se transportarán en cada subtrama.

Esta cadena $\text{CQI} \rightarrow \text{MCS} \rightarrow \text{TBS}$ es la base de la adaptación de enlace en LTE y constituye la clave para mantener un BLER (*Block Error Rate*) objetivo (10 % en los canales de datos y 1 % en los canales de control) incluso cuando las condiciones del canal varían rápidamente. Su comportamiento se observará en la práctica en el Capítulo 6, donde se muestra cómo el CQI fluctúa con la posición del DeepRacer dentro del laboratorio y cómo esa fluctuación se traduce, casi linealmente, en variaciones del MCS y, por tanto, del número de *frames* de vídeo entregados por segundo.

4.7. Recapitulación

LTE es un sistema móvil definido por 3GPP cuya arquitectura se articula en dos grandes bloques: una red de acceso simplificada (E-UTRAN, integrada únicamente por eNBs) y un núcleo de red orientado a paquetes (EPC, formado por MME, S-GW, P-GW y HSS, con PCRF y ANDSF como elementos opcionales). La interfaz radio combina OFDMA en downlink y SC-FDMA en uplink, soporta MIMO 2×2 y permite hasta 64-QAM. La unidad mínima de asignación en el dominio tiempo-frecuencia es el Resource Block (180 kHz durante 0,5 ms), y la adaptación al canal se materializa mediante la cadena $\text{CQI} \rightarrow \text{MCS} \rightarrow \text{TBS}$ reportada por el UE al eNB.

Sobre esta base teórica, el siguiente capítulo aborda el diseño concreto de una red 4G dimensionada para el servicio de vehículo conectado en el tramo Valladolid-Segovia y describe en detalle el despliegue de la demostración funcional desarrollada en el laboratorio del equipo G2TP2. La demostración aquí empleada tiene su base en el trabajo de investigación de Ignacio Royuela González para la elaboración de su TFM [González, 2022].

5 Infraestructura de red 4G

Una vez consolidados los objetivos del proyecto y los fundamentos teóricos de la arquitectura LTE, es crucial contrastar el diseño conceptual con las realidades operativas, técnicas y comerciales de su futura implementación en campo. Este análisis DAFO (Debilidades, Amenazas, Fortalezas y Oportunidades) surge de la necesidad de evaluar la viabilidad del despliegue de la infraestructura 4G orientada al vehículo conectado, extrayendo las conclusiones obtenidas en el laboratorio para proyectarlas sobre el escenario real de la autovía A-601. El valor fundamental de este diagnóstico radica en su capacidad para equilibrar las ventajas competitivas de la solución; como el uso de *software* de código abierto, la soberanía tecnológica y las mínimas latencias del procesamiento en el borde, frente a los retos críticos de capitalización, la exigencia de personal altamente especializado y la convivencia con tecnologías emergentes. Con ello, este apartado abandona el marco puramente teórico para consolidar una herramienta de toma de decisiones que valide la sostenibilidad, seguridad y replicabilidad del servicio propuesto.

Debilidades (aspectos negativos internos)

- La inversión inicial estimada (12 M€ de CAPEX y 850 K€ de OPEX anual) condiciona el modelo de negocio a la existencia de un socio financiero público o público-privado.
- La elección de un *stack open source* aporta independencia, pero exige un equipo técnico con experiencia específica en LTE para la operación y el mantenimiento; no es una solución *plug-and-play*.
- El sistema opera sobre 4G y no 5G, lo que puede percibirse como una desventaja comercial frente a propuestas competidoras orientadas a *5G-ready*.
- La carga estimada por vehículo (2 Mbit/s sobre H.264) es un valor medio: los picos en escenas dinámicas pueden saturar la celda sin un control de tasa adaptativo activo.
- Existen casos límite del modelo de IA (señales de límite de velocidad muy similares, condiciones meteorológicas adversas) que requieren refinamiento adicional, típicamente mediante una segunda etapa de entrenamiento y validación.
- La validación funcional se ha realizado a escala (AWS DeepRacer), por lo que la extrapolación al comportamiento sobre vehículo real conserva cierta incertidumbre.

Amenazas (aspectos negativos externos)

- El despliegue acelerado del 5G en stand-alone por parte de los operadores comerciales podría cubrir parte del trazado en un horizonte de tres a cinco años, erosionando el argumento de necesidad de infraestructura dedicada.
- La armonización del espectro a nivel europeo podría alterar la disponibilidad de la banda 7 en el medio plazo, obligando a una migración no planificada.
- Las redes privadas constituyen un blanco atractivo para ciberataques de tipo OT/IT, lo que exige un plan de ciberseguridad robusto y una inversión adicional no trivial.
- La competencia con fabricantes consolidados (Ericsson, Nokia, Huawei) que ofrecen soluciones llave en mano respaldadas por contratos de mantenimiento corporativos puede dificultar la adjudicación.
- Los cambios políticos en la administración autonómica podrían condicionar la continuidad presupuestaria del programa.

- La aceptación social del despliegue de nuevas torres en zona de pinar puede generar oposición vecinal y retrasos en los trámites urbanísticos.
- La adopción de vehículos con V2X nativo es lenta, lo que reduce el tamaño del mercado direccionable durante los primeros ejercicios de operación.

Fortalezas (aspectos positivos internos)

- Arquitectura validada en laboratorio con *hardware* y *software* reales (srsRAN, BladeRF), no únicamente sobre simulación. El testbed permite trasladar las mediciones de capacidad y latencia directamente al despliegue de campo.
- La plataforma se apoya íntegramente sobre tecnología *open source*, lo que elimina costes recurrentes de licencia y proporciona soberanía técnica frente a fabricantes propietarios.
- El procesamiento MEC al pie de cada estación garantiza una latencia extremo a extremo inferior a 500 ms, lo que habilita el caso de uso de asistencia a la conducción.
- El diseño cumple con holgura el marco regulatorio español (Ley 11/2022, RD 123/2017 y RD 1066/2001) y se sustenta sobre estándares 3GPP *Release* 10.
- El *site sharing* sobre torres ya existentes de Vodafone y Telefónica reduce sustancialmente la inversión inicial frente a un despliegue íntegramente de nueva planta.

Oportunidades (aspectos positivos externos)

- El Reglamento (UE) 2019/2144 obliga a incorporar sistemas ADAS en los vehículos nuevos desde julio de 2024, generando una demanda creciente de infraestructura conectada que sustente esos sistemas más allá del propio coche.
- La estrategia nacional de reducción de la siniestralidad en vías interurbanas (1 119 fallecidos en 2025 según la DGT) prioriza precisamente este tipo de inversiones, alineando el proyecto con objetivos de política pública con presupuesto asignado. Castilla y León cuenta con una red extensa de autovías secundarias de perfil análogo al de la A-601 (A-62, A-66, A-67, A-231, entre otras), por lo que un piloto exitoso es directamente replicable y multiplicable.
- El abandono comercial de zonas de IMD baja por parte de los operadores móviles deja un nicho desatendido idóneo para iniciativas de servicio público.
- Los fondos Next Generation EU y FEDER habilitan vías de cofinanciación específicas para movilidad inteligente e infraestructura digital.
- La red desplegada admite usos secundarios (flotas, vehículos de emergencia, sensores ambientales del propio entorno del pinar) que diversifican el retorno de la inversión.

5.1. Componentes esenciales de una red 4G

Una red 4G operativa requiere los siguientes componentes:

1. **Estación base (eNodeB):** punto de acceso radio.
2. **Core (EPC):** MME, S-GW, P-GW y HSS.
3. **Terminal de usuario:** UE compatible con LTE.
4. **Antenas y torres de comunicación:** fijación física y radiación.

5. **Backhaul:** conexión troncal del eNB al core, normalmente fibra óptica [Rodríguez Cayetano, 2026].
6. **Elementos de seguridad:** autenticación (MILENAGE) y cifrado (EEA0/EIA1 o superior).

5.2. Propuesta de infraestructura real

Con el objetivo de cumplir con la cláusula tercera del pliego del proyecto y llevar a cabo el despliegue de un servicio de detección de señales de tráfico para coche conectado en el tramo de la autovía A-601 de Pinares [Wikipedia, 2026]: Valladolid–Segovia (km 0 al km 104), se ha procedido del siguiente modo:

1. **Estudio de cobertura** con Xirio Online para determinar el número y emplazamiento de las estaciones base necesarias.
2. **Estudio de viabilidad** a partir del cálculo teórico de la capacidad de celda.



Figura 5.1: Autovía A-601 de Pinares, Valladolid-Segovia.

5.2.1. Estudio de cobertura

Elección de la herramienta y del método de cálculo

La realización del estudio de planificación radioeléctrica se fundamenta en el uso de una herramienta de simulación profesional capaz de modelar redes inalámbricas complejas, integrando datos topográficos y modelos de propagación específicos para servicios móviles. Tras evaluar varias herramientas disponibles; como Radio Mobile Online, HeyWhatsThat y Xirio Online, se ha optado por **Xirio Online** [Aptica, 2026] por su capacidad para manejar proyectos complejos, la disponibilidad de un nivel gratuito de baja resolución y la posibilidad de configurar manualmente bandas no incluidas en el catálogo por defecto.

Al tratarse de un despliegue sobre la Banda 7 (2,6 GHz) para una aplicación de vehículo conectado en un tramo interurbano de 104 km, la elección del método de cálculo es el parámetro más sensible para asegurar que los resultados de cobertura sean realistas y no induzcan a errores en la ubicación de las estaciones base. Tras evaluar exhaustivamente el catálogo de algoritmos que ofrece Xirio, se ha procedido a descartar de forma razonada aquellos modelos cuyas limitaciones técnicas impedirían una simulación precisa del escenario Valladolid-Segovia.

Modelos determinísticos basados en difracción En primer lugar, se han desestimado los métodos determinísticos basados en difracción, como las recomendaciones **UIT-R P.526-11 y UIT-R P.526-15**, así como el método de **Deygout**. Aunque estos modelos son extremadamente precisos, su efectividad está supeditada a la disponibilidad de cartografía de alta resolución y modelos digitales de elevación (MDE) que incluyan edificios y vegetación detallada. El uso de estos algoritmos con cartografía de resolución media; que es propia de las versiones gratuitas o de baja resolución de la herramienta, suele derivar en resultados excesivamente pesimistas o erróneos al no poder modelar correctamente la difracción en el terreno, lo que invalidaría el análisis de las zonas de sombra reales.

Modelos urbanos Los modelos diseñados específicamente para el entorno urbano o de corto alcance también se han descartado:

- El método **Xia-Bertoni** requiere obligatoriamente información detallada de la edificación para modelar las pérdidas por difracción sobre los tejados. En un trazado interurbano carece de sentido.
- El modelo **COST 231 Walfish-Ikegami** es una evolución del Walfish-Bertoni para entornos urbanos densos, con factores de calle y altura de edificio. Inaplicable al escenario propuesto.
- La recomendación **UIT-R P.1411-9** establece modelos de propagación para enlaces de corto alcance (< 1 km) en exteriores e interiores en entornos urbanos. La distancia entre estaciones base del proyecto (varios kilómetros) la sitúa fuera de su rango de aplicabilidad.

Modelos para radiodifusión La recomendación **UIT-R P.1546** se ha desestimado por estar orientada principalmente a servicios de radiodifusión y frecuencias inferiores a 1 GHz, lo que la sitúa fuera del rango de la banda 7. Por la misma razón se descartó el método **Interior**, ajeno al escenario de propagación en exteriores que requiere el coche conectado.

Modelos limitados a frecuencias inferiores a 2 GHz Un punto crítico en la decisión ha sido el rango de frecuencia de operación. Los métodos de **Okumura-Hata** y su variante **Okumura-Hata Modificado**, ampliamente utilizados en estudios de 2G y 3G, han sido descartados debido a que su validez técnica finaliza en los 2 GHz. Aplicar estos modelos a la banda de 2,6 GHz requeriría una extrapolación matemática que comprometería la fiabilidad de los niveles de señal recibida en el terminal móvil.

Elección final: Stanford University Interim (SUI) En consecuencia, el estudio se ha implementado utilizando el método **Stanford University Interim (SUI)**. Este modelo empírico se erige como la opción más robusta para nuestro despliegue por tres razones fundamentales:

1. Permite trabajar con frecuencias de **hasta 11 GHz**, cubriendo holgadamente nuestra banda de 2,6 GHz.
2. Está **específicamente diseñado para servicios móviles de banda ancha** de alta velocidad (LTE, WiMAX) en entornos rurales y mixtos.
3. Ofrece predicciones **estables** incluso cuando no se dispone de cartografía urbana con edificios detallados.

Gracias al uso del modelo SUI, es posible identificar con precisión los puntos donde la señal RSRP desciende de los -95 dBm, umbral de sensibilidad crítica para nuestro ancho de banda de 10 MHz, permitiendo así una planificación eficiente del despliegue.

Configuración de la banda de frecuencias

Se ha definido manualmente la banda 7 (2600 MHz) en el catálogo de Xirio. La configuración técnica establecida se recoge en la siguiente captura.

Propiedades de la Banda de Frecuencias

Banda ☆

Nombre:

Descripción 1:

Descripción 2:

☐ **Color:**

Parámetros de la banda

Ancho de canal / Separación entre portadoras: MHz ▾

Ordinal del primer canal:

Tramo inferior:

Frecuencia inicial: MHz ▾

Frecuencia final: MHz ▾

Frecuencia primera portadora: MHz ▾

☒ **Tramo superior:**

Frecuencia inicial: MHz ▾

Frecuencia final: MHz ▾

Frecuencia primera portadora: MHz ▾

Canales prohibidos:

Canales prioritarios:

Introduzca una lista de canales separados por comas y/o intervalos de canales (Ejemplo: 2, 2', 5-7, 12'-21').

Figura 5.2: Propiedades de la Banda de Frecuencias en el estudio de Xirio.

Parámetros de las estaciones base y de los terminales

Para maximizar el alcance en carretera y permitir mayor separación entre torres, se ha optado por **antenas sectoriales directivas de 65°**. Estas antenas ofrecen una ganancia de 17,5 dBi, frente a los escasos 2 dBi de una omnidireccional, concentrando la energía a lo largo del trazado y minimizando las pérdidas hacia zonas no deseadas. Los parámetros completos del eNB y del

terminal se pueden observar en la captura de las características del transmisor.

Propiedades del sector

Sector ☆

Nombre: Tx

Parámetros de radio

Tipo sistema: Estándar

Antena: 3G/4G/5G 17,5 dBi 65°

Altura antena: 30 m

Orientación: 150 °

Inclinación mecánica: 2 °

Inclinación eléctrica: 5 °

Referencia de alturas de antenas

Alturas respecto a: Nivel del terreno

Usar altura de edificio: Capa de elevación (MDE)

Altura edificio: 0 m

Frecuencias de transmisión

Frecuencias
2650.000 MHz

Polarización: Cruzada

Feeder:

Longitud del feeder: 0 m

Pérdidas del feeder: 0.00 dB

Pérdidas pasivos: 2 dB

Potencia: 60 W

Potencia CRS: W

Figura 5.3: Propiedades del sector (transmisor).

La sensibilidad RSRP de -92 dBm y la potencia de transmisión de 23 dBm se obtienen de la *Release 10* de 3GPP para la banda 7 con 20 MHz de ancho de banda.

Conviene destacar que los parámetros introducidos en el simulador no son valores arbitrarios, sino que se corresponden con las especificaciones reales de los equipos seleccionados en el capítulo de valoración económica (Capítulo 8). En concreto, la potencia de salida y el número de puertos de antena se toman de la hoja de características de la RRU *Baicells* elegida, la ganancia de $17,5$ dBi corresponde a la antena sectorial *Kathrein* presupuestada, y la altura de mástil se ajusta a la torre de celosía de 30 m contemplada en la obra civil. De este modo, la elección de los equipos y el estudio de cobertura no son dos ejercicios independientes: las especificaciones del hardware

alimentan directamente la simulación de Xirio, y el resultado de cobertura valida *a posteriori* que los equipos escogidos son adecuados para garantizar el RSRP objetivo en todo el trazado. Si en una iteración de diseño se cambiara el modelo de RRU o de antena, el estudio de cobertura debería repetirse con los nuevos parámetros.

Justificación de los rangos de cobertura RSRP

Para que el mapa de cobertura refleje fielmente la experiencia de conectividad del vehículo, se ha definido la escala cromática, basada en los umbrales de sensibilidad de referencia (REFSENS) recogidos en 3GPP TS 36.101.

Propiedades del rango de colores

Rango de colores ☆			
Nombre:		cobertura 4G	
Rangos			
Color	Rango	Descripción	
	$[-100, -98)$	sin cobertura	
	$[-98, -95)$	cobertura BW 5-10	
	$[-95, -93.2)$	cobertura 10-15	
	$[-93.2, -92)$	cobertura 15-20	
	$[-92, +\infty)$	cobertura total	
Visualizar niveles de señal ▼			

Figura 5.4: Propiedades del rango de colores.

Evolución del despliegue y resultados

Antes de presentar los resultados conviene explicar cómo deben interpretarse los mapas de cobertura de las figuras siguientes. Cada punto del trazado se colorea según el nivel de señal RSRP que recibiría allí el terminal del vehículo, de acuerdo con la escala cromática de la Figura 5.4: las zonas **sin color (blanco)** corresponden a RSRP inferior a -95 dBm (zona de sombra, sin servicio garantizado); las zonas en **amarillo** indican cobertura inestable o de cortesía (-98 dBm a -95 dBm); las zonas en **verde** (en sus distintas tonalidades, de oscuro a claro) representan cobertura de servicio garantizado para el ancho de banda objetivo, y las zonas en **naranja** corresponden a los niveles de señal más altos, óptimos para modulaciones de orden superior. El objetivo del diseño es que la totalidad del trazado de la autovía quede cubierta por las tonalidades verde y naranja, es decir, con RSRP por encima del umbral de -92 dBm.

El estudio se ha ejecutado en tres fases incrementales para optimizar la inversión:

- **Fase I — Infraestructura de Vodafone.** Se inició el cálculo utilizando exclusivamente los emplazamientos existentes de Vodafone en la zona, identificados a través de la base de datos InfoAntenas [Ministerio para la Transformación Digital y de la Función Pública, 2026b]. El resultado fue una cobertura fragmentada con múltiples zonas de sombra, especialmente en los tramos interurbanos.

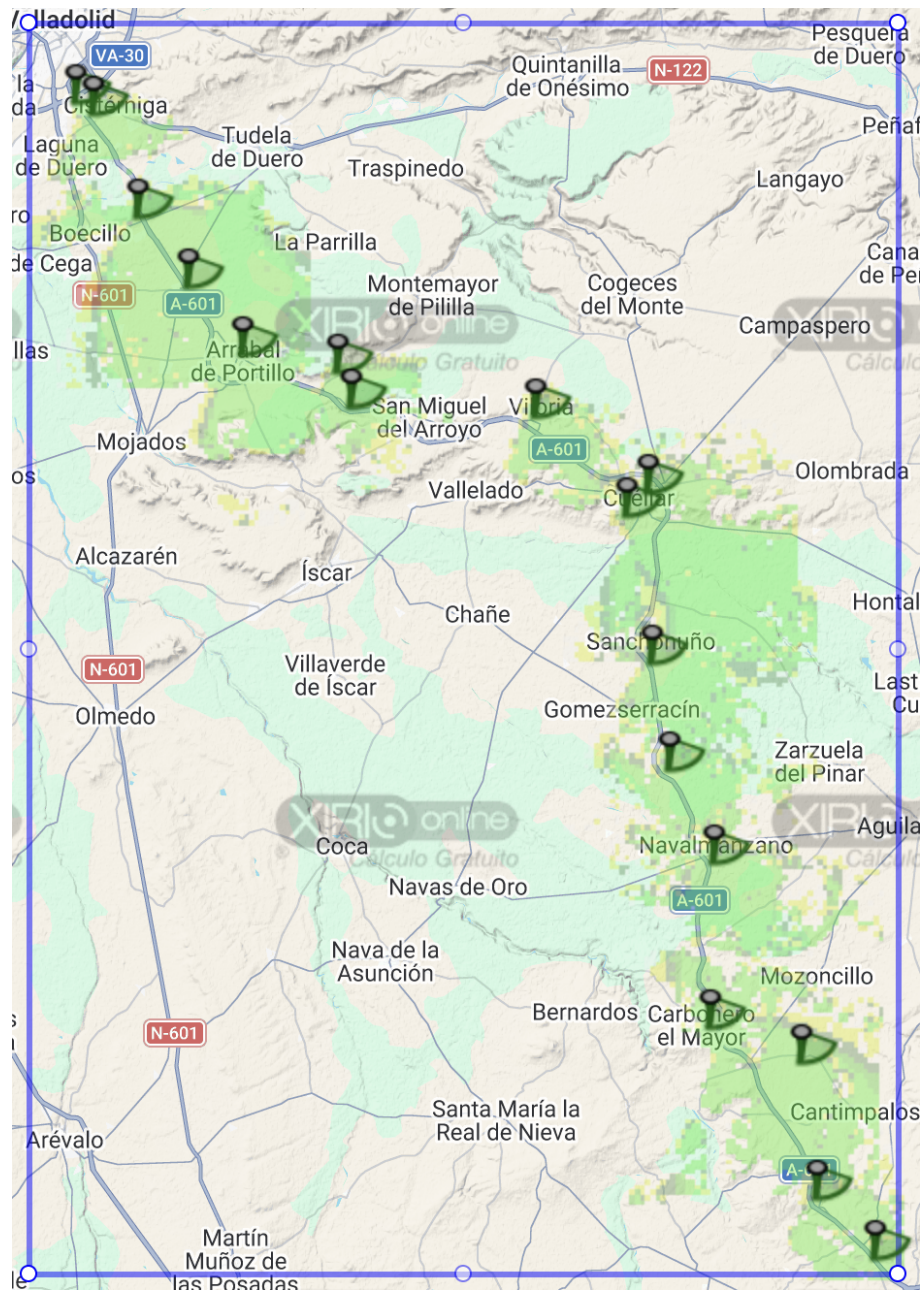


Figura 5.5: Estudio de Cobertura con las antenas de Vodafone.

- **Fase II — Multitransmisor (Vodafone + Telefónica).** Se añadieron 4 estaciones de Telefónica ya existentes, recurriendo a la figura del *site sharing*. La cobertura mejoró sustancialmente, pero persistían puntos críticos en el tramo central.

Tabla 5.1: Resumen de la infraestructura propuesta.

E-UTRAN	EPC
Antena sectorial UE-eNB → 2/estación	1 servidor Core (MME, S-GW, P-GW, HSS)
Antena direccional eNB-eNB → 1/estación	1 OLT (Olmedo)
RRU → 1/antena	1 ONT/estación
BBU → 1/estación	2 conmutadores ópticos (Valladolid, Segovia)
Fibra óptica RRU-BBU → 10 m/eNB	Fibra óptica para los 104 km
Torre de comunicación → 1/estación	Cómputo de IA en la nube (computación)
Despliegue e instalación → 1/estación	Cómputo de IA en la nube (control de instancias)
Racks → toda la red	Cómputo de IA en la nube (métricas)

El despliegue total propuesto consiste en **10 estaciones base nuevas** y **21 emplazamientos en alquiler** sobre torres ya existentes de Vodafone y Telefónica.

5.2.3. Estudio de viabilidad y capacidad de red

Hasta este punto, el diseño de la red ha priorizado la disponibilidad espacial de la señal, definiendo la ubicación óptima de los nodos (eNodeB) para garantizar niveles de RSRP suficientes en todo el trazado. Sin embargo, en un escenario de vehículo conectado, la cobertura radioeléctrica es solo un prerequisite. El factor limitante para la viabilidad del sistema es la capacidad de la celda en el enlace ascendente (Uplink), ya que la red debe soportar la llegada concurrente de flujos de vídeo en tiempo real desde múltiples vehículos hacia el *Edge Computing* para su posterior procesamiento.

Requisitos de Tráfico Generado por Vehículo

El diseño de la capacidad de red exige definir primero la carga (*throughput*) demandada por cada usuario, que depende de la tasa de fotogramas por segundo (fps) que el vehículo debe transmitir. Es importante razonar este valor correctamente: la tasa de fps no se deriva de la separación *media* entre señales de tráfico, sino de dos factores que sí imponen un requisito físico sobre el sistema.

Rango de detección del modelo de IA. El primer factor es la distancia a la que el modelo es capaz de detectar una señal con fiabilidad. Una señal de tráfico solo es detectable cuando ocupa en la imagen un número de píxeles suficiente para que el modelo la clasifique correctamente; por debajo de cierto tamaño aparente (es decir, a partir de cierta distancia), la señal es demasiado pequeña y la precisión es muy baja. En la práctica, y dependiendo de la resolución de la cámara y del tamaño de la señal, este rango útil de detección suele ser inferior a los 300 m [Galvao et al., 2021], y en muchos casos, bastante menor. Lo relevante para el dimensionamiento no es, por tanto, cada cuántos metros hay una señal, sino garantizar que, dentro de la ventana de distancia en la que la señal es detectable, el vehículo capte un número suficiente de fotogramas de esa señal para no omitirla.

Caso peor: señales próximas entre sí. El segundo factor, y el verdaderamente limitante, es el caso peor: dos señales consecutivas situadas a muy poca distancia una de otra. La separación entre señales en una vía no es uniforme; es habitual encontrar señales agrupadas, por ejemplo, una señal de obras o de peligro seguida pocos metros después de una señal de límite de velocidad o de un STOP. Si dos señales distan, pongamos, 25 m, el sistema debe ser capaz de captar, clasificar y reaccionar ante ambas de forma independiente. Es este escenario; y no la separación media de 200 m a 400 m [Ministerio de Fomento, 2014], el que fija el requisito mínimo de fps. En

nuestro modelo entrenado en el laboratorio, se realizaron fotografías con más de una señal para que el modelo sea capaz de solventar este problema.

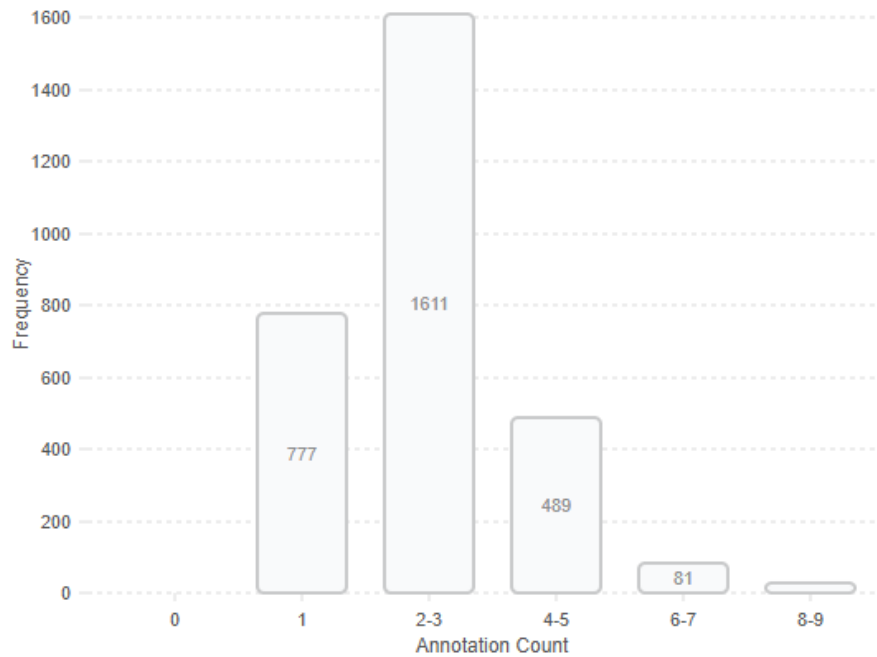


Figura 5.8: Numero de señales etiquetadas por cada fotograma.

Determinación de la tasa de fotogramas. Considerando un vehículo que circula a la velocidad máxima de la vía ($120 \text{ km/h} \approx 33,3 \text{ m/s}$) y exigiendo captar al menos varios fotogramas de cada señal dentro de su ventana de detección útil; y en particular ser capaz de resolver dos señales separadas por unas pocas decenas de metros, se establece una tasa de transmisión objetivo de **30 fotogramas por segundo**. A esta cadencia, el vehículo captura una imagen cada 1,1 m recorridos, lo que garantiza que incluso dos señales muy próximas (separadas por 25 m) generen cada una varios fotogramas independientes (del orden de 3 a 4 imágenes en ese tramo), suficientes para una detección y una reacción fiables. Esta tasa supone, además, un compromiso razonable con la carga que se inyecta en el enlace ascendente de la red LTE, como se cuantifica a continuación.

La cámara de a bordo (*dashcam*) está configurada para emitir un flujo de vídeo H.264 con resolución 480p a 30 fps y a un bitrate de 2 Mbit/s [González, 2022].

Conviene matizar que este valor de 2 Mbit/s es un valor constante, y no el promedio de una tasa variable. La codificación H.264 se puede configurar de modo que el bitrate sea constante o adaptativo, donde el peso de cada fotograma depende fuertemente del contenido de la escena y de su variación entre fotogramas consecutivos. En un vehículo en movimiento, donde la escena cambia continuamente, los fotogramas que presenten más cambios pueden provocar que la tasa instantánea presente picos muy por encima de la media. En nuestro caso hemos preferido evitarlo garantizando un bitrate constante. Esto tiene su contrapartida ya que codificador degradará la calidad de imagen para respetar la tasa objetivo en las escenas más dinámicas [Casaseca de la Higuera, 2026b].

Capacidad Teórica Sectorial (Enlace Ascendente)

Con la demanda individual definida, es necesario calcular la capacidad de absorción de tráfico de cada estación base. El sistema opera en la Banda 7, utilizando un canal con un ancho de banda

dedicado de 20 MHz para el *Uplink*.

Para garantizar la estabilidad del enlace, minimizando la tasa de error por bloque (BLER), se ha descartado el uso de modulaciones de orden superior (como 64-QAM) en favor de la modulación 16-QAM. Esta decisión de diseño es más conservadora y robusta, requiriendo una relación señal a ruido (SNR) superior a 15 dB, un valor que nuestras celdas sectoriales de 65° pueden garantizar en la totalidad del tramo de autovía.

El cálculo de la capacidad se realiza a partir de la estructura física de la trama LTE, y no como un producto directo del ancho de banda por la eficiencia espectral, lo que da una estimación más fiel. Un canal de 20 MHz se compone de **100 Resource Blocks (PRB)**. Cada PRB consta de 12 subportadoras, y cada subtrama de 1 ms contiene 14 símbolos OFDM (2 *slots* de 7 símbolos con prefijo cíclico normal). El número de elementos de recurso (RE) por subtrama es, por tanto:

$$RE_{\text{subtrama}} = 100 \text{ PRB} \times 12 \text{ subport.} \times 14 \text{ símbolos} = 16\,800 \text{ RE}$$

Empleando modulación 16-QAM, cada RE transporta 4 bits, lo que da una tasa bruta de:

$$R_{\text{bruta}} = 16\,800 \text{ RE} \times 4 \frac{\text{bits}}{\text{RE}} \times 1\,000 \frac{\text{subtramas}}{\text{s}} = 67,2 \text{ Mbit/s}$$

Sobre esta tasa bruta debe descontarse el *overhead* de la interfaz radio: las señales de referencia de demodulación (DMRS) del enlace ascendente, los canales de control (PUCCH), el prefijo cíclico y, sobre todo, la tasa de codificación del canal (típicamente en torno a 0,75 para 16-QAM con un BLER objetivo del 10 %). Agrupando todas estas pérdidas en un factor de eficiencia conjunto del 70 %, la capacidad neta del sector en el enlace ascendente resulta:

$$\text{Throughput}_{UL} = 67,2 \text{ Mbit/s} \times 0,70 \approx 47 \text{ Mbit/s}$$

Es importante señalar que **no se ha aplicado ninguna ganancia por MIMO** en este cálculo. Aunque las estaciones base se han dimensionado con MIMO 2×2, la multiplexación espacial en el enlace ascendente (SU-MIMO en UL) solo está disponible para categorías de UE elevadas y en condiciones de canal muy favorables; un módem convencional como el del vehículo opera habitualmente con una única capa en transmisión. Por prudencia de diseño, la capacidad se calcula en el caso conservador (una capa espacial), y la eventual ganancia MIMO en UL se consideraría un margen adicional, no un recurso garantizado.

Conociendo la capacidad neta de la celda y la demanda por vehículo (2 Mbit/s), el número máximo de usuarios (m) a los que se les puede dar servicio de forma simultánea, garantizando la Calidad de Servicio (QoS) estipulada, es:

$$m = \frac{47 \text{ Mbit/s}}{2 \frac{\text{Mbit/s}}{\text{usuario}}} \approx 23 \text{ usuarios concurrentes}$$

Consideraciones Estadísticas del Tráfico (Teoría de Colas)

El cálculo anterior establece que una celda puede dar servicio simultáneo a unos 23 vehículos. Queda por comprobar si ese valor es suficiente frente al tráfico real que circula por la autovía. La clave de este análisis, y un error frecuente, es no confundir el *flujo* de vehículos que atraviesan la celda a lo largo del día con el número de vehículos que se encuentran *simultáneamente* dentro de su cobertura demandando servicio. Lo segundo es lo que determina la carga real de la celda [Asensio-Pérez, 2023/2024a].

Número de vehículos simultáneos en la celda Se toma como referencia un tramo exigente con una Intensidad Media Diaria (IMD) de 20 000 vehículos/día. Dicho dato se ha extraído del último Mapa de Tráfico disponible de la Red Regional de Carreteras de Valladolid [Junta de Castilla y León, 2024] y que puede verse en la Figura 5.9.

En base a ello, el flujo horario medio es:

$$\Phi = \frac{20\,000 \text{ veh/día}}{24 \text{ h}} \approx 833 \text{ veh/h}$$

A una velocidad de circulación de 120 km/h, la densidad lineal de vehículos sobre la vía es:

$$\rho = \frac{\Phi}{v} = \frac{833 \text{ veh/h}}{120 \text{ km/h}} \approx 6,9 \text{ veh/km}$$

El número de vehículos presentes simultáneamente bajo la cobertura de una celda depende, por tanto, de la longitud de vía que esta cubre. La red diseñada consta de 31 estaciones sobre los 104 km del trazado, lo que da una separación media entre emplazamientos de 3,35 km. No obstante, las antenas sectoriales directivas no cubren la totalidad de ese tramo de forma exclusiva: existe un solapamiento entre celdas necesario para el *handover*, de modo que la longitud de vía efectivamente servida por un único sector es del orden de 1,5 km a 2 km. Tomando un valor conservador de 2 km de cobertura efectiva por sector:

$$N_{\text{simult}} = \rho \times L_{\text{sector}} = 6,9 \frac{\text{veh}}{\text{km}} \times 2 \text{ km} \approx 14 \text{ vehículos}$$

Es decir, en el tramo más cargado y en circulación fluida, cada sector debe atender del orden de **14 vehículos simultáneos**, frente a una capacidad de 23. El sistema dispone, por tanto, de un margen de aproximadamente el 40 % sobre la demanda nominal.

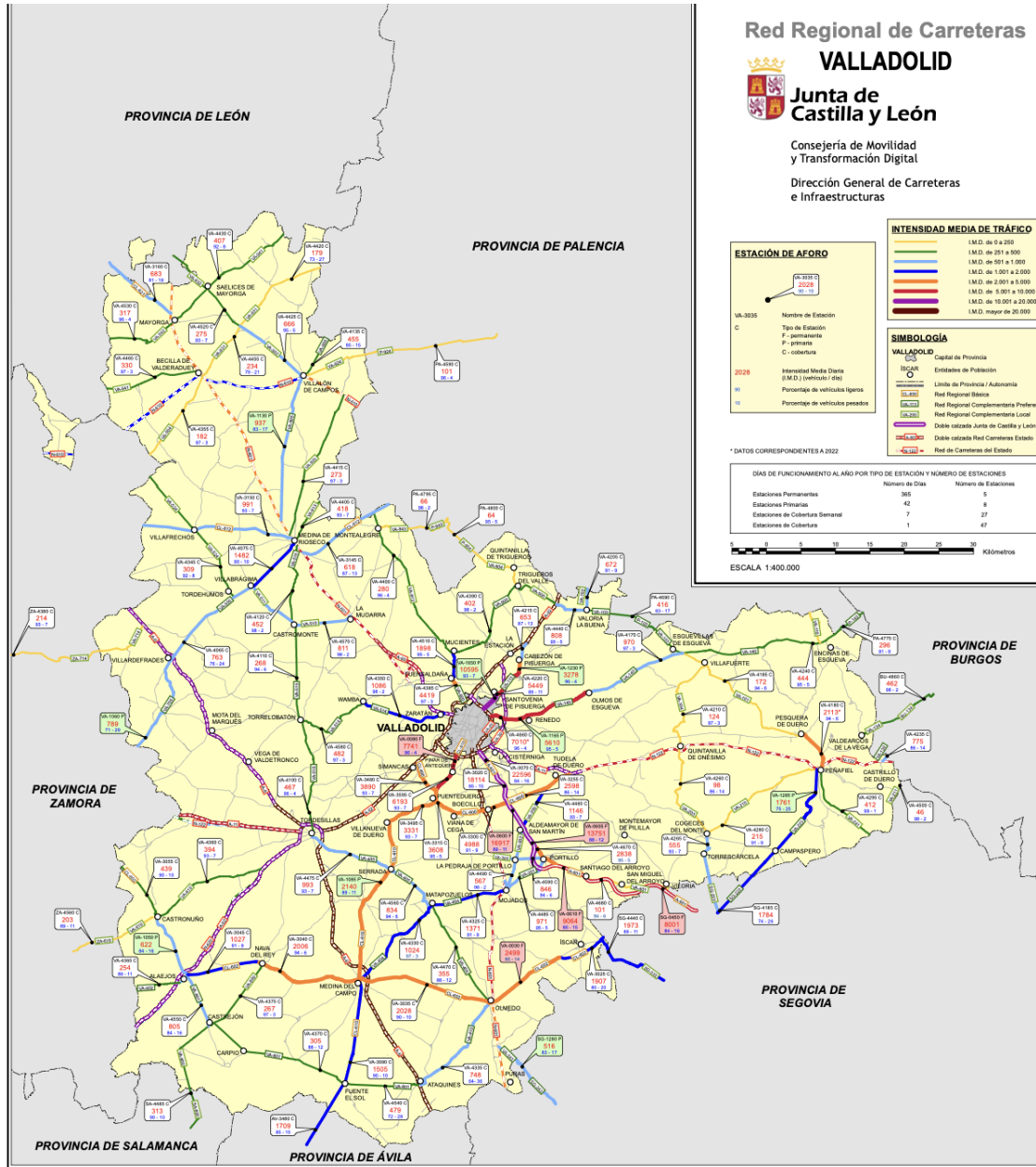


Figura 5.9: Tráfico (IMD) en la Autovía A-601 de Pinares, Valladolid-Segovia.

Modelado del sistema de colas Para evaluar el comportamiento frente a la variabilidad estadística de las llegadas, se modela cada sector como un sistema de pérdidas $M/M/m/m$ (proceso de llegadas de Poisson, tiempos de servicio exponenciales, m servidores iguales y sin cola de espera: si los recursos están agotados, el vehículo entrante no obtiene la QoS plena) [Asensio-Pérez, 2023/2024b]. El tráfico ofrecido al sistema, medido en Erlangs, coincide con el número medio de vehículos presentes simultáneamente:

$$A = N_{\text{simult}} \approx 14 \text{ Erlang}$$

Aplicando la fórmula de Erlang-B sobre $m = 23$ servidores [Asensio-Pérez, 2023/2024b]:

$$P_b = \frac{A^m/m!}{\sum_{n=0}^m A^n/n!}$$

$$P_b(A = 14, m = 23) \approx 0,004$$

La probabilidad de bloqueo en régimen nominal es de aproximadamente el 0,4 %, un valor perfectamente asumible para el servicio.

Análisis de estrés: escenario de congestión El modelo permite analizar qué ocurre en una retención, cuando la velocidad media de la vía cae a 60 km/h. La densidad lineal de vehículos se duplica al circular estos más despacio:

$$\rho_{\text{cong}} = \frac{833 \text{ veh/h}}{60 \text{ km/h}} \approx 13,9 \text{ veh/km} \Rightarrow A_{\text{cong}} \approx 13,9 \times 2 \approx 28 \text{ Erlang}$$

Con un tráfico ofrecido de 28 Erlang sobre 23 servidores, la probabilidad de bloqueo se dispara:

$$P_b(A = 28, m = 23) \approx 0,25$$

En un escenario de retención, en torno a una cuarta parte de los vehículos no podría mantener los 5 fps nominales. Este resultado, lejos de invalidar el diseño, identifica con precisión su límite operativo y motiva la incorporación de un mecanismo de **control de congestión**: al detectar saturación del sector, los vehículos reducirían automáticamente su tasa de emisión (por ejemplo, de 5 fps a 2 fps). Esta degradación es asumible desde el punto de vista de la seguridad, ya que a 60 km/h el tiempo de reacción disponible ante una señal se duplica respecto a la circulación a 120 km/h. Reduciendo la carga por usuario a 0,4 Mbit/s, la capacidad del sector ascendería a unos 58 usuarios y el bloqueo volvería a ser despreciable [Asensio-Pérez, 2023/2024c].

Conclusión final de viabilidad

El cruce del estudio de cobertura en Xirio con el modelo estocástico de tráfico demuestra que la infraestructura de 31 estaciones base operando en la Banda 7 (10 MHz) es viable para el servicio propuesto. En régimen nominal (circulación fluida a 120 km/h), cada sector atiende del orden de 14 vehículos frente a una capacidad de 23, con una probabilidad de bloqueo inferior al 0,5 %. En escenarios de congestión la red presenta una degradación predecible que se gestiona de forma segura mediante control de tasa adaptativo. Conviene subrayar que estos cálculos asumen una carga por vehículo constante de 2 Mbit/s; en la práctica, la codificación H.264 es adaptativa y una cámara en movimiento, con cambios rápidos de escena, puede generar picos de tasa superiores, por lo que el dimensionamiento real debería contemplar un margen adicional o un perfil de tasa variable.

5.3. Infraestructura de red 4G en el laboratorio

Como objetivo paralelo al diseño teórico, se ha desplegado una red LTE funcional en el laboratorio del equipo. La red sirve como demostrador del servicio y permite validar mediciones de rendimiento sobre *hardware* real.

5.3.1. Selección del software 4G: DAFO

Antes de iniciar el despliegue se realizó un análisis DAFO (Debilidades, Amenazas, Fortalezas y Oportunidades) de los principales paquetes *open source* para desplegar redes LTE: srsRAN, OpenAirInterface (OAI) y Open5GS, partiendo del análisis comparativo de [González, 2022]. El análisis resultante se resume en las Tablas 5.2 y 5.3.

Tabla 5.2: DAFO de los *softwares* 4G *open source* candidatos.

Debilidades srsRAN se encuentra en desarrollo activo, lo que puede provocar cambios constantes y <i>breaking changes</i> . Requiere mayor conocimiento técnico que OAI u Open5GS para configuraciones avanzadas. OAI requiere una mayor cantidad de recursos para ser desplegado, seguido de Open5GS y después srsRAN.	Amenazas Al ser <i>softwares</i> de código abierto pueden estar sujetos a vulnerabilidades. srsRAN y OAI pueden presentar problemas de interoperabilidad entre sí. OAI solo es compatible con Ubuntu 18.04 (limita el sistema operativo). OAI se enfoca en el desarrollo del entorno completo, Open5GS en el <i>core</i> y srsRAN en el despliegue eficiente de la RAN.
Fortalezas srsRAN es una plataforma de código abierto, con mayor facilidad para encontrar soluciones en línea y documentación. Todos los candidatos son gratuitos. srsRAN tiene una mayor facilidad de implementación que OAI y Open5GS. srsRAN es compatible con más versiones de Ubuntu.	Oportunidades Instalar y configurar el entorno más rápido que con otros <i>softwares</i> debido a la gran documentación y tutoriales existentes. Posibilidad de cambiar la versión de Ubuntu dentro de un cierto límite sin afectar al sistema.

Para apoyar la decisión de manera cuantitativa, se realizó una tabla de puntuaciones (Tabla 5.3) en la que se asigna a cada candidato un nivel de cumplimiento de cada característica (0 = no cumple, 1 = cumple ligeramente, 2 = cumple consistentemente, 3 = excelente), ponderado por la importancia de dicha característica para el caso de estudio ($\times 1$, $\times 2$ o $\times 3$).

Tabla 5.3: Comparativa cuantitativa de *softwares* 4G candidatos.

Característica	srsRAN	OAI	Open5GS
Existencia de documentación ($\times 3$)	3	2	1
Actualizaciones y soporte ($\times 1$)	3	3	1
Facilidad de implementación ($\times 3$)	3	2	1
Compatibilidad ($\times 3$)	3	3	1
Prestaciones ($\times 3$)	3	3	1
Seguridad ($\times 1$)	2	3	1
TOTAL	41	36	14

Las características evaluadas se definen como sigue: *existencia de documentación* (cantidad de documentos, tutoriales, blogs y vídeos relacionados); *actualizaciones y soporte* (frecuencia de *releases* y soporte oficial o de la comunidad); *facilidad de implementación* (en función de la opinión de foros y de usuarios de la comunidad); *compatibilidad* (grado de interoperabilidad con distintos sistemas operativos, *hardware* y otros programas); *prestaciones* (eficiencia y completitud); *seguridad* (resistencia a ataques y configuraciones inseguras por defecto).

Con **41 puntos sobre un máximo de 51**, srsRAN se erige como la opción más equilibrada para el demostrador, especialmente por su facilidad de implementación y la abundante documentación en línea, factores críticos para un proyecto académico con tiempo limitado.

5.3.2. Componentes empleados

- **SDR BladeRF 2.0 micro xA9** de Nuand, con dos antenas omnidireccionales tipo dipolo para la banda 7. Actúa como frontal radio del eNodeB. Esta SDR ofrece dos cadenas de Tx y Rx (MIMO 2×2), un rango de frecuencias de 47 MHz a 6000 MHz y un ancho de banda instantáneo de hasta 56 MHz.



Figura 5.10: Blade RF empleada en el entorno del laboratorio.

- **Dos ordenadores de sobremesa con Ubuntu 22.04 LTS** (kernel de baja latencia). Se ha optado por una arquitectura de dos máquinas, separando físicamente el *core* y la estación base para reflejar fielmente el modelo real de una red celular y simplificar la depuración.
- **Módem Huawei E3372h-320 LTE USB Stick** con SIM programable, integrado en el AWS DeepRacer. Soporta categoría LTE Cat. 4 (hasta 150 Mbit/s en DL).



Figura 5.11: Módem USB de Huawei empleado en el entorno del laboratorio.

- **AWS DeepRacer** como UE móvil con cámara, IMU y servicio ARTEMIS para control y telemetría. Dispone de un mini-ordenador integrado con CPU Intel Atom y batería para varias horas de funcionamiento.



Figura 5.12: Coche DeepRacer de AWS empleado en el entorno del laboratorio.

- **NVIDIA Jetson Orin Nano** (8 GB, GPU Ampere de 1024 núcleos CUDA y 32 *Tensor cores*) para inferencia local del modelo de IA. Se incorpora en una etapa avanzada del proyecto para sustituir la inferencia en la nube.



Figura 5.13: NVIDIA Jetson Orin nano empleada en el entorno del laboratorio.

- **Router Linksys** para interconectar la Jetson y el PC del laboratorio sobre una subred dedicada.

5.3.3. Arquitectura software: srsRAN sobre Docker Engine nativo

La elección del *stack software* ha estado marcada por una decisión inicial crítica: **utilizar Docker Engine nativo en lugar de Docker Desktop**. Docker Desktop en Linux ejecuta los

contenedores dentro de una máquina virtual ligera con kernel `linuxkit`, lo que impide el acceso directo al hardware USB del *host*. Esto hace inviable el uso del BladeRF, ya que el dispositivo no aparece dentro del contenedor incluso con `--privileged` y `-v /dev/bus/usb:/dev/bus/usb`.

Se evaluó también el uso de `usbip` como puente USB entre *host* y contenedor, pero el kernel `linuxkit` no dispone de paquetes `usbip` en los repositorios de Ubuntu, lo que invalidó esa alternativa. La sustitución por Docker Engine permite acceder al BladeRF de forma directa a través del montaje del bus USB.

Despliegue por fases

El proceso completo de despliegue del demostrador se documenta en la wiki interna del proyecto en Confluence (página [INFRAESTRUCTURA](#)). El desarrollo se estructuró como una cadena de decisiones técnicas en la que cada paso habilitaba el siguiente y, en varios casos, decisiones aparentemente colaterales (como la elección del *runtime* de Docker) resultaron críticas para el éxito global. A continuación se describen, de forma cronológica y razonada, las distintas fases que componen el despliegue. Cada fase incluye la descripción de lo realizado, la justificación técnica de las decisiones adoptadas y la manera en que condicionó o habilitó la fase siguiente.

Fase 1 — Elección del *runtime* de Docker. *Descripción.* La primera decisión estructural del proyecto fue sustituir Docker Desktop por Docker Engine nativo en las máquinas Linux Ubuntu 22.04 empleadas tanto como núcleo (*core*) como estación base (eNodeB). Se procedió a desinstalar completamente Docker Desktop, limpiar los repositorios conflictivos, añadir el repositorio oficial de Docker y arrancar el demonio nativo mediante `systemd`.

Justificación. Docker Desktop en Linux ejecuta los contenedores dentro de una máquina virtual ligera con un kernel especial denominado `linuxkit`. Esta arquitectura impide el acceso directo al hardware USB del *host*, lo cual resulta incompatible con el uso de un SDR como el BladeRF 2.0 micro, que requiere USB 3.0 SuperSpeed sin intermediarios. Se intentó inicialmente trabajar sobre Docker Desktop, pero el dispositivo no aparecía dentro del contenedor incluso con las banderas `--privileged` y `-v /dev/bus/usb:/dev/bus/usb`. El diagnóstico definitivo fue la aparición del kernel `linuxkit` al ejecutar `lsusb` dentro del contenedor. También se evaluó el uso de `usbip` como puente USB entre *host* y contenedor, pero el kernel `linuxkit` no tiene paquetes `usbip` disponibles en los repositorios de Ubuntu, lo que invalidó esta opción.

Impacto en fases posteriores. Esta decisión fue absolutamente crítica: sin Docker Engine nativo, el BladeRF nunca habría sido accesible dentro del contenedor y todo el proyecto habría sido inviable. Además, habilitó el uso del modo `--network host`, esencial para la comunicación entre los contenedores *core* y eNodeB ubicados en máquinas distintas.

Fase 2 — Compilación de la librería `bladeRF`. *Descripción.* Se creó un contenedor base a partir de la imagen oficial `ubuntu:22.04` con las banderas `--privileged` y el montaje de `/dev` para permitir el acceso USB. Dentro del contenedor se instalaron las dependencias de desarrollo (`cmake`, `build-essential`, `libusb`, `libtecla`, `libncurses`, `libcurl`, `libxml2`) y se clonó el repositorio oficial de Nuand. La compilación se realizó con `CMAKE_BUILD_TYPE=Release` e `INSTALL_UDEV_RULES=OFF`, finalizando con `ldconfig` para registrar las librerías en el sistema.

Justificación. Se optó por compilar desde el código fuente en lugar de usar los paquetes precompilados de Ubuntu por dos motivos: disponer de la versión más reciente y, sobre todo, garantizar la compatibilidad con el *bitstream* de FPGA que, posteriormente, sería necesario cargar. La opción `INSTALL_UDEV_RULES=OFF` es relevante porque dentro de un contenedor no existe demonio `udev` que gestione esas reglas; intentar instalarlas provoca errores. Los permisos USB se gestionaron en el *host* mediante `chmod` o mediante el montaje completo de `/dev/bus/usb`.

Impacto en fases posteriores. La disponibilidad de `bladeRF-cli` y `libbladeRF` era un requisito previo para la compilación de srsRAN con soporte BladeRF. Además, la detección correcta del

dispositivo (verificada con `bladeRF-cli -p`) confirmó que el enfoque de Docker Engine nativo funcionaba, validando la decisión de la fase anterior.

Fase 3 — Carga del bitstream de FPGA. *Descripción.* El BladeRF 2.0 micro xA9 requiere que se cargue un *bitstream* de FPGA para estar operativo. Se descargó la versión v0.15.3 del *bitstream* `hostedxA9` (archivo `.rbf`) desde el servidor de Nuand y se cargó en RAM mediante `bladeRF-cli -l`. Se verificó mediante *checksum* MD5 que el archivo descargado se correspondía con la versión solicitada, ya que el enlace `hostedxA9-latest.rbf` apunta siempre a la última versión publicada.

Justificación. Se eligió la versión v0.15.3 porque era la versión especificada por el equipo de desarrollo como estable para la configuración objetivo. La carga en RAM (mediante `-l`) en lugar de programación en *flash* (mediante `-L`) se prefirió por seguridad: no modifica nada permanente en el dispositivo y permite revertir cualquier cambio simplemente desconectando el USB. Se descartó explícitamente tocar el *firmware* FX3 del dispositivo, lo cual habría sido irreversible y habría bloqueado el dispositivo en caso de error.

Impacto en fases posteriores. Sin FPGA cargada, el dispositivo solo responde a comandos básicos, pero no puede procesar muestras IQ, lo que imposibilita su uso como frontal radio de srsRAN. La carga en RAM implica que cada reinicio del contenedor requiere repetir el paso, lo que se documentó como parte de la secuencia de arranque.

Fase 4 — Compilación e instalación de srsRAN 4G. *Descripción.* Se clonó el repositorio `srsRAN_4G` desde GitHub y se compiló con la bandera `-DUSE_BLADERF=ON`, que habilita la compilación del módulo de radio para BladeRF. Tras la instalación, quedaron disponibles los tres componentes principales: `srsepc` (núcleo EPC), `srsenb` (estación base eNodeB) y `srsue` (equipo de usuario simulado). El comando `srsran_install_configs.sh user` generó la plantilla de ficheros de configuración en `~/config/srsran/`.

Justificación. srsRAN 4G se eligió por ser una implementación de software completa y de código abierto de la pila LTE, con soporte nativo para múltiples SDR, incluido BladeRF. La bandera `USE_BLADERF=ON` era indispensable: sin ella, el binario `srsenb` no podría instanciar el frontal radio. La versión 25.10.0 incluye el *plugin* `libsrsran_rf_blade.so` como módulo dinámico, lo que permite que se cargue solo cuando se solicita.

Impacto en fases posteriores. Con srsRAN instalado, quedaba lista la base *software* para levantar la red LTE. Esta fase se replicó en las dos máquinas, pero con configuraciones divergentes: el *core* levantaría `srsepc` y el eNodeB levantaría `srsenb`, lo que motivó la fase siguiente de separación física de componentes.

Fase 5 — Arquitectura de dos máquinas y red *host*. *Descripción.* Se decidió separar el *core* y el eNodeB en dos máquinas físicas distintas conectadas a la misma LAN: la máquina *core* con IP `10.0.128.176` y la máquina eNodeB con IP `10.0.128.181`. Los contenedores se lanzaron con la bandera `--network host`, compartiendo directamente la pila de red del anfitrión en lugar de usar la red *bridge* por defecto de Docker.

Justificación. La separación física obedece al modelo real de una red celular: el *core* gestiona la autenticación, movilidad y salida a Internet; mientras que el eNodeB se encarga de la capa radio. Intentar poner ambos en una misma máquina complicaría la depuración y no reflejaría una arquitectura desplegable. En cuanto a la red Docker, la elección de `--network host` fue crítica porque con la red *bridge* por defecto los contenedores tendrían la IP `172.17.0.2` (la misma en ambas máquinas) y no podrían comunicarse directamente. Con `--network host` cada contenedor hereda la IP del anfitrión (`10.0.128.176` y `10.0.128.181` respectivamente) y la comunicación S1AP entre eNodeB y EPC funciona de forma transparente.

Impacto en fases posteriores. Esta decisión condicionó toda la configuración posterior: los ficheros `epc.conf` y `enb.conf` tuvieron que ajustarse con las IPs físicas de cada máquina. También

simplificó enormemente la exposición del *broker* MQTT y del servidor UDP, ya que escucharían directamente en la IP pública del *core* sin necesidad de reglas de *port forwarding*.

Fase 6 — Configuración del núcleo EPC. *Descripción.* Se editó el fichero `epc.conf` para establecer los parámetros de la red LTE: MCC 901 y MNC 70, TAC 7, dirección de enlace MME y GTP-U en la IP del *host* (10.0.128.176), APN `srsapn` y un servidor DNS externo. También se ajustó el HSS para leer la base de datos de usuarios desde `user_db.csv`.

Justificación. Los valores MCC 901 y MNC 70 corresponden a un PLMN de pruebas no asignado comercialmente, lo que permite operar sin colisionar con operadores reales. El TAC 7 se mantiene coherente con el configurado en el eNodeB para que el UE no rechace la celda por discrepancia de área. Se mantuvieron los algoritmos de seguridad por defecto (EEA0 para cifrado, EIA1 para integridad) por compatibilidad con SIMs de prueba. El APN `srsapn` es el nombre canónico de srsRAN para la portadora por defecto.

Impacto en fases posteriores. Con el EPC correctamente configurado, quedaba habilitado el punto de entrada a la red. El MCC/MNC aquí establecido debía coincidir obligatoriamente con el IMSI de la SIM del UE, lo que condicionó la elección de la tarjeta programable. Además, la interfaz virtual `srs_spgw_sgi` (172.16.0.1/24) que crea `srsepc` al arrancar se convertiría en el *gateway* de toda la red UE.

Fase 7 — Configuración del eNodeB y problema del USB 3.0. *Descripción.* En la máquina del eNodeB se editó `enb.conf` para fijar `mme_addr` apuntando al *core* (10.0.128.176), las direcciones de enlace GTP y S1C a la IP propia (10.0.128.181), el EARFCN 3350 (banda 7, DL 2680 MHz, UL 2560 MHz), 50 PRB (10 MHz de ancho de banda) y `device_name=bladeRF`. Durante las pruebas iniciales, apareció un problema: con la BladeRF conectada a un puerto USB 3.0 (azul, bus 004), el contenedor no lo detectaba, aunque el *host* sí.

Justificación. El EARFCN 3350 se eligió en banda 7 (2,6 GHz), una banda FDD muy extendida en Europa y bien soportada por los módems comerciales. La configuración de 50 PRB (10 MHz) ofrece un equilibrio razonable entre capacidad y carga de procesamiento para la BladeRF. En cuanto al problema del USB 3.0, se diagnosticó que Docker monta `/dev/bus/usb` de forma estática al arrancar el contenedor: si el bus 004 (USB 3.0) existe al arrancar se ve, si no, no se propaga dinámicamente. Tras probar sin éxito el paso `--device` explícito, se optó por la solución pragmática de conectar la BladeRF a un puerto USB 2.0 (negro, bus 001 o 002), que el contenedor sí detectaba. Alternativamente funcionó montar `/dev:/dev` completo, pero se consideró menos seguro.

Impacto en fases posteriores. La elección del puerto USB 2.0 implica una limitación teórica de ancho de banda USB, pero en la práctica no afectó al rendimiento porque 10 MHz de LTE no satura un USB 2.0. Con el eNodeB operativo y conectado al EPC por S1AP, la red LTE estaba desplegada y lista para aceptar UEs.

Fase 8 — Registro de la SIM en el HSS. *Descripción.* Se añadió la tarjeta SIM del módem Huawei del vehículo al fichero `user_db.csv` con su IMSI (901700000052128), clave Ki, OPc, algoritmo MILENAGE y asignación dinámica de IP. Al intentar la primera conexión, se detectó que el UE era rechazado con un código de error, lo que llevó a revisar que el MCC/MNC de la red coincidiera con el del IMSI de la SIM.

Justificación. La autenticación MILENAGE es el algoritmo estándar de 3GPP y más seguro que el XOR de pruebas, por lo que se prefirió, aunque requiera disponer de la clave OPc exacta. El IMSI 901700000052128 comienza por 901 (MCC) y 70 (MNC), por lo que la red LTE debía configurarse con esos mismos valores para que el UE aceptara la celda. Se actualizó entonces tanto `epc.conf` como `enb.conf` con MCC=901 y MNC=70, descartando los valores iniciales 001/01. Este detalle es una fuente habitual de fallos en redes privadas: la SIM y la red deben declarar el mismo PLMN.

Impacto en fases posteriores. Una vez corregido el MCC/MNC, el flujo `Attach Request` → `Authentication` → `Security Mode` → `Attach Complete` se completó correctamente y el UE recibió la IP 172.16.0.2 del rango UE. Con esto quedaba validada la conectividad LTE extremo a extremo a nivel de señalización.

Fase 9 — NAT y salida a Internet. *Descripción.* Para que los UEs conectados (172.16.0.0/24) pudieran alcanzar Internet, se configuraron reglas `iptables` en el *host* del *core*: `MASQUERADE` sobre la interfaz física `eno1` para las IPs del rango UE y reglas `FORWARD` en ambos sentidos. Adicionalmente, se activó el reenvío IP en el kernel mediante `net.ipv4.ip_forward=1`.

Justificación. Sin NAT, los UEs tendrían conectividad al *gateway* 172.16.0.1 pero no podrían salir a Internet porque la red 172.16.0.0/24 no es enrutable públicamente. `MASQUERADE` es la opción natural en IP dinámica porque reescribe la IP origen con la actual del *host* sin necesidad de fijarla. Se decidió aplicar las reglas en el *host* y no dentro del contenedor porque con `--network host` ambos comparten la pila de red y es más limpio gestionar el *firewall* en el anfitrión.

Impacto en fases posteriores. Con NAT funcionando, el UE podía resolver DNS y hacer `ping` a 8.8.8.8 a través de la red LTE, lo que demostró la operatividad completa de la red celular. Además, permitió al vehículo descargar actualizaciones o recursos externos si fuera necesario durante la operación.

Fase 10 — Instalación del *broker* MQTT. *Descripción.* Se instaló Mosquitto dentro del contenedor del *core* con una configuración mínima (`listener 1883 0.0.0.0` y `allow_anonymous true`) en `/etc/mosquitto/conf.d/default.conf`. Se creó el directorio `/run/mosquitto` con propietario `mosquitto` para que pudiera escribir su fichero PID, y se arrancó con `mosquitto -c mosquitto.conf -d`.

Justificación. Mosquitto se eligió por ser el *broker* MQTT más extendido y ligero, suficiente para el tráfico de telemetría y comandos del vehículo. Se optó por ubicarlo dentro del contenedor del *core* (y no en el *host*) para mantener toda la infraestructura de red encapsulada en la imagen Docker, facilitando su distribución y réplica. La escucha en 0.0.0.0 permite conexiones desde cualquier interfaz, lo que es necesario porque el vehículo se conecta a través de la red LTE y el *broker* debe estar accesible en 172.16.0.1 (*gateway* del UE) o en la IP pública del *host*. Se permitió acceso anónimo por simplicidad en el entorno de laboratorio, decisión que debería revisarse para un despliegue productivo.

Impacto en fases posteriores. El *broker* operativo era requisito indispensable para la señalización del vehículo: sin MQTT no hay canal de comandos AM-Cloud, AM-Local o AM-Off del protocolo ARTEMIS del DeepRacer. Además, se convirtió en el punto donde recibir la telemetría del IMU, batería y estado de conexión.

Fase 11 — Configuración del DeepRacer. *Descripción.* En el vehículo DeepRacer se editó el fichero `vehicle.conf` indicando `vehicle_id=3`, `mqtt_server_ip` y `cloud_server_ip` apuntando al *gateway* 172.16.0.1, puerto MQTT 1883 y puerto *cloud* 20003. Se activó el módem USB Huawei mediante `usb_modeswitch` y se reinició el servicio ARTEMIS.

Justificación. El módem Huawei utilizado arranca por defecto en modo *mass storage* (simulando una unidad USB con *drivers*), lo que impide que el sistema lo vea como interfaz de red. El comando `usb_modeswitch -v 12d1 -p 1f01 -J` envía una secuencia específica que fuerza el cambio a modo módem, creando una interfaz de red que negocia con la red LTE privada. Se eligió 172.16.0.1 como IP de destino (*gateway* UE) en lugar de la IP pública del *core* (10.0.128.176) porque; desde el punto de vista del vehículo, el *broker* está en el otro extremo de la red LTE y debe alcanzarse a través de ella. El servicio ARTEMIS es el *software* cliente que gestiona la conexión MQTT y el control del vehículo; su reinicio asegura que tome los nuevos parámetros del `vehicle.conf`.

Impacto en fases posteriores. Tras esta configuración, el vehículo se conectó con éxito a la red LTE y al *broker* MQTT, indicándolo con la luz amarilla definida en el protocolo ARTEMIS. Se verificó en el *core* la recepción de mensajes de estado y configuración publicados por el vehículo, lo que confirmó que el canal de control estaba establecido.

Fase 12 — Servidor de control por UDP. *Descripción.* Se implementó un servidor Python (`manual_control_server.py`) que escucha en UDP sobre el puerto 20003 en la IP 172.16.0.1. El servidor recibe del vehículo paquetes de imagen (tipo 'I'), *lidar* (tipo 'L') y batería (tipo 'B'), y responde con paquetes de control (tipo 'C') que contienen los valores de giro y acelerador. Se introdujo un hilo de control continuo que reenvía el estado de los controles cada 0,1 s para evitar que el vehículo corte la conexión por *timeout*.

Justificación. Se eligió UDP porque el vídeo en tiempo real tolera mejor la pérdida de paquetes que su retransmisión, y la baja latencia es prioritaria sobre la fiabilidad. El formato binario de los paquetes (1 *byte* de tipo + *payload* serializado con `pickle`) es el protocolo definido por los *scripts* originales de ARTEMIS en el vehículo, que no se podían modificar. El hilo de control periódico fue una adición clave: el *script* original solo enviaba control cuando llegaba un *frame* de imagen, y si el vehículo detectaba 200 ms sin control, respondía parando el motor, lo que causaba paradas intermitentes. Enviando control cada 100 ms se garantiza que nunca expire el *timer* del vehículo.

Impacto en fases posteriores. Con el servidor UDP operativo y el hilo de control continuo funcionando, el vehículo respondía a los comandos de teclado de forma estable. La visualización OpenCV requirió configurar el DISPLAY y permitir conexiones X11 desde Docker mediante `xhost +`, así como deshabilitar MIT-SHM con `QT_X11_NO_MITSHM=1` para evitar errores `BadAccess` del servidor X.

Fase 13 — Persistencia de contenedores y distribución. *Descripción.* Para preservar el estado de los contenedores tras cada sesión de trabajo se creó una imagen Docker congelada mediante `docker commit`, que posteriormente se exportó a un archivo `.tar.gz` mediante `docker save | gzip`. Estos archivos (de aproximadamente 650 MiB cada uno) sirvieron para transferir el entorno completo entre máquinas mediante `scp`.

Justificación. Los contenedores Docker, por defecto, no persisten los cambios realizados durante la ejecución si no se usan volúmenes nombrados. `docker commit` convierte el estado actual del contenedor en una nueva imagen reutilizable, lo que permite reiniciar el sistema sin perder la compilación de srsRAN, BladeRF, Mosquitto y el resto de *software* instalado. La compresión `gzip` reduce significativamente el tamaño del archivo y facilita el transporte por red. Se optó por esta estrategia en lugar de usar un `Dockerfile` porque el entorno incluye descargas externas (*bitstream* FPGA, SIM configurada en HSS) y ajustes manuales que habrían complicado la reproducción automatizada. La automatización con `Dockerfile` y `docker-compose` se contempla como línea futura.

Impacto en fases posteriores. La imagen persistente permitió replicar el entorno en distintas máquinas y recuperar el sistema rápidamente tras reinicios o apagones, aspecto clave para un proyecto de laboratorio con múltiples sesiones de trabajo y cambios de emplazamiento.

Conclusiones del despliegue del laboratorio

El despliegue por fases descrito demuestra la viabilidad de montar una red LTE privada completamente funcional sobre *hardware* de bajo coste (BladeRF + dos PCs de sobremesa) y *software* libre (srsRAN, Mosquitto, Python). Los tres componentes principales (`srsepc` como núcleo, `srsenb` como estación base con BladeRF como frontal radio, y Mosquitto como *broker* MQTT) quedaron integrados dentro de contenedores Docker portables y replicables.

La decisión técnica más determinante del proyecto fue, sin duda, la sustitución de Docker Desktop por Docker Engine nativo. Sin ella, el acceso al SDR BladeRF desde el contenedor ha-

bría sido imposible y el proyecto no habría llegado a materializarse. Esta experiencia subraya la importancia de comprender las diferencias entre las distintas distribuciones de una misma tecnología cuando se trabaja con *hardware* especializado. En segundo lugar, la elección de `--network host` sobre la red *bridge* por defecto de Docker fue clave para permitir la comunicación S1AP entre las dos máquinas y simplificó enormemente la configuración del *broker* MQTT y del servidor UDP.

Durante el desarrollo se identificaron varios puntos críticos que conviene destacar como aprendizajes:

- La necesidad de hacer coincidir el MCC/MNC de la red con el IMSI de la SIM.
- La incompatibilidad del BladeRF con ciertos puertos USB 3.0 dentro de Docker.
- La necesidad de cambiar el modo de operación del módem Huawei mediante `usb_modeswitch`.
- La importancia de mantener un flujo continuo de paquetes de control UDP para evitar que el vehículo corte la conexión por *timeout*.

Cada uno de estos obstáculos requirió un diagnóstico cuidadoso y la consulta de *logs* detallados, reforzando la importancia de la observabilidad en sistemas distribuidos.

5.3.4. Comandos de arranque

La secuencia operativa para levantar la red en cada sesión de laboratorio se ha automatizado en los siguientes *snippets*:

Inicio del eNodeB:

```
docker start 4Genode
docker exec -it 4Genode bash

# Dentro del contenedor (esperar a que el core muestre "SP-GW Initialized"):
bladeRF-cli -l /tmp/hostedxA9.rbf
sysctl -w net.ipv4.ip_forward=1
srsenb ~/.config/srsran/enb.conf
```

Inicio del core:

```
docker start 4Gcore
docker exec -it 4Gcore bash

# Dentro del contenedor:
sysctl -w net.ipv4.ip_forward=1
mkdir -p /run/mosquitto && chown mosquitto:mosquitto /run/mosquitto
cd /etc/mosquitto && mosquitto -c mosquitto.conf -d
srsepc ~/.config/srsran/epc.conf

# En otra terminal:
docker exec -it 4Gcore bash
python3 /coche_autonomo.py

# En una tercera terminal:
docker exec -it 4Gcore bash
mosquitto_pub -h localhost -t 3/command -m "AM-Cloud"
```

NAT en el host del core:

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo iptables -t nat -A POSTROUTING -s 172.16.0.0/24 -o eno1 -j MASQUERADE
sudo iptables -A FORWARD -s 172.16.0.0/24 -j ACCEPT
sudo iptables -A FORWARD -d 172.16.0.0/24 -j ACCEPT
xhost +
```

5.3.5. Esquema final del laboratorio

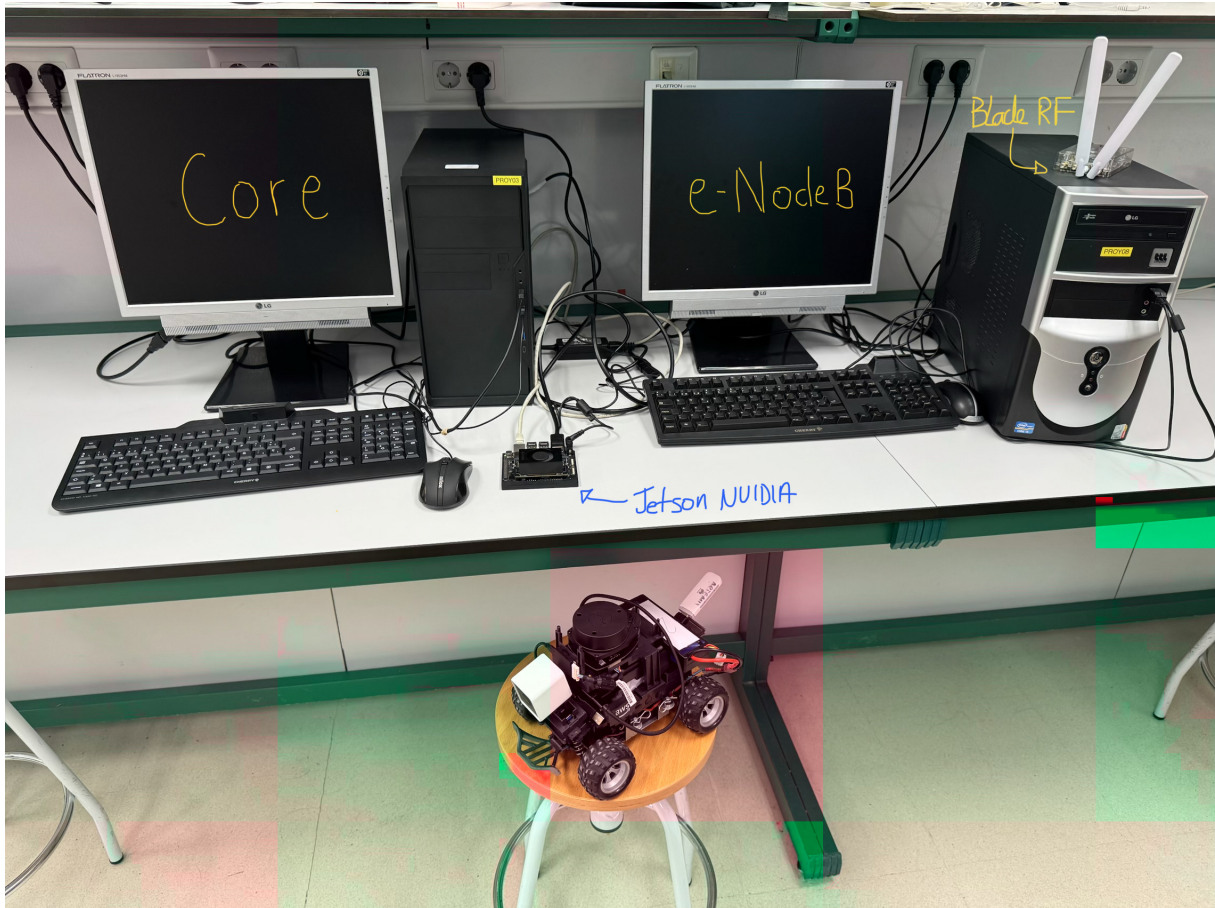


Figura 5.14: Esquema laboratorio

6 Rendimiento de la red 4G

En este capítulo se documentan las mediciones de rendimiento obtenidas con la red LTE de laboratorio, así como del ordenador del *core*, encargado de la gestión de la red y el coche. Los parámetros de interés son la latencia, el *jitter*, la capacidad disponible (*throughput*) en ambos sentidos y el comportamiento del enlace bajo carga, especialmente en presencia del flujo de vídeo del vehículo. Las pruebas se han realizado tanto con el DeepRacer estático como con el DeepRacer en movimiento dentro del laboratorio.

6.1. Procedimiento de prueba

El procedimiento de prueba sigue un protocolo común para todas las sesiones:

1. Arranque del *core* (**srsepc**) en la máquina con IP 10.0.128.176 y verificación de la inicialización del SP-GW.
2. Arranque del eNodeB (**srsenb**) en la máquina con IP 10.0.128.181 con la BladeRF conectada.
3. Encendido del DeepRacer y conexión del módem Huawei.
4. Ejecución del cambio de modo del módem (**usb_modeswitch**).
5. Verificación del proceso de *attach*: **Attach Request** → **Authentication** → **Security Mode** → **Attach Complete**.
6. Verificación de la asignación de IP al UE (típicamente 172.16.0.2).
7. Verificación de conectividad a Internet mediante el comando **ping 8.8.8.8** desde el DeepRacer.
8. Lanzamiento del *script* de control en el *core* y medición.
- 9a. Lanzamiento del servidor del programa de medición **iperf3** en el ordenador del Core.
- 10a. Ejecución de **iperf3** en el cliente (*Deepracer*) para a continuación medir las tasas de transferencia y pérdidas.
- 9b. Lanzamiento del programa **ping** con el objetivo de medir la latencia y el jitter.

6.2. Capacidad disponible: estimación teórica

Antes de las mediciones experimentales, conviene recordar la capacidad teórica calculada en el Capítulo 5. Partiendo de la estructura física de la trama LTE (100 PRB para 20 MHz, 12 subportadoras por PRB, 14 símbolos por subtrama y 16-QAM), la tasa bruta es de 67,2 Mbit/s y, descontando un 30% de *overhead* (DMRS, control y codificación de canal), la capacidad neta del enlace ascendente es de aproximadamente:

$$\text{Throughput}_{UL,\text{teórico}} \approx 47 \text{ Mbit/s}$$

Este valor corresponde al caso conservador de una única capa espacial (sin ganancia MIMO en UL). En el demostrador con BladeRF y antenas omnidireccionales de baja ganancia, la potencia disponible es muy inferior a la de un eNodeB comercial, por lo que las modulaciones efectivas alcanzadas son normalmente más bajas (QPSK, o 16-QAM solo en condiciones muy favorables) y el *throughput* medido se reduce en consecuencia.

6.3. Mediciones de la capacidad de la red

6.3.1. Funcionamiento y limitaciones de `iperf3`

`iperf3` es una herramienta de software libre y de línea de comandos utilizada para medir el rendimiento máximo de ancho de banda en redes IP. Si bien no se trata de la herramienta más profesional disponible sí es bastante mejor para nuestro propósito que los test de velocidad en web ya que elimina las afectaciones que puedan causar el navegador o los distintos servidores web a los que te conectes o saltos que atraveses durante la conexión[Gúeant, 2017].

Para que la herramienta funcione hacen falta dos dispositivos conectados a la red cuyo rendimiento queremos medir. Uno actuará de servidor quedándose a la escucha. El otro, de cliente, se conectará al servidor y enviará los paquetes o dará la orden de que el servidor se los envíe en caso de usar el parámetro *reverse*.

Esta estructura de funcionamiento puede suponer que las mediciones se vean ligeramente afectadas ya que estamos ejecutando `iperf3` y el software del *core* de la red desde el mismo PC, lo que podría limitar el rendimiento a altas velocidades. De todos modos, como se verá en la Figura 6.9, esto no debería suponer un problema en nuestro caso ya que la carga de la CPU y de la memoria no son elevadas. Además hay que sumar que las velocidades de transmisión en nuestra red no son suficientes como para saturar la CPU.

Por otra parte hay varias razones por las que se pueden obtener medidas más altas que las reales, superando incluso el límite teórico de la red. Los motivos principales son los siguientes:

1. Descarga de red (*offloading*): Las tarjetas de red modernas utilizan las tecnologías TSO (*TCP Segmentation Offload*) y LRO (*Large Receive Offload*) para procesar internamente los paquetes y liberar a la CPU. Esto puede provocar que el sistema operativo esté midiendo la velocidad a la que entrega los paquetes a la tarjeta de red de forma interna y no la velocidad a la que esta los está transmitiendo a la red.
2. Compresión de datos en la ruta: Si en la conexión se utiliza un enlace con compresión de datos en tiempo real (por ejemplo en un túnel VPN), la información se puede estar enviando comprimida pero el SO la mide una vez se haya descomprimido, pudiendo resultar en medidas varias veces mayores que la capacidad real. Esto no ocurre en nuestra red ya que en ningún enlace hay compresión de datos.
3. Efecto ráfaga (*buffering*): Al iniciar la prueba, la memoria caché absorbe datos a máxima velocidad hasta saturarse. Esto puede generar un pico inicial que falsee el promedio, especialmente en pruebas cortas.

6.3.2. Mediciones con TCP

Para iniciar las mediciones, primero se lanza en el servidor el comando siguiente:

```
$ iperf3 -s -p 8000
```

En el coche, para iniciar la conexión y con ello la prueba ejecutamos :

```
$ iperf3 -c 172.16.0.1 -p 8000
```

Para medir la capacidad en sentido contrario (*Downlink*) le añadimos el parámetro *reverse* al comando:

```
$ iperf3 -c 172.16.0.1 -p 8000 -R
```

Así, queda establecida la conexión TCP entre cliente (coche) y servidor.

En las pruebas de subida y bajada realizadas con un CQI de valor 15 (el máximo posible) los resultados muestran unas tasas de transferencia de 22,4 Mbit/s en el *uplink* y de 35,3 Mbit/s

en el *downlink*.

Como era de esperar los resultados muestran una tasa de bajada mayor que la tasa de subida, característica habitual en el diseño de las redes móviles debido a que la demanda de tráfico de bajada suele ser mayor que de subida.

```
deepracer@amss-zp4t:~$ iperf3 -c 172.16.0.1 -p 8000 -R
Connecting to host 172.16.0.1, port 8000
Reverse mode, remote host 172.16.0.1 is sending
[ 4] local 192.168.8.105 port 38934 connected to 172.16.0.1 port 8000
[ ID] Interval      Transfer      Bandwidth
[ 4] 0.00-1.00 sec  3.91 MBytes  32.8 Mbits/sec
[ 4] 1.00-2.00 sec  4.20 MBytes  35.2 Mbits/sec
[ 4] 2.00-3.00 sec  4.20 MBytes  35.2 Mbits/sec
[ 4] 3.00-4.00 sec  4.20 MBytes  35.2 Mbits/sec
[ 4] 4.00-5.00 sec  4.20 MBytes  35.2 Mbits/sec
[ 4] 5.00-6.00 sec  4.20 MBytes  35.2 Mbits/sec
[ 4] 6.00-7.00 sec  4.20 MBytes  35.2 Mbits/sec
[ 4] 7.00-8.00 sec  4.20 MBytes  35.2 Mbits/sec
[ 4] 8.00-9.00 sec  4.20 MBytes  35.2 Mbits/sec
[ 4] 9.00-10.00 sec 4.20 MBytes  35.2 Mbits/sec
[ ID] Interval      Transfer      Bandwidth      Retr
[ 4] 0.00-10.00 sec 44.0 MBytes  36.9 Mbits/sec    19
[ 4] 0.00-10.00 sec 42.1 MBytes  35.3 Mbits/sec
iperf Done.
```

(a) *Downlink*.

```
deepracer@amss-zp4t:~$ iperf3 -c 172.16.0.1 -p 8000
Connecting to host 172.16.0.1, port 8000
[ 4] local 192.168.8.105 port 38826 connected to 172.16.0.1 port 8000
[ ID] Interval      Transfer      Bandwidth      Retr Cwnd
[ 4] 0.00-1.00 sec  2.63 MBytes  22.0 Mbits/sec    0   191 KBytes
[ 4] 1.00-2.00 sec  2.78 MBytes  23.3 Mbits/sec    0   322 KBytes
[ 4] 2.00-3.00 sec  2.76 MBytes  23.2 Mbits/sec    0   455 KBytes
[ 4] 3.00-4.00 sec  2.69 MBytes  22.6 Mbits/sec    0   588 KBytes
[ 4] 4.00-5.00 sec  2.94 MBytes  24.7 Mbits/sec    0   720 KBytes
[ 4] 5.00-6.00 sec  2.68 MBytes  22.5 Mbits/sec    0   853 KBytes
[ 4] 6.00-7.00 sec  2.67 MBytes  22.4 Mbits/sec    0   984 KBytes
[ 4] 7.00-8.00 sec  2.23 MBytes  18.7 Mbits/sec    0  1.09 MBytes
[ 4] 8.00-9.00 sec  2.67 MBytes  22.4 Mbits/sec    0  1.22 MBytes
[ 4] 9.00-10.00 sec 2.68 MBytes  22.5 Mbits/sec    0  1.30 MBytes
[ ID] Interval      Transfer      Bandwidth      Retr
[ 4] 0.00-10.00 sec 26.7 MBytes  22.4 Mbits/sec    0
[ 4] 0.00-10.00 sec 26.7 MBytes  22.4 Mbits/sec
iperf Done.
```

(b) *Uplink*.

Figura 6.1: Prueba iperf3 para el protocolo TCP y con CQI 15.

Si realizamos estas pruebas cuando el CQI tiene un valor más realista para el entorno en el que vamos a operar (se obtuvo un valor de 11 de forma bastante estable), se obtienen unas velocidades de 15,4 Mbit/s en el canal de *uplink* y de 23,9 Mbit/s en el de *downlink*.

En este caso los resultados muestran una relación similar entre las tasas de bajada y de subida a aquella del caso anterior. También muestran como ambas tasas son aproximadamente un 50 % más bajas que en condiciones ideales.

-----DL-----										-----UL-----									
rat	pci	rnti	cqi	ri	mcs	brat	ok	nok	(%)	pusch	pucch	phr	mcs	brat	ok	nok	(%)	bsr	
lte	1	47	11	0	23	23M	922	75	7%	20.8	31.9	7	22	8.0M	391	0	0%	0.0	
lte	1	47	11	0	22	23M	947	47	4%	21.9	32.6	9	22	8.1M	391	0	0%	0.0	
lte	1	47	12	0	25	26M	907	87	8%	20.0	31.5	10	22	7.7M	373	0	0%	16.2k	
lte	1	47	11	0	21	21M	984	15	1%	22.0	32.9	7	23	8.1M	385	1	0%	0.0	
lte	1	47	11	0	22	23M	951	48	4%	21.7	32.6	7	22	8.0M	397	0	0%	0.0	
lte	1	47	11	0	21	21M	936	62	6%	17.7	30.0	8	21	7.6M	397	2	0%	1.22k	
lte	1	47	11	0	22	11M	466	6	1%	19.0	30.0	11	22	7.3M	371	1	0%	0.0	

Figura 6.2: Captura de las medidas de CQI al realizar las pruebas a una distancia realista.

```
root@grupo2core-System-Product-Name:~$ iperf3 -s -p 8000
Server listening on 8000
Accepted connection from 172.16.0.3, port 55340
[ 5] local 172.16.0.1 port 8000 connected to 172.16.0.3 port 55342
[ ID] Interval      Transfer      Bitrate      Retr  Cwnd
[ 5] 0.00-1.00 sec  3.33 MBytes  27.9 Mbits/sec    0   232 KBytes
[ 5] 1.00-2.00 sec  3.11 MBytes  26.1 Mbits/sec    0   354 KBytes
[ 5] 2.00-3.00 sec  3.36 MBytes  28.2 Mbits/sec   12   308 KBytes
[ 5] 3.00-4.00 sec  2.55 MBytes  21.4 Mbits/sec    0   356 KBytes
[ 5] 4.00-5.00 sec  3.42 MBytes  28.7 Mbits/sec    0   386 KBytes
[ 5] 5.00-6.00 sec  2.49 MBytes  20.9 Mbits/sec    3   288 KBytes
[ 5] 6.00-7.00 sec  2.67 MBytes  22.4 Mbits/sec    0   313 KBytes
[ 5] 7.00-8.00 sec  2.55 MBytes  21.4 Mbits/sec    0   328 KBytes
[ 5] 8.00-9.00 sec  2.61 MBytes  21.9 Mbits/sec    0   335 KBytes
[ 5] 9.00-10.00 sec 2.49 MBytes  20.9 Mbits/sec    0   335 KBytes
[ ID] Interval      Transfer      Bitrate      Retr
[ 5] 0.00-10.04 sec 28.6 MBytes  23.9 Mbits/sec   15
sender
```

(a) *Downlink*.

```
Server listening on 8000
Accepted connection from 172.16.0.3, port 52150
[ 5] local 172.16.0.1 port 8000 connected to 172.16.0.3 port 52152
[ ID] Interval      Transfer      Bitrate
[ 5] 0.00-1.11 sec  1.75 MBytes  13.2 Mbits/sec
[ 5] 1.11-2.09 sec  1.25 MBytes  10.7 Mbits/sec
[ 5] 2.09-3.11 sec  1.50 MBytes  12.3 Mbits/sec
[ 5] 3.11-4.08 sec  1.75 MBytes  15.1 Mbits/sec
[ 5] 4.08-5.09 sec  2.12 MBytes  17.6 Mbits/sec
[ 5] 5.09-6.11 sec  2.12 MBytes  17.6 Mbits/sec
[ 5] 6.11-7.10 sec  2.12 MBytes  18.0 Mbits/sec
[ 5] 7.10-8.09 sec  2.12 MBytes  18.0 Mbits/sec
[ 5] 8.09-9.08 sec  2.12 MBytes  17.9 Mbits/sec
[ 5] 9.08-10.12 sec 2.00 MBytes  16.2 Mbits/sec
[ 5] 10.12-10.74 sec 896 KBytes  11.9 Mbits/sec
[ ID] Interval      Transfer      Bitrate
[ 5] 0.00-10.74 sec 19.8 MBytes  15.4 Mbits/sec
receiver
```

(b) *Uplink*.

Figura 6.3: Prueba iperf3 para el protocolo TCP y con CQI 11.

Como se ha observado, con un CQI máximo la velocidad medida es aproximadamente un 5 % superior al máximo teórico. Esta diferencia no es muy significativa y puede deberse al añadido de bits de relleno (*padding*) o a pequeñas asincronías o desviaciones en los relojes o el oscilador de la BladeRF.

6.3.3. Mediciones con UDP

En la práctica, el protocolo que se utiliza para el envío de video y de la información de control entre el *deepracer* y el controlador es UDP, por lo que resulta fundamental realizar pruebas de rendimiento con este protocolo.

Al tratarse de un protocolo **no orientado a conexión** la tasa de transmisión no se regula sola segun la capacidad efectiva de la red, si no que se pueden enviar todos los paquetes que se deseen pero sin ninguna garantia de alcanzar el destino. Por ello estas pruebas de rendimiento consisten en forzar el envío de grandes cantidades de datos y comprobar a partir de que punto las perdidas son demasiado elevadas como para que la información transmitida sea usable.

Para realizar las pruebas se lanza el servidor igual que en las pruebas con TCP y posteriormente se lanza en el cliente el comando:

```
iperf3 -c 172.16.0.1 -p 8000 -u -b 16M -l 16KB
```

donde el parametro `-u` indica el uso del protocolo UDP, `-b` permite cambiar la tasa binaria y `-l` permite modificar el tamaño de los paquetes.

En las pruebas se ha realizado para cada sentido (*uplink* y *downlink* con el parámetro `-R`) una prueba a 16 Mbit/s y dos pruebas a 20 Mbit/s con el objetivo de comprobar la variabilidad de los resultados.

Con un CQI estable en torno a un valor de 10, se observa que hasta 16 Mbit/s la comunicación apenas sufre pérdida de paquetes (0,24% en *uplink* y 0% en el *downlink*).

```

iperf done
deepracer@ms-zp4t1:~$ iperf3 -c 172.16.0.1 -p 8000 -u -b 16M -l 16KB -R
Connecting to host 172.16.0.1, port 8000
Reverse mode, remote host 172.16.0.1 is sending
[ 4] local 192.168.8.105 port 42344 connected to 172.16.0.1 port 8000
[ ID] Interval      Transfer     Bandwidth   Jitter    Lost/Tot.  Datagrams
[ 4] 0.00-1.00    sec  1.75 MBytes  14.7 Mbits/sec  86052317.273 ms  3/115 (2.6%)
[ 4] 1.00-2.00    sec  2.05 MBytes  17.2 Mbits/sec  18325.360 ms  0/131 (0%)
[ 4] 2.00-3.00    sec  1.91 MBytes  16.0 Mbits/sec  8.527 ms  0/122 (0%)
[ 4] 3.00-4.00    sec  1.91 MBytes  16.0 Mbits/sec  4.475 ms  0/122 (0%)
[ 4] 4.00-5.00    sec  1.91 MBytes  16.0 Mbits/sec  6.108 ms  0/122 (0%)
[ 4] 5.00-6.00    sec  1.86 MBytes  15.6 Mbits/sec  6.966 ms  0/119 (0%)
[ 4] 6.00-7.00    sec  1.95 MBytes  16.4 Mbits/sec  8.874 ms  0/125 (0%)
[ 4] 7.00-8.00    sec  1.88 MBytes  15.7 Mbits/sec  3.436 ms  0/120 (0%)
[ 4] 8.00-9.00    sec  1.92 MBytes  16.1 Mbits/sec  5.034 ms  0/123 (0%)
[ 4] 9.00-10.00   sec  1.91 MBytes  16.0 Mbits/sec  5.595 ms  0/122 (0%)
[ ID] Interval      Transfer     Bandwidth   Jitter    Lost/Tot.  Datagrams
[ 4] 0.00-10.00   sec  19.2 MBytes  16.1 Mbits/sec  5.616 ms  3/1225 (0.24%)
[ 4] Sent 1225 datagrams

iperf Done.

```

(a) Downlink.

```

Server listening on 8000
Accepted connection from 172.16.0.3, port 59264
[ 5] local 172.16.0.1 port 8000 connected to 172.16.0.3 port 44641
[ ID] Interval      Transfer     Bitrate     Jitter    Lost/Tot.  Datagrams
[ 5] 0.00-1.00    sec  1.66 MBytes  13.9 Mbits/sec  11.533 ms  0/106 (0%)
[ 5] 1.00-2.00    sec  1.94 MBytes  16.2 Mbits/sec  9.942 ms  0/124 (0%)
[ 5] 2.00-3.00    sec  1.80 MBytes  15.1 Mbits/sec  12.769 ms  0/115 (0%)
[ 5] 3.00-4.00    sec  1.97 MBytes  16.5 Mbits/sec  10.198 ms  0/126 (0%)
[ 5] 4.00-5.00    sec  1.84 MBytes  15.5 Mbits/sec  12.494 ms  0/118 (0%)
[ 5] 5.00-6.00    sec  1.98 MBytes  16.6 Mbits/sec  10.926 ms  0/127 (0%)
[ 5] 6.00-7.00    sec  1.92 MBytes  16.1 Mbits/sec  9.749 ms  0/123 (0%)
[ 5] 7.00-8.00    sec  1.92 MBytes  16.1 Mbits/sec  9.513 ms  0/123 (0%)
[ 5] 8.00-9.00    sec  1.92 MBytes  16.1 Mbits/sec  9.561 ms  0/123 (0%)
[ 5] 9.00-10.00   sec  1.89 MBytes  15.9 Mbits/sec  9.524 ms  0/121 (0%)
[ 5] 10.00-10.05  sec  64.0 KBytes  11.3 Mbits/sec  8.583 ms  0/4 (0%)
[ ID] Interval      Transfer     Bitrate     Jitter    Lost/Tot.  Datagrams
[ 5] 0.00-10.05   sec  18.9 MBytes  15.8 Mbits/sec  8.583 ms  0/1210 (0%) receiver

```

(b) Uplink.

Figura 6.4: Prueba iperf3 para el protocolo UDP y una tasa de transmisión de 16 Mbit/s.

Sin embargo para tasas de transmisión forzadas a 20 Mbit/s las pérdidas son muy altas (en varios casos ni siquiera se alcanza una tasa de transmisión efectiva de 16 Mbit/s) y además hay una enorme variabilidad en resultados como se observa en las Figuras 6.5 y 6.6:

```

iperf done
deepracer@ms-zp4t1:~$ iperf3 -c 172.16.0.1 -p 8000 -u -b 20M -l 16KB -R
Connecting to host 172.16.0.1, port 8000
Reverse mode, remote host 172.16.0.1 is sending
[ 4] local 192.168.8.105 port 42823 connected to 172.16.0.1 port 8000
[ ID] Interval      Transfer     Bandwidth   Jitter    Lost/Tot.  Datagrams
[ 4] 0.00-1.00    sec  2.48 MBytes  20.8 Mbits/sec  4143938.877 ms  0/159 (0%)
[ 4] 1.00-2.00    sec  2.36 MBytes  19.8 Mbits/sec  250.899 ms  0/151 (0%)
[ 4] 2.00-3.00    sec  2.17 MBytes  18.2 Mbits/sec  5.139 ms  0/130 (0%)
[ 4] 3.00-4.00    sec  1.67 MBytes  14.0 Mbits/sec  5.329 ms  41/148 (28%)
[ 4] 4.00-5.00    sec  1.25 MBytes  10.5 Mbits/sec  5.590 ms  79/159 (50%)
[ 4] 5.00-6.00    sec  2.58 MBytes  21.6 Mbits/sec  5.448 ms  0/165 (0%)
[ 4] 6.00-7.00    sec  2.38 MBytes  19.9 Mbits/sec  6.612 ms  0/152 (0%)
[ 4] 7.00-8.00    sec  2.41 MBytes  20.2 Mbits/sec  4.750 ms  0/154 (0%)
[ 4] 8.00-9.00    sec  2.41 MBytes  20.2 Mbits/sec  5.483 ms  0/154 (0%)
[ 4] 9.00-10.00   sec  2.19 MBytes  18.3 Mbits/sec  5.197 ms  0/140 (0%)
[ ID] Interval      Transfer     Bandwidth   Jitter    Lost/Tot.  Datagrams
[ 4] 0.00-10.00   sec  24.0 MBytes  20.1 Mbits/sec  6.038 ms  120/1536 (7.8%)
[ 4] Sent 1536 datagrams

iperf Done.

```

(a) Downlink.

```

Server listening on 8000
Accepted connection from 172.16.0.3, port 59306
[ 5] local 172.16.0.1 port 8000 connected to 172.16.0.3 port 45333
[ ID] Interval      Transfer     Bitrate     Jitter    Lost/Tot.  Datagrams
[ 5] 0.00-1.00    sec  1.12 MBytes  9.44 Mbits/sec  11.057 ms  0/72 (0%)
[ 5] 1.00-2.00    sec  1.73 MBytes  14.5 Mbits/sec  10.827 ms  26/137 (19%)
[ 5] 2.00-3.00    sec  2.05 MBytes  17.2 Mbits/sec  10.981 ms  33/164 (20%)
[ 5] 3.00-4.00    sec  1.91 MBytes  16.0 Mbits/sec  12.592 ms  29/151 (19%)
[ 5] 4.00-5.00    sec  1.64 MBytes  13.8 Mbits/sec  12.186 ms  19/124 (15%)
[ 5] 5.00-6.00    sec  1.56 MBytes  13.1 Mbits/sec  11.674 ms  52/152 (34%)
[ 5] 6.00-7.00    sec  1.25 MBytes  10.5 Mbits/sec  11.248 ms  48/128 (38%)
[ 5] 7.00-8.00    sec  1.69 MBytes  14.2 Mbits/sec  10.243 ms  89/197 (45%)
[ 5] 8.00-9.00    sec  1.53 MBytes  12.8 Mbits/sec  9.982 ms  28/126 (22%)
[ 5] 9.00-10.00   sec  1.92 MBytes  16.1 Mbits/sec  9.167 ms  66/183 (33%)
[ 5] 10.00-10.50  sec  1.89 MBytes  16.4 Mbits/sec  8.337 ms  7/77 (9.1%)
[ ID] Interval      Transfer     Bitrate     Jitter    Lost/Tot.  Datagrams
[ 5] 0.00-10.50   sec  17.5 MBytes  14.0 Mbits/sec  8.337 ms  391/1511 (26%) receiver

```

(b) Uplink.

Figura 6.5: Prueba iperf3 para el protocolo UDP y una tasa de transmisión de 20 Mbit/s.

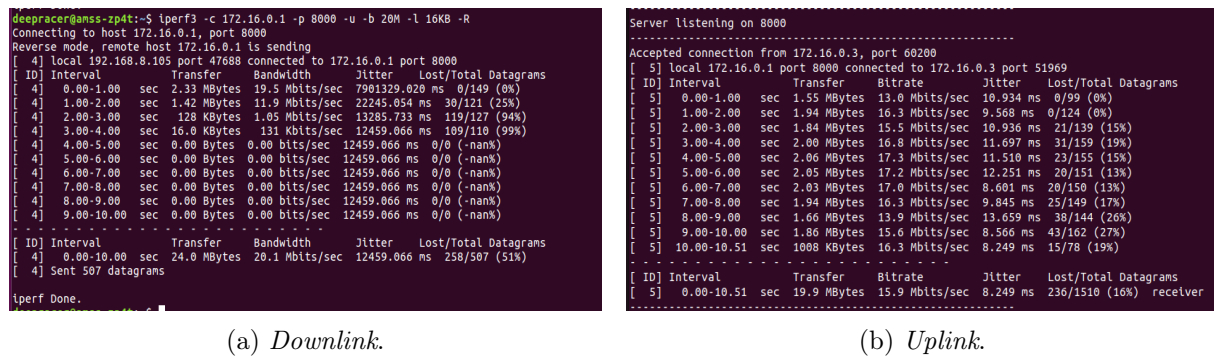


Figura 6.6: Segunda prueba iperf3 para el protocolo UDP y una tasa de transmisión de 16 Mbit/s.

6.4. Mediciones de latencia y jitter

Para medir la latencia y su varianza (*jitter*) la única posibilidad es realizar las mediciones de forma bidireccional, ya que es físicamente imposible hacerlo de forma unidireccional ya que para ello se necesitaría incorporar cabeceras con el instante de envío de los paquetes así como relojes precisos y sincronizados en los dispositivos.

En nuestro caso hemos decidido usar para las mediciones la herramienta `ping`, lanzando el programa desde el coche hasta el *core* en dos situaciones opuestas:

- Una red completamente disponible y sin ningún tráfico más allá del que genera la propia herramienta `ping` (condiciones ideales).

- Una red muy saturada debido al lanzamiento en paralelo del control del coche autónomo, lo que genera un flujo de paquetes que ocupa todo el *bitrate* disponible.

Como se observa en la Figura 6.7, en una red con amplia disponibilidad la latencia es aceptable (en torno a 20 ms). Además el *jitter* (9,2 ms) no supone un problema grave para un video limitado a 30 fps ya que el tiempo entre fotogramas es de 33,3 ms, muy superior al anterior.

Por el contrario, en la Figura 6.8 se puede ver como en una red saturada ambos parámetros harían el sistema de control del coche autónomo completamente inútil debido a que la latencia causaría un enorme retardo en la transferencia de los paquetes y además el *jitter* provocaría que los paquetes pudiesen llegar en un orden diferente a aquél en el que se generaron.

```

deepracer@amss-zp4t: ~
deepracer@amss-zp4t:~$ ping 10.0.128.176 -c 20
PING 10.0.128.176 (10.0.128.176) 56(84) bytes of data.
64 bytes from 10.0.128.176: icmp_seq=1 ttl=63 time=30.7 ms
64 bytes from 10.0.128.176: icmp_seq=2 ttl=63 time=40.2 ms
64 bytes from 10.0.128.176: icmp_seq=3 ttl=63 time=47.9 ms
64 bytes from 10.0.128.176: icmp_seq=4 ttl=63 time=48.4 ms
64 bytes from 10.0.128.176: icmp_seq=5 ttl=63 time=45.4 ms
64 bytes from 10.0.128.176: icmp_seq=6 ttl=63 time=36.4 ms
64 bytes from 10.0.128.176: icmp_seq=7 ttl=63 time=42.9 ms
64 bytes from 10.0.128.176: icmp_seq=8 ttl=63 time=34.4 ms
64 bytes from 10.0.128.176: icmp_seq=9 ttl=63 time=40.2 ms
64 bytes from 10.0.128.176: icmp_seq=10 ttl=63 time=38.9 ms
64 bytes from 10.0.128.176: icmp_seq=11 ttl=63 time=61.4 ms
64 bytes from 10.0.128.176: icmp_seq=12 ttl=63 time=38.2 ms
64 bytes from 10.0.128.176: icmp_seq=13 ttl=63 time=36.5 ms
64 bytes from 10.0.128.176: icmp_seq=14 ttl=63 time=58.4 ms
64 bytes from 10.0.128.176: icmp_seq=15 ttl=63 time=34.5 ms
64 bytes from 10.0.128.176: icmp_seq=16 ttl=63 time=56.4 ms
64 bytes from 10.0.128.176: icmp_seq=17 ttl=63 time=55.4 ms
64 bytes from 10.0.128.176: icmp_seq=18 ttl=63 time=54.4 ms
64 bytes from 10.0.128.176: icmp_seq=19 ttl=63 time=43.6 ms
64 bytes from 10.0.128.176: icmp_seq=20 ttl=63 time=58.8 ms

--- 10.0.128.176 ping statistics ---
20 packets transmitted, 20 received, 0% packet loss, time 19018ms
rtt min/avg/max/mdev = 30.747/45.190/61.402/9.203 ms
deepracer@amss-zp4t:~$

```

Figura 6.7: Ejecución de ping en condiciones de uso de red ideales.

```

deepracer@amss-zp4t: ~
64 bytes from 10.0.128.181: icmp_seq=15 ttl=62 time=1055 ms

--- 10.0.128.181 ping statistics ---
15 packets transmitted, 15 received, 0% packet loss, time 14012ms
rtt min/avg/max/mdev = 29.243/377.570/1055.728/318.012 ms
deepracer@amss-zp4t:~$ ping 10.0.128.181 -c 12
PING 10.0.128.181 (10.0.128.181) 56(84) bytes of data.
64 bytes from 10.0.128.181: icmp_seq=1 ttl=62 time=174 ms
64 bytes from 10.0.128.181: icmp_seq=2 ttl=62 time=216 ms
64 bytes from 10.0.128.181: icmp_seq=3 ttl=62 time=278 ms
64 bytes from 10.0.128.181: icmp_seq=4 ttl=62 time=316 ms
64 bytes from 10.0.128.181: icmp_seq=5 ttl=62 time=511 ms
64 bytes from 10.0.128.181: icmp_seq=6 ttl=62 time=648 ms
64 bytes from 10.0.128.181: icmp_seq=7 ttl=62 time=741 ms
64 bytes from 10.0.128.181: icmp_seq=8 ttl=62 time=755 ms
64 bytes from 10.0.128.181: icmp_seq=9 ttl=62 time=608 ms
64 bytes from 10.0.128.181: icmp_seq=10 ttl=62 time=233 ms
64 bytes from 10.0.128.181: icmp_seq=11 ttl=62 time=33.3 ms
64 bytes from 10.0.128.181: icmp_seq=12 ttl=62 time=41.8 ms

--- 10.0.128.181 ping statistics ---
12 packets transmitted, 12 received, 0% packet loss, time 11008ms
rtt min/avg/max/mdev = 33.309/379.927/755.456/250.432 ms
deepracer@amss-zp4t:~$

```

Figura 6.8: Ejecución de ping sobre una red saturada.

6.5. Mediciones de rendimiento del ordenador del *core*

La ejecución del *core* de la red así como del control autónomo del coche, ambos en el mismo ordenador, podría suponer un problema de sobrecarga para la CPU de este. Por ello, con el comando \$ `htop`, monitorizamos el uso de recursos del PC para asegurarnos de que no habría problema alguno de rendimiento como muestra la Figura 6.9 en donde se observa como la carga de los nucleos no alcanza el 20 % en el peor de los casos y la memoria tiene más de 11 GB libres.

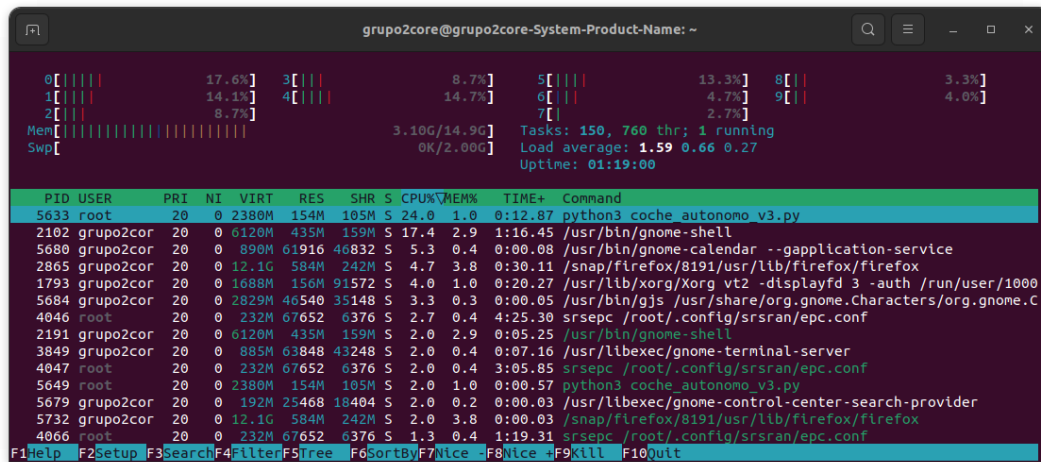


Figura 6.9: Carga de CPU y memoria durante el funcionamiento del control autónomo.

6.6. Problemas observados y soluciones aplicadas

6.6.1. Latencia elevada y cortes de vídeo

Durante las primeras pruebas se observaron picos de latencia en el flujo UDP que cortaban el vídeo en tiempo real.

El verdadero problema que causa los cortes de vídeo es la **variabilidad de la latencia** (el *jitter*): cuando el retardo de los paquetes fluctúa, el receptor no recibe los *frames* a una cadencia regular y el vídeo se entrecorta. Para diagnosticar la causa se intentó introducir una marca temporal por *frame* para conocer el orden de estos, pero el *script* ARTEMIS del DeepRacer no permite esto y modificarlo habría requerido recompilarlo, opción descartada por el riesgo que conlleva.

Como forma de mitigar el problema, se introdujo un **hilo de control continuo** en el servidor UDP que reenvía cada 100 ms el último estado de control conocido (*throttle* y *steering*). Sin este envío periódico, el DeepRacer cortaba la conexión cuando detectaba más de 200 ms sin recibir control, lo que causaba paradas intermitentes del motor. Con el hilo de control activo, los cortes se redujeron y la conducción se mantuvo estable.

6.6.2. Pérdida de conexión por interferencias

A mitad de algunas sesiones la conexión del DeepRacer se perdía sin causa aparente. La hipótesis más probable, dado que en el aula trabajaban simultáneamente otros equipos, es la **interferencia con otros equipos** operando en la misma banda.

La forma de solucionar este problema de forma permanente sería repartirse el total del espectro disponible entre los distintos grupos o, como terminamos haciendo, repartirse el tiempo de transmisión.

6.6.3. Conflicto de IPs entre la Jetson y el core

Con el *core* en 10.0.128.176 (LAN del laboratorio) y la Jetson en 192.168.172.51 (red Wi-Fi del aula), no había conectividad directa entre ambos equipos.

La solución implementada fue establecer una **conexión RJ45 directa** entre el *core* y la Jetson, configurando IPs estáticas para ambos. Esta conexión dedicada elimina la latencia y el *jitter* de la Wi-Fi compartida, garantizando una conexión estable de bajo retardo entre ambos equipos.

6.6.4. Visualización OpenCV y problema X11

Al lanzar el visor OpenCV en el contenedor Docker para mostrar las imágenes recibidas, aparecía un error `BadAccess` del servidor X. La causa era doble:

1. El contenedor no tenía permiso para conectar al servidor X del *host*. Se solucionó con `xhost +` en el *host* (con la advertencia de que esto baja la seguridad y debería revertirse tras la sesión).
2. OpenCV intentaba usar memoria compartida MIT-SHM para acelerar la visualización, lo cual no funciona desde un contenedor Docker. Se solucionó ejecutando `QT_X11_NO_MITSHM=1` antes de lanzar el visor.

6.7. Conclusiones del rendimiento

La demostración del laboratorio valida la **viabilidad funcional** del servicio de vehículo conectado sobre LTE: el flujo de control, la telemetría, el vídeo y los comandos de retorno se intercambian correctamente a través de la red LTE. La detección de señales en la Jetson y la actuación sobre el vehículo se realizan con latencias suficientes para reaccionar a las señales con la anticipación necesaria.

Las limitaciones de *throughput* y latencia observadas en el laboratorio son atribuibles en parte al hardware empleado (BladeRF de baja potencia, antenas omnidireccionales). La extrapolación al despliegue real propuesto en el Capítulo 5, con eNodeB comerciales y antenas sectoriales directivas, mejoraría claramente el rendimiento mostrado y lo llevaría a niveles de prestaciones acordes a las necesidades de funcionamiento.

7 Inteligencia Artificial

En este capítulo se aborda el bloque de inteligencia artificial del proyecto: la justificación teórica de los conceptos empleados, la elección de las herramientas, la generación y depuración de un *dataset* propio, las siete iteraciones del modelo de detección de señales hasta llegar a una versión final con métricas excelentes, y el despliegue del modelo final sobre la NVIDIA Jetson Orin Nano para inferencia *edge* en tiempo real.

7.1. Conceptos teóricos y objetivos

La inteligencia artificial es la disciplina que busca dotar a las máquinas de capacidades tradicionalmente asociadas a la inteligencia humana, como el aprendizaje, el razonamiento, la percepción o el lenguaje. Dentro de la IA, el aprendizaje automático (*machine learning*) agrupa los algoritmos que aprenden a partir de datos sin haber sido programados explícitamente para cada tarea. Y dentro del aprendizaje automático, el aprendizaje profundo (*deep learning*) utiliza redes neuronales artificiales con múltiples capas para extraer representaciones jerárquicas de los datos.

Los tres conceptos — IA, ML y DL — se relacionan jerárquicamente: el aprendizaje profundo es un subconjunto del aprendizaje automático, que a su vez es un subconjunto de la inteligencia artificial. La detección de objetos en imágenes es un problema clásico de visión por ordenador que, en los últimos años, ha sido resuelto de forma cada vez más eficiente mediante redes neuronales convolucionales (CNN), un tipo específico de arquitectura de aprendizaje profundo especializada en datos con estructura espacial.

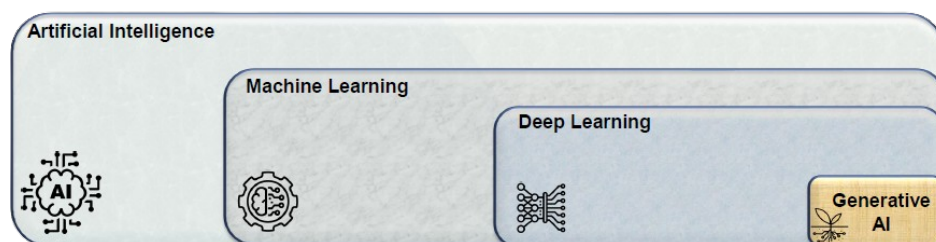


Figura 7.1: Subcategorías de la Inteligencia Artificial.

7.1.1. Redes neuronales convolucionales (CNN)

Una red neuronal convolucional aplica filtros (*kernels*) que se desplazan por la imagen extrayendo características locales: bordes, esquinas, texturas. Las primeras capas detectan rasgos básicos; las capas siguientes combinan esos rasgos en estructuras más complejas; las últimas capas identifican objetos completos. Las CNN se entrenan minimizando una función de coste que mide la discrepancia entre las predicciones de la red y las etiquetas reales del *dataset*, ajustando los pesos de los filtros mediante descenso de gradiente con *backpropagation* [Durán Barroso y de Miguel, 2026b].

Para la detección de objetos en concreto, la red debe predecir no solo la clase del objeto presente en la imagen, sino también su posición y dimensiones (las *bounding boxes*). Existen dos grandes familias de arquitecturas de detección [Sharkyun, 2023]:

- **Detectores en dos etapas (two-stage)**, como Faster R-CNN. Una primera etapa propone regiones candidatas y una segunda etapa las clasifica. Son más precisos pero también más lentos.
- **Detectores en una etapa (one-stage)**, como YOLO o SSD. Predicen clases y *bounding boxes* en una sola pasada. Son menos precisos pero significativamente más rápidos, lo que

los hace adecuados para vídeo en tiempo real. Por ese motivo emplearemos un modelo *one-stage* para nuestro despliegue.

7.1.2. Métricas de evaluación

Para evaluar el rendimiento de un modelo de detección se utilizan varias métricas. Las fundamentales son [Durán Barroso y de Miguel, 2026a][Casaseca de la Higuera, 2026a]:

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

Figura 7.2: Matriz de confusión genérica.

Precisión (*precision*)

Fracción de detecciones positivas que son correctas: $P = TP / (TP + FP)$.

Sensibilidad (*recall*)

Fracción de objetos reales que el modelo detecta: $R = TP / (TP + FN)$.

F1-score Media armónica de precisión y sensibilidad: $F1 = 2PR / (P + R)$.

IoU (*Intersection over Union*)

Solapamiento entre la *bounding box* predicha y la real, expresado como el área de la intersección dividida entre el área de la unión. Un $IoU \geq 0,5$ se considera una detección correcta.

mAP (*mean Average Precision*)

Métrica global que promedia la precisión en distintos puntos de la curva precisión–sensibilidad para cada clase, y luego promedia entre clases. Se reporta habitualmente con un umbral de IoU del 50 % (mAP@50) o promediada sobre múltiples umbrales (mAP@50:95).

Para el modelo final del proyecto se ha establecido como objetivo una precisión global superior al 85 %, en línea con los requisitos del enunciado.

7.1.3. Objetivos de la parte de IA del proyecto

El objetivo principal es el de tener un modelo de IA que pueda reconocer y clasificar señales de tráfico para que el coche conectado sea capaz de recibir esa clasificación de señales y pueda reaccionar a ellas.

Los objetivos específicos de este bloque son:

1. Generar un *dataset* de imágenes de señales de tráfico relevantes para el escenario de demostración (entorno de laboratorio con señales de juguete).

2. Etiquetar y depurar el *dataset*, partiendo de tres corpus heredados de equipos anteriores.
3. Seleccionar un modelo de detección de objetos adecuado para ejecución en *hardware* embarcado y en tiempo real.
4. Entrenar el modelo iterativamente, ajustando *dataset* e *augmentation* hasta alcanzar precisiones superiores al 85 %.
5. Migrar la inferencia de la nube (Roboflow) a un dispositivo *edge* (Jetson Orin Nano) para minimizar la latencia y eliminar la dependencia de Internet.
6. Integrar la inferencia con el flujo de vídeo del DeepRacer a través de la red 4G del laboratorio.

7.2. Selección del modelo de detección

Se ha realizado un análisis comparativo entre los principales modelos de detección de objetos en imágenes (Tabla 7.2), con un enfoque DAFO seguido de una valoración cuantitativa, seleccionando finalmente **YOLOv11** de Ultralytics [Jocher y Qiu, 2024] por las siguientes razones:

- **Velocidad y precisión:** YOLOv11 procesa vídeo a más de 60fps en GPU dedicada y obtiene *mAP* elevadas con modelos de reducido tamaño (variantes **nano**, **small**, **medium**, **large** y **extra-large**).
- **Compatibilidad con la familia Jetson:** la documentación oficial de Ultralytics y NVIDIA garantiza el soporte sobre arquitectura ARM aarch64 y GPU Ampere mediante PyTorch optimizado [NVIDIA Corporation, 2025].
- **Integración con Roboflow:** el flujo etiquetado → entrenamiento → exportación de pesos **.pt** es directo, lo que reduce el tiempo de iteración entre versiones del modelo [Roboflow Inc., 2026].
- **Documentación y comunidad:** Ultralytics mantiene una documentación abundante y actualizada, lo que minimiza el coste de aprendizaje. Existen *notebooks* de ejemplo y guías para casi cualquier escenario.
- **Detección multi-clase simultánea:** a diferencia de algoritmos más simples, YOLOv11 detecta múltiples clases en la misma imagen, lo cual es indispensable para el escenario del proyecto (varios conos, varias señales, etc.).

7.2.1. DAFO de YOLOv11

Tabla 7.1: DAFO de YOLOv11 como modelo de detección.

Debilidades Menor documentación, ejemplos y recursos de aprendizaje que otras versiones más maduras como YOLOv5. Funcional solamente con imágenes (no series temporales). Menor corrección de <i>bugs</i> que versiones anteriores con más años de uso. Mayor riesgo de incompatibilidad con versiones nuevas de <i>software</i> y <i>hardware</i> .	Amenazas No disponer de la suficiente información para implementar configuraciones avanzadas. No escalable a otros ámbitos distintos del análisis de imágenes. Sensible a la resolución de las imágenes y a las condiciones ambientales (iluminación, lluvia, nieve). Requiere recursos computacionales considerables para entrenamiento.
Fortalezas Diseñado específicamente para analizar imágenes con detección y clasificación simultáneas. Más rápido, eficiente y preciso que versiones anteriores de YOLO en igualdad de hardware. Modelo compatible con ejecución en tiempo real (más de 60 fps). Detección de múltiples clases simultáneamente.	Oportunidades Implementar un modelo de manera más sencilla y rápida que con otros algoritmos. Diseño de un sistema basado en detección de objetos en imágenes. Posibilidad de ejecutarlo en entornos como las tarjetas de NVIDIA Jetson, gracias al soporte de PyTorch.

7.2.2. Comparativa cuantitativa

Tabla 7.2: Comparativa cuantitativa entre modelos candidatos. Puntuación: 0=no cumple, 1=cumple ligeramente, 2=cumple consistentemente, 3=excelente.

Característica	YOLOv11	YOLOv9	Faster R-CNN	SSD
Documentación (×3)	1	2	2	2
Actualizaciones (×1)	3	1	2	2
Velocidad/precisión (×3)	3	3	2	1
Compatibilidad Jetson (×3)	3	3	1	1
Capacidad multiclase (×2)	3	3	1	1
Facilidad implementación (×3)	3	2	1	1
TOTAL	39	37	22	19

Los modelos descartados se evaluaron por los siguientes motivos [More et al., 2024]:

- SSD (Single Shot MultiBox Detector)** Aunque permite detectar y clasificar en una sola pasada, es más lento que YOLO y obtiene precisiones inferiores en *datasets* pequeños como el del proyecto.
- Faster R-CNN** Es muy preciso, especialmente con objetos pequeños, pero su arquitectura en dos etapas lo hace demasiado lento para vídeo en tiempo real (< 10 fps incluso con GPU dedicada). No es viable en una Jetson Orin Nano.
- AlexNet y otros modelos clásicos** Fueron pioneros en clasificación de imágenes pero están en desuso para tareas de detección modernas.
- YOLOv7, YOLOv9** Versiones intermedias que ofrecen prestaciones inferiores a YOLOv11 pero con menor soporte de la comunidad en 2026.

Con **39 puntos sobre un máximo de 51**, YOLOv11 es la elección clara, por sus funcionalidades actualizadas, su posibilidad de detección y clasificación simultánea, su integración en Roboflow, su capacidad de procesamiento en tiempo real y su compatibilidad con la JETSON

de NVIDIA. Adicionalmente, su variante **nano** podría ofrecer un modelo de menos de 6 MiB compatible con dispositivos *edge* de recursos limitados (aunque no fue empleado).

7.3. Herramientas y entorno de trabajo

Para la parte de IA se ha utilizado el siguiente conjunto de herramientas:

Roboflow Plataforma *online* para etiquetar imágenes, dividir el *dataset* en *train/validation/-test*, aplicar preprocesado (redimensionado y ecualización) y *augmentation* (ajuste de brillo, desenfoque y *cutout*). Roboflow también ha permitido entrenar los modelos directamente en su nube con nuestro *dataset* y exportarlos los pesos `.pt` de PyTorch [Roboflow Inc., 2026].

PyCharm

IDE para Python utilizado como entorno principal de desarrollo del código de inferencia local y de los *scripts* auxiliares para la obtención de métricas.

Ultralytics

Librería oficial de YOLOv11. Proporciona tanto la API de Python (`from ultralytics import YOLO`) como herramientas de línea de comandos para validación y exportación [Jocher y Qiu, 2024].

Supervision

Librería de Roboflow para anotar imágenes con *bounding boxes* y etiquetas a partir de los resultados de inferencia. Sustituida en versiones posteriores por `results[0].plot()` de Ultralytics.

OpenCV Para captura de imágenes desde la cámara del DeepRacer y para el preprocesado en tiempo real.

NVIDIA Jetson Orin Nano

La tarjeta gráfica (Figura 5.13) que ha permitido ejecutar a la perfección el modelo de IA para la detección y clasificación de las señales en el borde de la red (*edge*) [NVIDIA Corporation, 2025].

7.4. Datasets y etiquetado

7.4.1. Punto de partida: tres corpus heredados

Como punto de partida se han combinado **tres *datasets*** cedidos por equipos de años anteriores del Taller de Proyectos II, a los cuales les damos las gracias. Juntando estos *datasets* contamos inicialmente con un total de **2 988 imágenes etiquetadas**:

- *Dataset* 1: 628 imágenes.
- *Dataset* 2: 1 373 imágenes.
- *Dataset* 3: 987 imágenes.

Estos corpus se habían generado en años anteriores con coches y entornos similares pero parcialmente diferentes, lo que introdujo heterogeneidad en las imágenes (resoluciones, condiciones de iluminación, ángulos de captura). Esta heterogeneidad fue positiva en términos de generalización del modelo, pero también introdujo ruido que requirió de depuración.

7.4.2. Limpieza y homogeneización

Tras combinar los tres corpus, se realizó una limpieza exhaustiva en varias fases:

1. **Recuento de etiquetas existentes:** se inventarió todas las clases presentes en los tres *datasets*.
2. **Renombrado y unificación:** se homogeneizaron los nombres de clase (por ejemplo, **stop**, **STOP** y **Stop** se unificaron en **STOP**) para evitar duplicidades.
3. **Revisión visual de las imágenes:** se revisaron todas las imágenes una a una, corrigiendo las *bounding boxes* mal posicionadas y añadiendo etiquetas faltantes.
4. **Eliminación de imágenes inutilizables:** se descartaron las imágenes excesivamente borrosas, sobreexpuestas o con contenido irrelevante.

El resultado fue la versión inicial del corpus combinado, denominada **v2 2026-02-24**, con 22 clases y aproximadamente 2 900 imágenes utilizables.

7.4.3. Catálogo inicial de 22 clases

El catálogo inicial de etiquetas combinado de los tres corpus heredados era el siguiente:

Tabla 7.3: Catálogo inicial de 22 clases con su volumen de etiquetas.

Clase	N. etiquetas	Clase	N. etiquetas
CONO	1 079	PELIGRO-ROTONDA	337
OBLIGATORIO-RECTO	557	PELIGRO-OPERARIO	254
OBLIGATORIO-GIRAR-DERECHA	512	PELIGRO-SEMAFORO	218
OBLIGATORIO-GIRAR-IZQUIERDA	406	PELIGRO-NINOS	212
LIMITE-90	244	PELIGRO-ESTRECHAMIENTO-DERECHA	151
LIMITE-30	367	PELIGRO-TREX	138
STOP	429	PELIGRO-PENDIENTE10	130
VALLA	497	PELIGRO-OPERARIO-OBRAS	91
PROHIBIDO-PASO	439	PELIGRO-AVION	50
PROHIBIDO-ADELANTAR	347	—	—
PROHIBIDO-CLAXON	297	—	—
PREFERENCIA-CONTRARIOS	300	—	—
PELIGRO-OTROS	461	—	—

7.4.4. Carencias identificadas

Tras analizar el volumen de cada clase, se identificaron **ocho clases con menos de 250 etiquetas**, lo que en la práctica impedirá al modelo aprender a clasificarlas con suficiente precisión:

- PELIGRO-AVION (50 etiquetas).
- PELIGRO-OPERARIO-OBRAS (91).
- PELIGRO-PENDIENTE10 (130).

- PELIGRO-TREX (138).
- PELIGRO-ESTRECHAMIENTO-DERECHA (151).
- PELIGRO-NINOS (212).
- PELIGRO-SEMAFORO (218).
- LIMITE-90 (244).

Adicionalmente, el análisis del mapa de calor de las etiquetas reveló que la mayoría de las anotaciones se concentraban en la zona central-superior de la imagen, mientras que la zona inferior-central (la más cercana al vehículo, donde aparecen señales próximas al campo de visión) estaba infrarrepresentada. Este sesgo espacial podría afectar negativamente al rendimiento del modelo en escenarios reales.

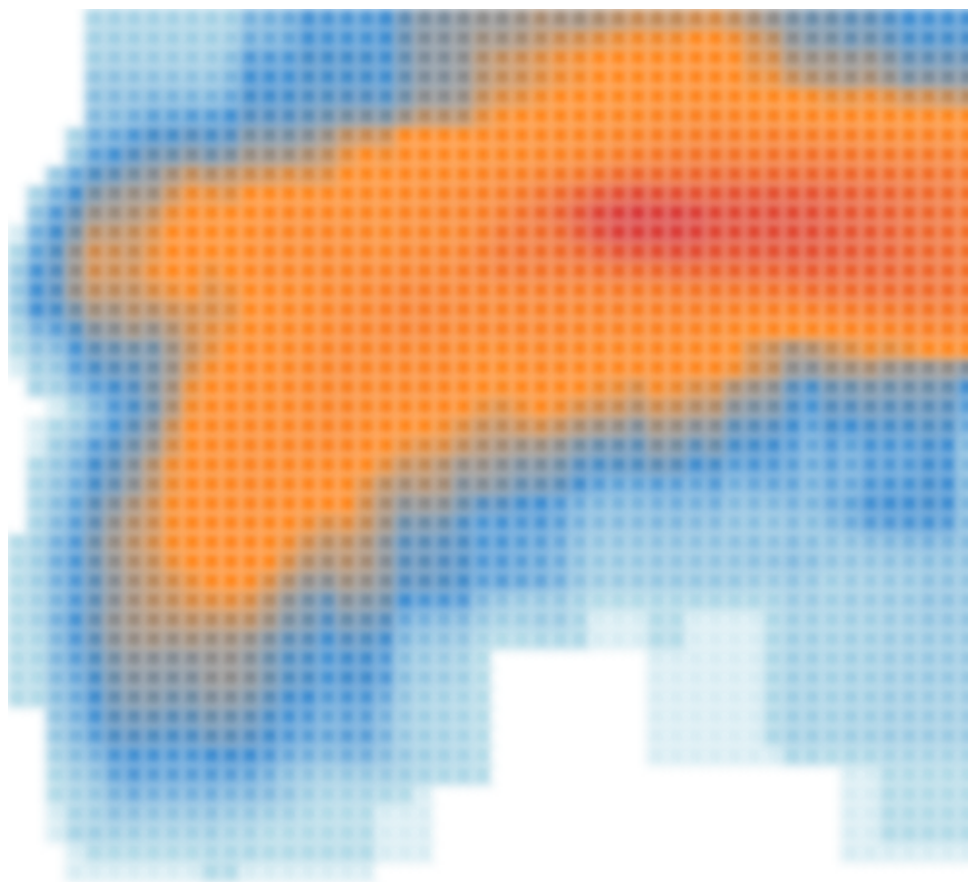


Figura 7.3: Mapa de calor de las etiquetas iniciales del *dataset*.

Finalmente, el análisis de número de etiquetas por imagen mostró que en **1 611 imágenes** había entre 2 y 3 etiquetas, pero apenas existían imágenes con 6 o 7 etiquetas (escenarios densos) ni imágenes vacías sin ninguna señal (*negative samples*), ambos casos importantes para una correcta generalización (Figura 5.8).

7.4.5. Reducción del *dataset*: 11 clases finales

Tras una consulta con el equipo docente y un análisis del escenario de demostración (un circuito en miniatura con señales de juguete específicas), se decidió eliminar las clases que no se

iban a emplear en la simulación y que podrían generar confusión al modelo. Las clases eliminadas fueron:

- PELIGRO-AVION (no tiene una reacción en el vehículo).
- PELIGRO-ESTRECHAMIENTO-DERECHA (no tiene una reacción en el vehículo).
- PELIGRO-OPERARIO (se mantiene la específica de obras).
- PELIGRO-OTROS (categoría comodín).
- PELIGRO-PENDIENTE10 (no tiene una reacción en el vehículo).
- PELIGRO-ROTONDA (no aplicable al escenario de la demostración).
- PELIGRO-SEMAFORO (no se usan semáforos en la demostración).
- PELIGRO-TREX (no tiene una reacción en el vehículo).
- PREFERENCIA-CONTRARIOS (no tiene una reacción en el vehículo).
- PROHIBIDO-ADELANTAR (no aplicable al escenario de la demostración).
- PROHIBIDO-CLAXON (no tiene una reacción en el vehículo).

Las clases **finales** del modelo quedaron reducidas a las recogidas en la Tabla 7.4.

Tabla 7.4: Clases finales del modelo YOLOv11 entrenado por el equipo G2TP2.

CONO	LIMITE-30	LIMITE-90
OBLIGATORIO-GIRAR-DERECHA	OBLIGATORIO-GIRAR-IZQUIERDA	OBLIGATORIO-RECTO
PELIGRO-NIÑOS	PELIGRO-OPERARIO-OBRAS	PROHIBIDO-PASO
STOP	VALLA	—

7.4.6. Distribución del nuevo dataset

Tras la reducción del *dataset*, la incorporación de fotografías nuevas tomadas con la cámara del DeepRacer (de aquellas clases que tenían carencias) y un nuevo etiquetado completo, la distribución de etiquetas del corpus depurado quedó como se muestra en la Tabla 7.5.

Tabla 7.5: Distribución de etiquetas del corpus depurado.

Clase	N. etiquetas
CONO	1 142
OBLIGATORIO-RECTO	589
OBLIGATORIO-GIRAR-DERECHA	542
VALLA	509
PROHIBIDO-PASO	470
STOP	458
OBLIGATORIO-GIRAR-IZQUIERDA	430
LIMITE-30	383
LIMITE-90	268
PELIGRO-NINOS	237
PELIGRO-OPERARIO-OBRAS	114
TOTAL	5 142

Puede observarse que persistió cierta descompensación: **PELIGRO-OPERARIO-OBRA**s que sigue por debajo de las 250 muestras y **PELIGRO-NINOS** apenas alcanza esa cifra. Estas dos clases serían las que peor rendimiento mostraron en las métricas finales de este modelo.

7.4.7. Captura y etiquetado de imágenes propias con el DeepRacer

Para complementar los corpus heredados, el equipo realizó varias sesiones de captura de imágenes propias con la cámara del AWS DeepRacer. Para ello se utilizó el comando **cheese** en el ordenador del DeepRacer.

En las primeras capturas aun no se disponía de la infraestructura 4G para obtener dichas imágenes en el core. De manera que se hizo una conexión local, conectando la salida HDMI, y los puertos USB directamente al deepRacer.

Estas sesiones de capturas de imágenes fueron haciéndose según las necesidades que iban surgiendo de los modelos que se entrenaban, principalmente debido a la confusión de las clases **LIMITE-30**, **LIMITE-90**, **OBLIGATORIO-GIRAR-IZQUIERDA** y **OBLIGATORIO-GIRAR-DERECHA**. Un ejemplo de estas capturas se encuentra en la Figura 7.6.

- Captura de imágenes básicas de las señales clave (**STOP**, **LIMITE-30**, giros en ambos sentidos, **PROHIBIDO-PASO** y **CONO**).
- Captura masiva de imágenes con el coche en distintas posiciones y orientaciones, simulando el movimiento real del vehículo en el circuito.
- Captura específica de las clases problemáticas (**LIMITE-30/LIMITE-90**, **OBLIGATORIO-GIRAR-DERECHA/IZQUIERDA**) tras detectarse confusiones recurrentes en el modelo V3.

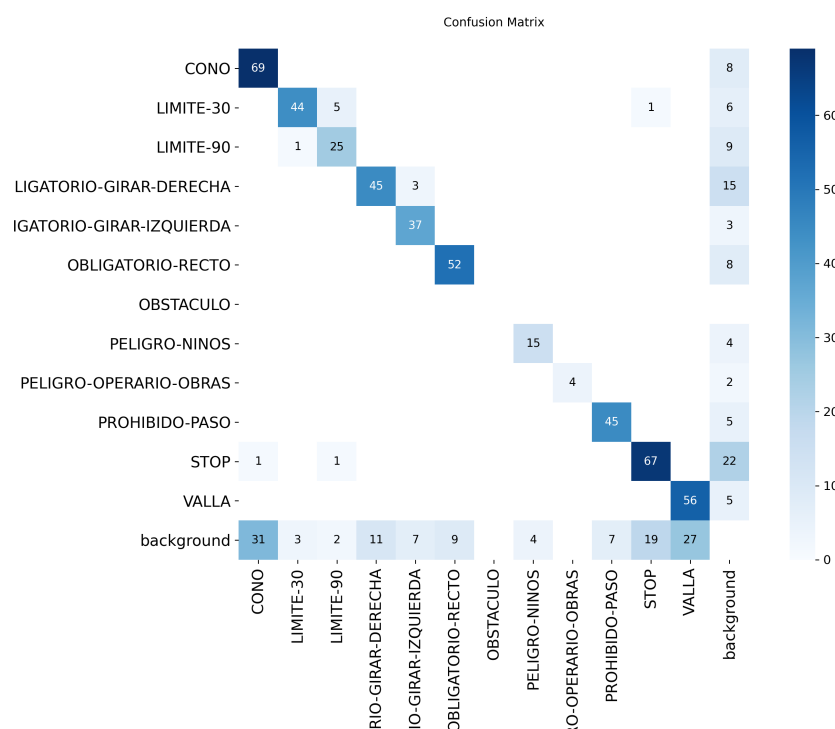


Figura 7.4: Matriz de confusión del modelo V3 con confusiones de etiquetas.

7.5. Iteraciones del modelo

A lo largo del proyecto se han entrenado **siete versiones del modelo**, cada una resolviendo problemas detectados en la iteración anterior. La Tabla 7.6 resume la evolución; los apartados

siguientes describen cada versión con detalle.

Tabla 7.6: Iteraciones del modelo YOLOv11 entrenadas durante el proyecto.

Versión	Cambios principales	Resultado
V1	Primer entrenamiento sobre los 3 <i>datasets</i> unificados (22 clases).	Buenas precisiones globales pero clases minoritarias por debajo de 0,7.
V2	Reetiquetado, eliminación de 11 clases problemáticas.	Mejora en clases mayoritarias pero confusión derecha/izquierda.
V3	Incorporación de fotografías del DeepRacer.	Aún confusión LIMITE-30/90 y giros.
V4	<i>Augmentation</i> intensiva (recortes, ruido).	La <i>augmentation</i> introduce confusión adicional; resultado peor que V3.
V5	Mismo <i>dataset</i> que V4 sin <i>augmentation</i> .	Menor capacidad de generalización; pierde detecciones que V3 sí hacía.
V6	<i>Augmentation</i> selectiva (sin rotaciones que afecten a flechas).	Resultados aceptables, pero VALLA y CONO por debajo de 0,7 con IoU=0,5.
V7	Preprocesado: división vertical de imagen para distinguir carriles, redimensionado a 640×640, ajuste de contraste con múltiples exposiciones.	mAP@50 = 0,967; Precisión = 0,940; Recall = 0,935; F1 = 0,937.

7.5.1. Modelo V1 — primer entrenamiento

El primer modelo se entrenó directamente sobre los tres *datasets* unificados con las 22 clases originales y la versión v3 2026-02-24 del corpus (redimensionada a 640×640 por recomendación de Ultralytics). El entrenamiento se realizó en la nube de Roboflow con la variante *yolov11n* (*nano*) durante 100 *epochs*.

Los resultados sobre el conjunto de test mostraron precisiones por encima del 90 % en la mayoría de clases, pero con cuatro clases especialmente problemáticas: PELIGRO-OPERARIO-OBRA, PELIGRO-ESTRECHAMIENTO-DERECHA, PELIGRO-SEMAFORO y PROHIBIDO-CLAXON. Las tres primeras coinciden con clases minoritarias en el análisis de carencias.

Las clases con más *falsos positivos* fueron CONO, OBLIGATORIO-GIRAR-DERECHA, OBLIGATORIO-RECTO, VALLA y PROHIBIDO-PASO. Las recomendaciones automáticas de Roboflow indicaban añadir más imágenes de las clases problemáticas y considerar técnicas de *tiling* para detección de objetos pequeños.

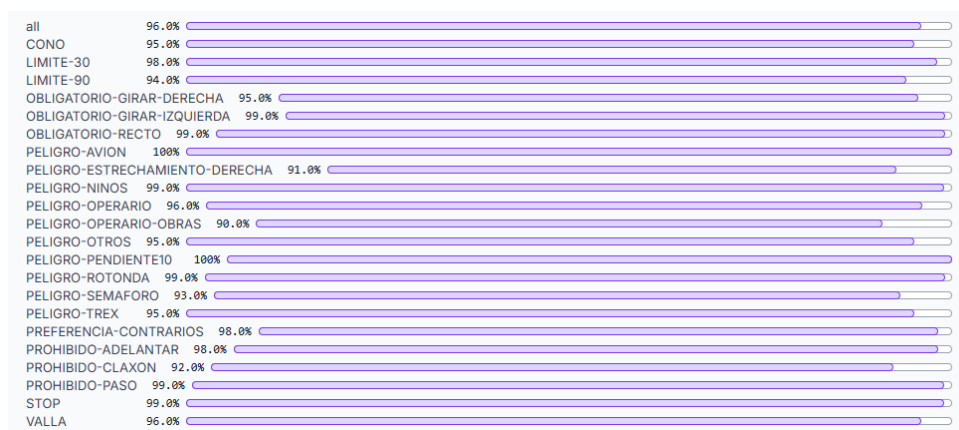


Figura 7.5: Precisiones del primer modelo entrenado de YoloV11 por clases.

7.5.2. Modelo V2 — reetiquetado y reducción de clases

A partir de las recomendaciones de la V1 y de la consulta con el equipo docente, se eliminaron 11 clases del catálogo (las no aplicables al escenario de la demo) y se reetiquetó el corpus completo. El modelo V2 entrenado con este corpus reducido mostró mejoras notables en las clases mayoritarias, pero seguía confundiendo:

- LIMITE-30 con LIMITE-90.
- OBLIGATORIO-GIRAR-DERECHA con OBLIGATORIO-GIRAR-IZQUIERDA.

Estas confusiones tienen una causa estructural: las señales son visualmente muy similares (mismo formato, mismos colores, solo cambia un dígito o la orientación de la flecha). En *data-sets* pequeños como el del proyecto, el modelo no recibe suficientes ejemplos para discriminar correctamente.



Figura 7.6: Imagen tomada durante el proceso de capturas para su etiquetado.

7.5.3. Modelo V3 — incorporación de fotografías propias

Para reducir las confusiones se realizó la primera sesión de captura de imágenes, incorporando varias docenas de fotografías nuevas de LIMITE-30, LIMITE-90 y de las dos versiones del giro obligatorio. El modelo V3, entrenado con el corpus ampliado, ofreció prestaciones mejores que V2 pero todavía con confusiones residuales.

V3 fue además el primer modelo probado en local, empleando el script en python que puede encontrarse en la sección D.3. En este *script* se realizó la inferencia de imágenes en estático, la prioridad de señales por tamaño y confianza. Esto permitió probar la inferencia local con llamadas a la API de Roboflow.

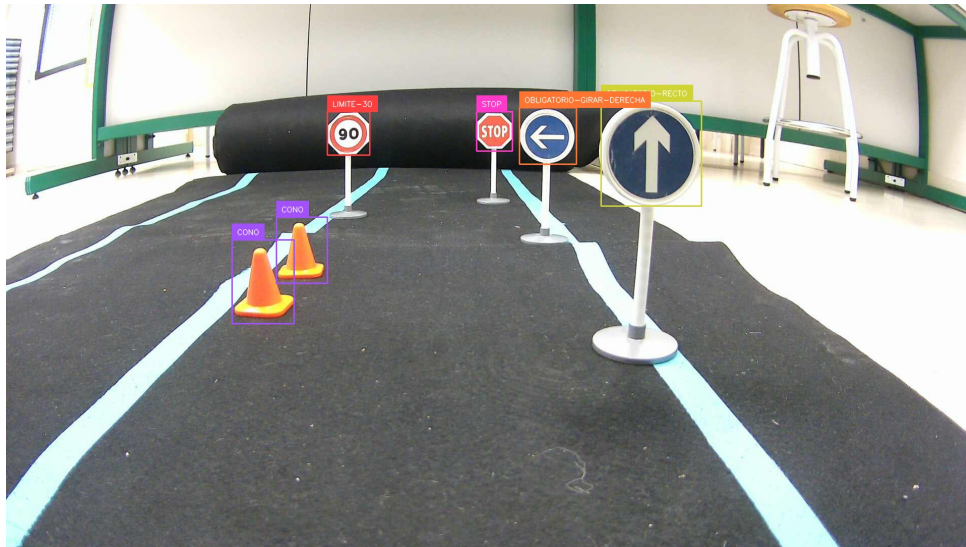


Figura 7.7: Inferencia realizada con el modelo V3.

7.5.4. Modelo V4 — augmentation intensiva

Buscando mejorar la generalización, V4 se entrenó con el mismo *dataset* que V3 pero con *data augmentation* intensiva: rotaciones de hasta $\pm 15^\circ$, recortes aleatorios, ruido gaussiano, ajuste de brillo y volteos horizontales. El resultado fue contraintuitivo: las prestaciones empeoraron respecto a V3.

El análisis identificó la causa: las rotaciones y los volteos horizontales introducen confusión adicional en clases como OBLIGATORIO-GIRAR-DERECHA, ya que un volteo horizontal convierte una flecha derecha en una flecha izquierda, generando etiquetas incorrectas. Lección aprendida: la *augmentation* debe seleccionarse con conocimiento del dominio, no aplicarse indiscriminadamente.

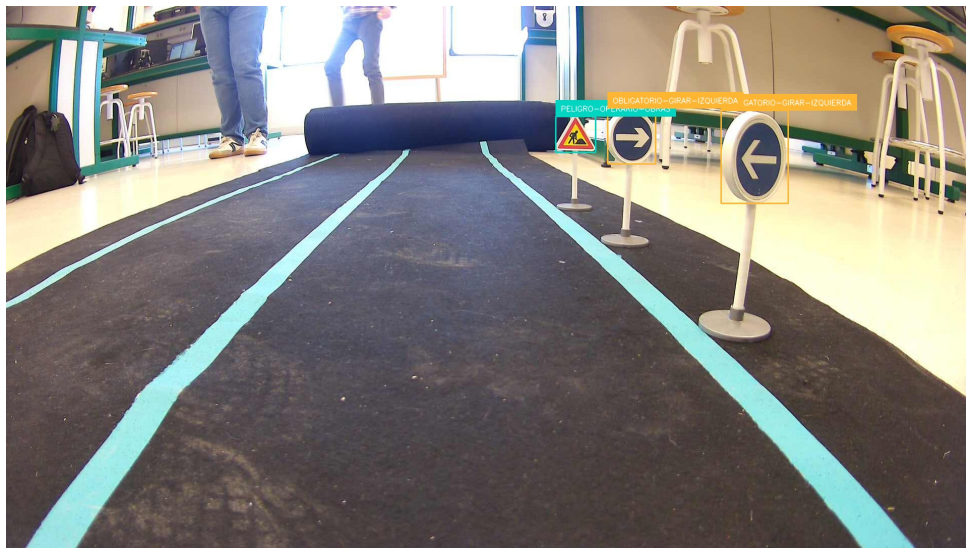


Figura 7.8: Inferencia realizada donde se produce error en las señales de giros.

7.5.5. Modelo V5 — sin augmentation

V5 es el opuesto de V4: mismo *dataset*, sin *augmentation* alguna. El número efectivo de imágenes de entrenamiento se reduce drásticamente (no hay multiplicación), lo que se traduce en

una menor capacidad de generalización. V5 perdía detecciones que V3 sí realizaba correctamente. Confirma que algo de *augmentation* es necesario, pero debe seleccionarse cuidadosamente.

7.5.6. Modelo V6 — *augmentation* selectiva

V6 incorpora *augmentation* pero **deseleccionando** las transformaciones que afectan a la semántica de la señal: se eliminan las rotaciones y los volteos horizontales que confundían LIMITE-30 con LIMITE-90 o las flechas. Se mantienen los ajustes de brillo, contraste y un nivel mínimo de ruido.

V6 da resultados aceptables y supera ligeramente a V3, pero las clases VALLA y CONO se quedan por debajo de una precisión del 0,7 cuando se evalúan con $\text{IoU} \geq 0,5$. Aún hay margen de mejora.

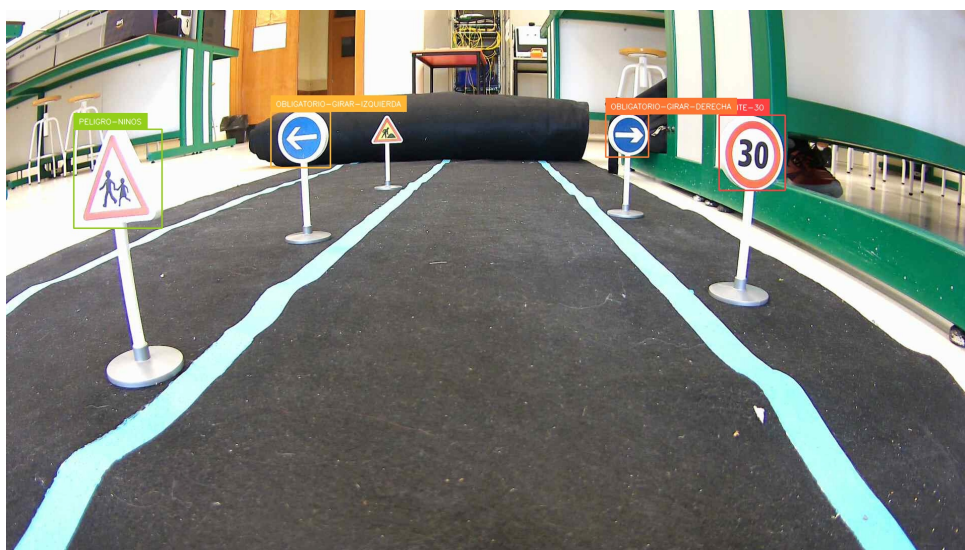


Figura 7.9: Inferencia realizada con el modelo V6.

7.5.7. Modelo V7 — versión final

V7 es la versión final del modelo. Se entrena sobre el mismo corpus que V6 pero con un preprocesado mucho más sofisticado:

- **División vertical de la imagen en dos mitades** para que el modelo aprenda a distinguir entre carriles (la mitad izquierda y la mitad derecha de la calzada) y de las señales de giro.
- **Redimensionado a 640×640** , valor recomendado oficialmente por YOLOv11 para equilibrio entre detalle y velocidad.
- **Ajuste de contraste con múltiples exposiciones** para que el modelo sea robusto a condiciones de iluminación variadas: contraluz, exposición baja, exposición alta.

Por otro lado se incluyó la siguiente *augmentation*:

- **Multiplicación máxima del *dataset*: 5.**
- **Exposición:** entre -4 % y +4 %.
- **Ruido máximo** del 0.5 % de los píxeles.
- **Efecto movimiento:** 10 px, ángulo de 0° , Frames: 1.

Las métricas obtenidas sobre el conjunto de test independiente fueron las siguientes:

- **mAP@0,5:** 0,935.
- **Precisión global:** 0,940.
- **Recall global:** 0,935.
- **F1-score:** 0,937.

Lo que hace que cumplamos con la cláusula segunda del pliego del proyecto.

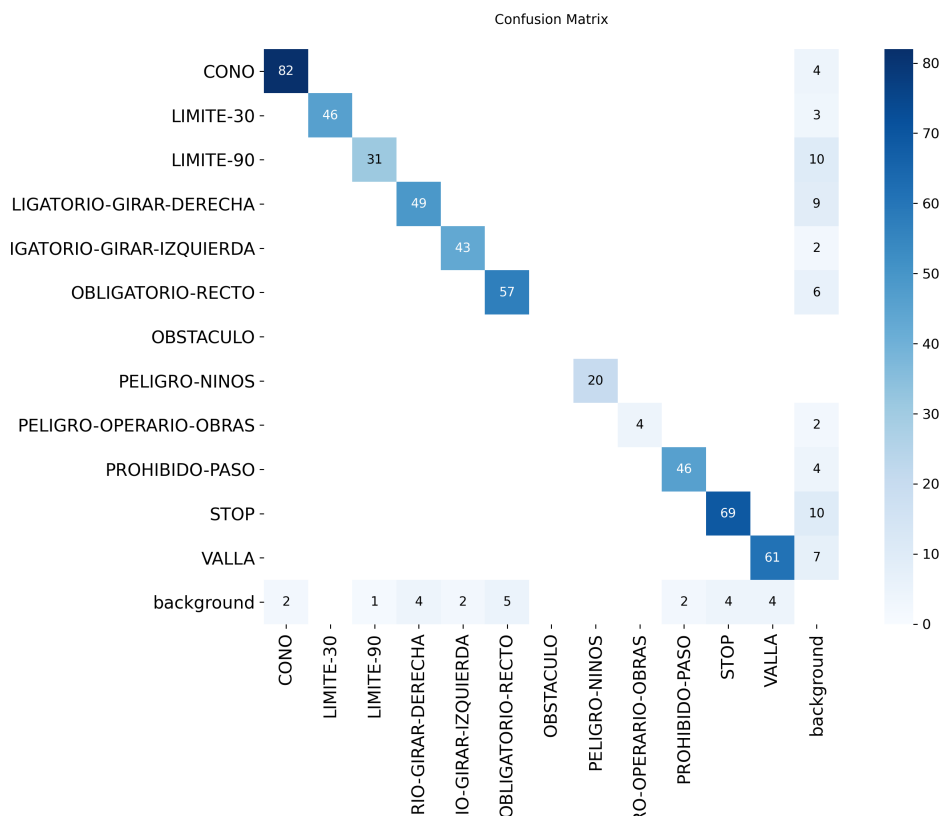


Figura 7.10: Matriz de confusión del modelo en su versión definitiva.

La matriz de confusión ¹ muestra que las pocas confusiones residuales se producen contra el fondo (**background**), no entre clases de señal, lo cual es el comportamiento deseado. En comparativa con la matriz de la Figura 7.4 se ha producido cierta mejora en la confusión de las clases **LIMITE-30 y 90** y las clases **OBLIGATORIO-GIRAR-DERECHA e IZQUIERDA**. Las clases con peor rendimiento son **PELIGRO-OPERARIO-OBRA** y **PELIGRO-NINOS**, en línea con el escaso volumen de muestras disponibles para esas clases.

¹Elaborada con el script disponible en la sección D.5, ya que para obtener las métricas disponibles en la plataforma Roboflow era necesario tener una cuenta premium.

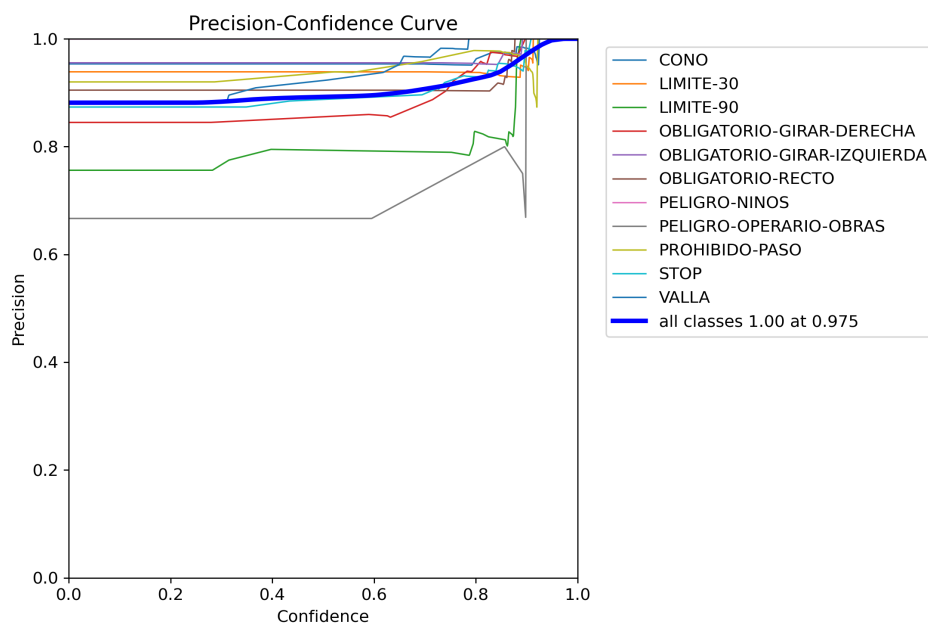


Figura 7.11: Curva Confianza-Precisión del modelo v7.

En la Figura 7.11 puede observarse que para confianzas por encima del 80 % se obtienen precisiones por encima del 80 % a excepción de las clases peligro-operario obras y limite 90. Para obtener una precisión del 100 % deberíamos poner una confianza del 0.975, pero perderíamos algunas detecciones. Para tener un equilibrio detecciones-confianza deberíamos tener un umbral de confianza entre 0.6 a 0.8, pero introduce más falsos positivos.

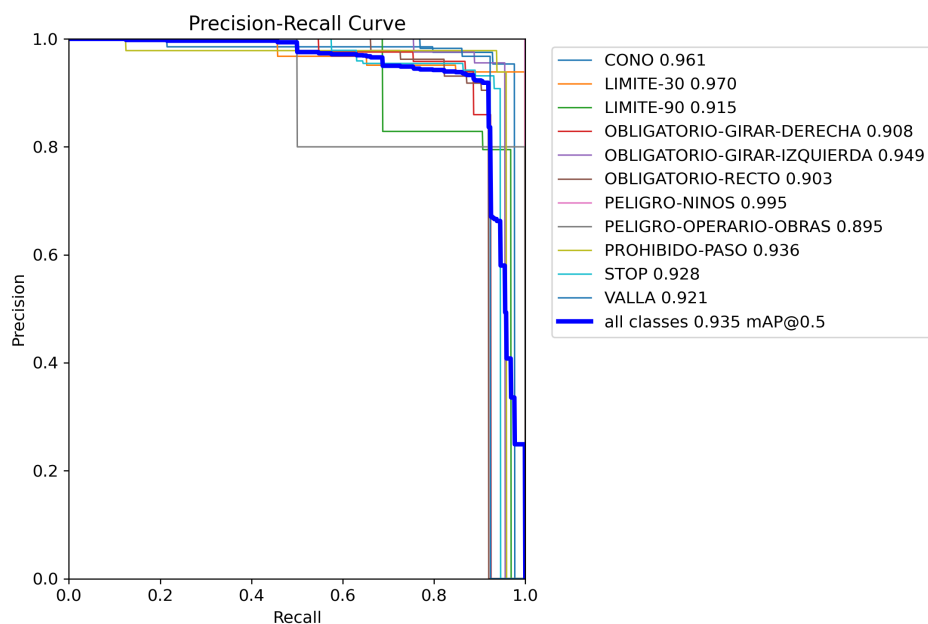


Figura 7.12: Curva Precisión-Recall del modelo v7.

La gráfica precisión-recall indica que el modelo es capaz de mantener una precisión altísima (0,935) incluso cuando intenta encontrar casi todas las señales (*recall* alto). La línea de PELIGRO-OPERARIO-OBRAS (0,895) cae antes que las demás, confirmando que esa clase necesita más muestras para mejorar. Para evitar posibles fallos en la demostración, se decide no emplear esta señal.

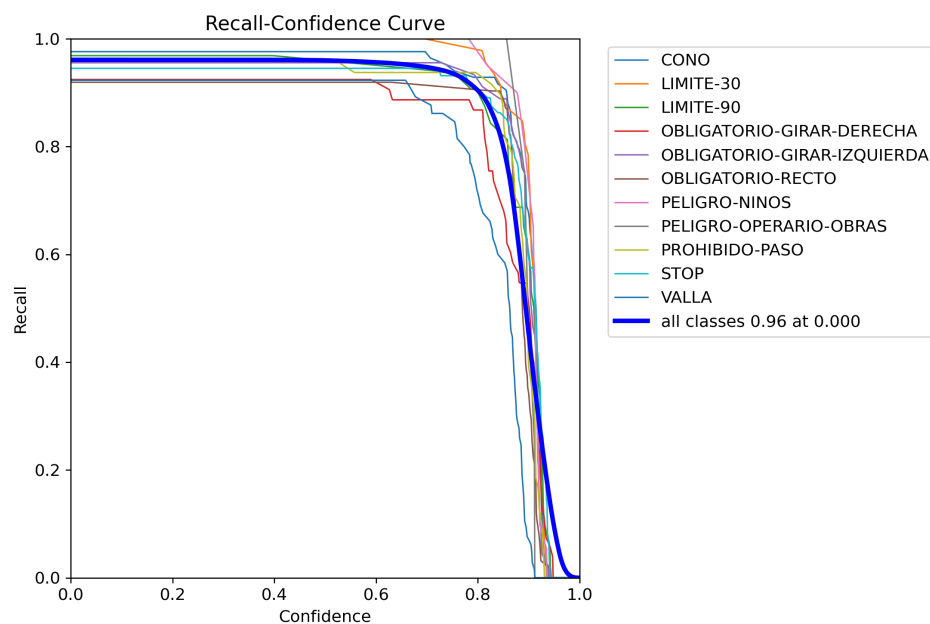


Figura 7.13: Curva Confianza-Recall del modelo v7.

La gráfica confianza-recall, finalmente, indica que con un umbral de confianza del 0 % (todas las detecciones admitidas) el modelo encuentra el 96 % de los objetos presentes en la imagen. Las líneas se mantienen muy planas y altas hasta llegar aproximadamente al 0,85 de confianza, momento en que comienzan a caer. Esto significa que la gran mayoría de las señales reales son detectadas con una confianza superior al 85 %, justificando la elección del umbral 0,85 para la operación final y cumpliendo con el objetivo de alcanzar dicho umbral de precisión para la detección de señales.

El umbral de confianza, por tanto, tras analizar todas las curvas es del **85 %**: por encima de este valor, la mayoría de las detecciones son fiables y se mantiene un *recall* cercano al 0,93.

CLASE	IMÁGENES	INSTANCIAS	PRECISIÓN (P)	EXHAUSTIVIDAD (R)	mAP50	mAP50-95	ESTADO
all	630	532	0.907	0.947	0.935	0.738	4340 ITEMS
CONO	58	84	0.953	0.966	0.961	0.782	VALIDACIÓN COMPLETADA 100%
LIMITE-30	46	45	0.939	0.998	0.977	0.812	
LIMITE-90	32	32	0.79	0.942	0.915	0.707	
OBLIGATORIO-GIRAR-DERECHA	52	53	0.883	0.887	0.908	0.677	
OBLIGATORIO-GIRAR-IZQUIERDA	45	45	0.956	0.956	0.943	0.788	
OBLIGATORIO-RECTO	62	62	0.904	0.913	0.903	0.734	
PELIGRO-NINOS	20	20	1	1	0.995	0.857	
PELIGRO-OPERARIO-OBRAS	4	4	0.723	1	0.895	0.745	
PROHIBIDO-PASO	48	48	0.96	0.938	0.936	0.671	
STOP	73	73	0.901	0.945	0.928	0.787	
VALLA	65	65	0.966	0.875	0.921	0.586	

VELOCIDAD POR IMAGEN: Preprocesamiento: 3.2ms | Inferencia: 445.6ms | Pérdida: 0.0ms | Postprocesamiento: 2.1ms
TIEMPO TOTAL: 450.9ms/img

Figura 7.14: Tabla resumen de las prestaciones del modelo final para cada clase.

El modelo entrenado, es un modelo robusto. Cumple con los objetivos establecidos y es válido para el reconocimiento y clasificación de señales en tiempo real. Ahora podremos realizar la inferencia de señales en el *edge*.

7.5.8. Conclusiones de la iteración del modelo

El proceso de siete iteraciones muestra varios aprendizajes importantes:

1. **Calidad del *dataset* > arquitectura del modelo.** Cambiar parámetros del modelo apenas aporta mejora si el *dataset* no está bien depurado. La V2 (re-etiquetado) supuso la mejora más sustancial.

2. **La *augmentation* debe ser específica del dominio.** V4 demuestra que aplicar *augmentation* agresiva sin pensar en las clases puede empeorar el modelo.
3. **El preprocesado.** V7 demuestra que el preprocesado correcto (división vertical, redimensionado adecuado, ajuste de contraste) puede ser la diferencia entre un modelo aceptable y uno excelente.
4. **Las clases minoritarias siempre van a costar más.** PELIGRO-OPERARIO-OBRA y LIMITE-90 siguen siendo las peores incluso en V7, simplemente porque tienen menos datos. Por tanto, siempre se debe tener la misma cantidad de fotografías para todas las clases, de manera que no se queden clases escuetas.

7.6. Inferencia local en la NVIDIA Jetson Orin Nano

7.6.1. Motivación del paso de Roboflow a inferencia local

El *script* de partida, que puede encontrarse en la sección D.3 realizaba inferencia contra la API de Roboflow (`InferenceHTTPClient`). Este enfoque impone tres limitaciones para nuestro sistema de vehículo conectado:

1. **Dependencia de red externa y API del proveedor:** cada inferencia requiere de buena conexión a Internet, lo que en un entorno con conectividad fluctuante (como la autovía o el laboratorio con interferencias entre grupos o red WIFI inestable) es un punto de fallo crítico, ya que no podrá realizar inferencias de las imágenes.
2. **Latencia variable:** el tiempo de ida y vuelta hasta los servidores de Roboflow añade entre 50 ms a 100 ms al tiempo de inferencia, comprometiendo el tiempo de reacción del coche. El retardo total que obtuvo [González, 2022] con inferencia en servidor fue de 90,4 ms. Este retardo permitía al vehículo ser guiado de forma estable siempre que no se supere la velocidad de $1\text{m/s} \approx 3,6\text{ km/h}$. Por ello el objetivo era el de reducir este tiempo de retardo.
3. **Coste de cómputo:** Roboflow realiza la facturación basada en peticiones. A pesar de que nosotros hemos empleado una versión gratuita, esta cuenta estaba limitada y no sería posible escalarla a un despliegue real con miles de vehículos.

Por estos motivos se decidió migrar a inferencia local sobre la **NVIDIA Jetson Orin Nano**, aprovechando los 1024 núcleos CUDA y 32 *Tensor cores* de la GPU Ampere integrada.

7.6.2. Despliegue del entorno: 9 fases

El despliegue del entorno de inferencia local en la Jetson siguió un proceso sistemático en nueve fases, documentado en la página JETSON de la wiki interna del proyecto.

Fase 1 — Evaluación de compatibilidad YOLOv11/Jetson. Antes de cualquier instalación, se realizó una investigación sobre la compatibilidad entre YOLOv11 de Ultralytics y la plataforma Jetson Orin Nano. La conclusión fue positiva: la arquitectura GPU Ampere del dispositivo, con 1024 núcleos CUDA y 32 *Tensor cores*, junto con el SDK JetPack de NVIDIA, proporciona soporte completo para YOLOv11.

Se identificaron de antemano los principales riesgos: incompatibilidades entre versiones de PyTorch y CUDA para la arquitectura ARM aarch64, problemas con librerías como `libcusparselt`, y la necesidad de compilar `torchvision` desde código fuente al no existir paquetes binarios compatibles con el `torch` personalizado de NVIDIA para Jetson.

Esta fase de análisis previo permitió anticipar los obstáculos y condicionó la estrategia de instalación, optando desde el principio por una instalación directa en lugar de contenedores Docker.

Fase 2 — Preparación del PC *host* con Ubuntu 22.04. El dispositivo Jetson Orin Nano se encontraba sin sistema operativo. Para flashearlos se requiere un PC *host* con Ubuntu. Se dispuso de un equipo con Ubuntu 22.04 LTS, compatible con las herramientas de NVIDIA.

Se instaló NVIDIA SDK Manager descargando el paquete `.deb` desde el portal oficial de desarrolladores de NVIDIA. Este proceso requirió crear una cuenta gratuita en <http://developer.nvidia.com>. SDK Manager es la herramienta oficial de NVIDIA para flashear JetPack en dispositivos Jetson y automatiza la instalación de CUDA, cuDNN y TensorRT.

Se optó inicialmente por esta herramienta frente a la instalación manual mediante tarjeta microSD, aunque finalmente se utilizó la segunda vía por no disponer del cable USB-C necesario para el modo Recovery.

Fase 3 — Flasheo de JetPack 6.2 mediante tarjeta microSD. Al no disponer de cable USB-C para conectar la Jetson al PC en modo Recovery, se optó por flashear JetPack 6.2 directamente sobre una tarjeta microSD de 128 GB mediante la imagen oficial de NVIDIA. Esta imagen se descargó desde <http://developer.nvidia.com> y se verificó su integridad antes de escribirla.

La herramienta Balena Etcher se instaló para realizar el flasheo de forma gráfica, pero presentó un error al intentar abrir el archivo `.zip` directamente. Se resolvió mediante el comando `dd` de Linux, que descomprime y escribe la imagen en un solo paso, demostrando mayor fiabilidad para archivos de gran tamaño.

La tarjeta era de 128 GB, lo que proporcionó espacio suficiente para el sistema operativo, CUDA, las librerías de IA y el modelo. Durante el primer arranque, el asistente de JetPack preguntó por el tamaño de partición, optándose por expandirla al máximo disponible para aprovechar toda la capacidad de almacenamiento.

Fase 4 — Configuración inicial de la Jetson y resolución del navegador. Tras el primer arranque con JetPack 6.2 (R36.4.3) sobre Ubuntu 22.04, se detectó que los navegadores Firefox y Chromium, instalados como paquetes `snap`, no funcionaban correctamente. El sistema `snap` presentaba problemas de permisos con `cap_dac_override` en el entorno Jetson.

La solución consistió en eliminar las versiones `snap` y reinstalar Firefox como paquete `.deb` nativo desde el repositorio PPA de Mozilla, priorizando este repositorio sobre el `snap`. Esto permitió disponer de un navegador funcional para acceder a documentación y recursos durante el resto del proceso.

Fase 5 — Verificación del entorno CUDA. Se verificó que JetPack había instalado correctamente todos los componentes necesarios. El comando `nvidia-smi` confirmó la presencia de la GPU Orin con CUDA 12.6. Sin embargo, el compilador `nvcc` no era accesible desde el `PATH`, lo que se resolvió añadiendo `/usr/local/cuda/bin` al archivo `.bashrc`.

Este paso fue fundamental para confirmar que la base de *hardware* y *software* estaba correctamente configurada antes de proceder con la instalación de PyTorch, evitando instalar dependencias sobre una base defectuosa.

Fase 6 — Instalación de PyTorch optimizado para Jetson. La instalación de PyTorch en la Jetson no puede realizarse mediante `pip` estándar porque los paquetes disponibles en PyPI están compilados para arquitectura `x86_64`, incompatible con la arquitectura ARM `aarch64` de la Jetson. NVIDIA distribuye *builds* específicas para cada versión de JetPack.

Se instaló la versión `torch-2.5.0a0+872d972e41.nv24.08` directamente desde el repositorio de NVIDIA (developer.download.nvidia.com). Durante el proceso surgió un error crítico: `libcusparselt.so.0 not found`. Esta librería no estaba incluida en JetPack 6.2, por lo que se descargó e instaló manualmente desde el repositorio de CUDA de NVIDIA.

Un problema recurrente fue que la instalación de otras librerías (`ultralytics`, `torchvision` genérico) sobrescribía el `torch` de Jetson por la versión estándar de PyPI, que no tiene soporte CUDA en ARM. Fue necesario reinstalar el `torch` de NVIDIA en varias ocasiones y controlar las versiones de `numpy`, que también generaba conflictos al actualizarse automáticamente a la versión 2.x.

La solución definitiva fue usar el `flag --no-deps` en todas las instalaciones posteriores a `torch`, evitando que `pip` resolviera dependencias automáticamente y sobrescribiera paquetes ya configurados correctamente.

Fase 7 — Compilación de torchvision desde código fuente. Ninguna versión binaria de `torchvision` disponible en PyPI era compatible con el `torch` personalizado de NVIDIA para Jetson. El operador `torchvision::nms`, esencial para la detección de objetos, producía un `RuntimeError` al intentar cargarse con versiones precompiladas genéricas.

La única solución viable fue clonar el repositorio oficial de `torchvision` en la versión `v0.20.0` y compilarlo desde código fuente directamente en la Jetson. El proceso de compilación, que utilizó `nvcc` (compilador CUDA) y generó `kernels` específicos para la arquitectura `compute_87` del Orin, tardó aproximadamente 25 min.

Este paso fue el más laborioso del proceso, pero fue necesario para garantizar la compatibilidad total entre `torch`, `torchvision` y los operadores CUDA de la GPU Orin. El resultado fue una versión `torchvision-0.20.0a0+afc54f7` específicamente compilada para el hardware del Orin Nano.

Fase 8 — Instalación de Ultralytics y primera inferencia. Con PyTorch y `torchvision` correctamente configurados, se instaló el paquete `ultralytics` usando el `flag --no-deps` para evitar que sobrescribiera las dependencias ya configuradas. Se instalaron manualmente las dependencias necesarias (`opencv-python`, `tqdm`, `psutil`, `ultralytics-thop`) de la misma forma.

La primera prueba de inferencia utilizó el modelo `yolo11n.pt` (el más ligero de la familia YOLOv11) sobre una imagen de prueba de Ultralytics. Los resultados confirmaron que la inferencia se ejecutaba correctamente en GPU (`device='cuda:0'`), detectando 4 personas y 1 autobús con alta confianza en aproximadamente 320 ms.

Este resultado validó todo el proceso de instalación y demostró que el entorno estaba correctamente configurado para ejecutar modelos YOLOv11 con aceleración GPU en la Jetson Orin Nano.

Fase 9 — Adaptación del *script* de inferencia al modelo propio. Se disponía de un *script* original que utilizaba la API de Roboflow (`inference_sdk`) para realizar inferencia en la nube con su modelo entrenado. Se decidió migrar completamente a inferencia local para eliminar la dependencia de red, reducir la latencia y aprovechar la GPU de la Jetson.

El modelo entrenado se descargó desde Roboflow en formato PyTorch (`.pt`) y se transfirió a la Jetson mediante `scp`. El *script* fue reescrito sustituyendo:

- `InferenceHTTPClient` de `inference_sdk` por YOLO de `ultralytics`.
- La anotación de `supervision` (`BoxAnnotator`, `LabelAnnotator`) por `results[0].plot()` de `Ultralytics`.

La primera ejecución con el modelo propio (`weights.pt`) sobre imágenes reales de señales de tráfico fue exitosa, detectando correctamente LIMITE-30 y OBLIGATORIO-GIRAR-IZQUIERDA con tiempos de inferencia de aproximadamente 212 ms en GPU.

7.6.3. Resumen del entorno final en la Jetson

Tabla 7.7: Configuración final del entorno de inferencia en la Jetson Orin Nano.

Componente	Versión
Sistema operativo	Ubuntu 22.04 LTS (JetPack R36.4.3)
CUDA	12.6
PyTorch	2.5.0a0+872d972e41.nv24.08
torchvision	0.20.0a0+afc54f7 (compilado desde fuente)
Ultralytics	8.4.37
Modelo	weights.pt (YOLOv11 V7, mAP@50=0,967)
Tiempo de inferencia	~212 ms (GPU, cuda:0)

7.6.4. Pipeline de inferencia

El *script* final de inferencia adapta la lógica original de llamada a la API de Roboflow a la inferencia local de Ultralytics, manteniendo la selección de la detección más cercana al vehículo (mayor área del *bounding box*) por encima de un umbral de confianza configurable.

Listing 7.1: Fragmento del script de inferencia local en la Jetson.

```

1 while True:
2     data, addr = recv_sock.recvfrom(99999)
3
4     # Verificar que es un frame de imagen (tipo 'I')
5     data_type = struct.unpack('c', bytes([data[0]]))[0]
6     if data_type != b'I':
7         continue
8
9     # Decodificar imagen (mismo formato que el Core)
10    payload = pickle.loads(bytes(data[1:]), encoding='latin1')
11    buf = np.frombuffer(payload, dtype=np.uint8) if isinstance(payload, bytes)
12    else payload
13    frame = cv2.imdecode(buf, cv2.IMREAD_COLOR)
14
15    if frame is None:
16        continue
17
18    # -- INFERENCIA YOLOV11 EN GPU -----
19    results = model(frame, device=0, conf=0.85, verbose=False)
20
21    # -- EXTRAER DETECCIONES -----
22    detecciones = []
23    names = results[0].names
24    for box in results[0].boxes:
25        x1, y1, x2, y2 = box.xyxy[0].tolist()
26        detecciones.append({
27            'class': names[int(box.cls)],
28            'confidence': float(box.conf),
29            'x': (x1 + x2) / 2,
30            'y': (y1 + y2) / 2,
31            'width': x2 - x1,
32            'height': y2 - y1,
33        })

```

El script completo puede encontrarse en la sección D.4.

7.6.5. Conclusiones del despliegue en Jetson

El proyecto ha logrado su objetivo principal: desplegar un modelo YOLOv11 entrenado con imágenes propias de señales de tráfico de juguete en una Jetson Orin Nano, ejecutando inferencia local con GPU's. Partiendo de un dispositivo sin sistema operativo, se completó todo el proceso de instalación y configuración en un entorno de producción funcional.

Las decisiones técnicas más determinantes del proyecto fueron las siguientes:

- Optar por instalación directa en lugar de Docker simplificó el proceso pero requirió gestión manual de dependencias complejas.
- El uso del *flag* `--no-deps` fue esencial para evitar que `pip` sobrescribiera el `torch` de NVIDIA con la versión genérica de PyPI, incompatible con CUDA en ARM.
- La compilación de `torchvision` desde código fuente, aunque costosa en tiempo, fue la única solución viable para garantizar compatibilidad con el `torch` personalizado de NVIDIA.
- La migración de inferencia en nube (Roboflow) a inferencia local (Ultralytics + GPU) reduce la latencia, elimina la dependencia de red y permite operar en entornos sin conexión a Internet.
- La instalación manual de `libcusparselt` cubrió una carencia del JetPack 6.2 que de otro modo habría bloqueado completamente el uso de PyTorch.

Como trabajo futuro, se recomienda exportar el modelo a formato TensorRT (`.engine`) para reducir el tiempo de inferencia de los actuales ~ 212 ms hasta los 20 ms a 50 ms esperados, y activar el modo de máximo rendimiento de la Jetson (`nvpmodel -m 0` y `jetson_clocks`) para maximizar el uso de los recursos de la GPU.

7.7. Integración con la red 4G y el DeepRacer

La integración final del sistema combina los tres bloques desarrollados a lo largo del proyecto:

1. El DeepRacer captura imágenes con su cámara y las envía como datagramas UDP a través de la red LTE privada del laboratorio (módem Huawei $\rightarrow$ BladeRF $\rightarrow$ `srsenb` $\rightarrow$ `srsepc`).
2. El servidor UDP en el *core* reenvía los *frames* a la Jetson mediante una conexión RJ45 directa.
3. La Jetson ejecuta la inferencia con YOLOv11 V7 y devuelve todas las detecciones más la señal más cercana al servidor de control.
4. El servidor evalúa la acción a tomar con la detección más cercana recibida (acelerar, frenar, girar) y la envía al DeepRacer mediante un nuevo datagrama UDP.

La latencia de la cadena completa (cámara $\rightarrow$ 4G $\rightarrow$ *core* $\rightarrow$ Jetson $\rightarrow$ inferencia $\rightarrow$ *core* $\rightarrow$ 4G $\rightarrow$ DeepRacer) se sitúa por debajo de los 500 ms, suficiente para reaccionar a señales sin requerimiento inmediato como límites de velocidad o señalización direccional.

Durante las pruebas se verificó que el sistema completo era capaz de:

- Reaccionar a señales de **STOP** deteniendo el vehículo en menos de 1 s.
- Reducir la velocidad al detectar señales **LIMITE-30**.
- Modificar la trayectoria al detectar señales **OBLIGATORIO-GIRAR-DERECHA** y **OBLIGATORIO-GIRAR-IZQUIERDA**.

- Evitar zonas con CONO y VALLA.
- Frenar suavemente al detectar PROHIBIDO-PASO.

Estos comportamientos validan, en un entorno controlado de laboratorio, la viabilidad del concepto de vehículo conectado con detección *edge* de señales sobre red 4G.

7.7.1. Comparativa de tiempos: nube vs local

La migración a inferencia local supuso una mejora drástica en el tiempo de respuesta. Las mediciones aproximadas en cada arquitectura son las recogidas en la Tabla 7.8.

Tabla 7.8: Comparativa de tiempos de inferencia: nube (Roboflow) vs local (Jetson).

Etapa	Nube (Roboflow)	Local (Jetson + GPU)
Subida de imagen	50 ms–100 ms	—
Inferencia	40 ms	85 ms
Descarga de respuesta	50 ms–100 ms	—
Total medio	~190 ms	~85 ms

La mejora es de un orden de magnitud: de aproximadamente 190 ms en la arquitectura nube (con red 4G saturada) a 85 ms en la arquitectura local, mejorando el retardo de [González, 2022]. Esto es lo que hace viable el control en tiempo real del DeepRacer.

7.8. Escalado IA al proyecto: vehículo conectado en la autovía Valladolid-Segovia

El paso de la demostración del laboratorio a un sistema de producción completamente operativo a lo largo de la autovía A-601 (Autovía de Pinares) exige un replanteamiento en la captura, etiquetado y entrenamiento del modelo de Inteligencia Artificial (IA). El objetivo fundamental de este escalado es que cualquier vehículo conectado que circule por el tramo sea capaz de procesar flujos de vídeo en tiempo real desde sus cámaras integradas, detectando y clasificando instantáneamente la señalización vertical para transmitir alertas críticas de seguridad vial.

Para el desarrollo y despliegue de este modelo de *Machine Learning* a gran escala, se contemplan dos soluciones complementarias:

7.8.1. Entrenamiento desde cero basado en la normativa nacional

Esta estrategia sigue la metodología análoga a la empleada en la demo de este proyecto, pero escalando el volumen de datos de forma masiva. El proceso de entrenamiento requiere la captura y el etiquetado de un *dataset* propio que cubra la totalidad de las señales de tráfico vigentes en el Reglamento General de Circulación de España y tipificadas por la Norma 8.1-IC de señalización vertical [Ministerio de Fomento, 2014] que se encuentren en el tramo de autovía del pliego.

Para garantizar la robustez del algoritmo ante las condiciones reales de la A-601, el *dataset* debe ampliarse para recoger las condiciones climatológicas adversas y habituales de la meseta castellana, tanto en invierno, tales como la niebla densa, la lluvia, la nieve o la presencia de hielo en la calzada, como en verano, calor intenso y luz radiante en el horizonte. Por otro lado se deben emplear técnicas avanzadas de (*data augmentation*) simulando variaciones de luminosidad (cambios día/noche, deslumbramientos). Esto aumentaría el tamaño del *dataset*, y con ello el trabajo de etiquetado, de entrenamiento y de medición de rendimiento, y con ello el coste humano que haría encarecer los costes. Por ello que se plantea la segunda opción.



Figura 7.15: Carrerera A-601, a la altura de la provincia de Segovia, cubierta de nieve. Fuente: ABC

7.8.2. Transferencia de aprendizaje y repositorios públicos

Como alternativa para la reducción de tiempo, coste y computación, el proyecto puede beneficiarse de arquitecturas de CNN profundas de última generación preentrenadas con millones de imágenes de señales de tráfico. Mediante un proceso de ajuste fino (*fine-tuning*) puede hacerse uso de bancos de datos globales de acceso público para entrenar modelos base sobre los que reentrenar.

Existen plataformas de referencia internacional especializadas en la compartición de conjuntos de datos de automoción y visión artificial de alta calidad. Un ejemplo es el repositorio de *Roboflow Universe*, donde se alojan colecciones específicas de señalización vial utilizables bajo este esquema de aprendizaje guiado, pero hay algunos otros:

- **Banco de Datasets Roboflow:** <https://universe.roboflow.com/search?q=traffic+sign>.
- **Banco de Datasets Kaggle:** <https://www.kaggle.com/datasets?search=traffic+signs>
- **Banco de Datasets Hugging face:** <https://huggingface.co/datasets?sort=trending&search=traffic+sign>

7.8.3. Optimización del algoritmo y viabilidad de infraestructura

Independientemente de la metodología de entrenamiento elegida, el modelo resultante debe someterse a un proceso estricto de optimización y evaluación. Esta optimización matemática es un requisito técnico imprescindible para garantizar que el modelo sea lo suficientemente ligero y eficiente como para ejecutarse bajo las condiciones de cómputo que se describirán detalladamente más adelante en el apartado 8.5.1.

Como se adelantará en dicha sección, la arquitectura descarta el uso de costosos servidores dedicados locales (modelo CAPEX) en favor de un despliegue flexible en la nube o en nodos *Multi-access Edge Computing* (MEC) de los operadores de red [ETSI White Paper, 2023]. Por lo tanto, el tamaño del modelo y su tiempo de inferencia por fotograma deben ser mínimos para cumplir con el estricto requisito de latencia impuesto por la conducción. A la velocidad máxima de diseño

de la vía (120 km/h), el vehículo recorre 33,3 m/s; en consecuencia, el retardo acumulado del bucle completo (captura de imagen → transmisión por el enlace ascendente 4G/LTE → inferencia en el nodo MEC → envío de la alerta por el enlace descendente) debe mantenerse en el orden de los milisegundos para que la asistencia al conductor sea efectiva y segura.

7.8.4. Consideraciones de la red y despliegue de mejoras relevantes para la A-601

Para la consolidación del servicio en el tramo de autovía Valladolid-Segovia, se pueden incorporar al diseño los siguientes factores de ingeniería de telecomunicación para la mejora del servicio y de la propia carretera:

- **Aprendizaje Continuo:** El sistema aprovechará la red de comunicaciones para reportar casos de “falso positivo” o detecciones con un índice de confianza inferior al 70 % (por ejemplo, debido a una señal deteriorada físicamente o parcialmente ocluida por vegetación). Estas imágenes se enviarán de forma automática a un servidor centralizado en la nube para su reetiquetado y la actualización progresiva del modelo de IA.
- **Mantenimiento Predictivo:** Podría asociarse cada detección correcta de una señal con las coordenadas geográficas exactas proporcionadas por el módulo GNSS/GPS del vehículo conectado, el sistema podría recopilar información de valor para la Dirección General de Carreteras o la Junta de Castilla y León. Esto permitiría generar un inventario dinámico y automatizado del estado de la señalización vertical de la A-601, detectando de forma prematura la pérdida de señales, roturas o deterioros de señales en la vía.

8 Valoración económica

En este capítulo se cuantifica el coste económico del despliegue propuesto en el Capítulo 5. El despliegue propuesto consiste en **10 estaciones base de nueva planta**, **21 estaciones en alquiler** sobre torres de Vodafone y Telefónica, y un EPC centralizado.

8.1. E-UTRAN — estaciones de nueva planta

La Tabla 8.1 desglosa el coste unitario de una estación base completamente nueva.

Tabla 8.1: Presupuesto de una E-UTRAN completa (nueva planta).

Componente	Unidades	Coste/u (€)	Coste (€)
Antena sectorial Kathrein	2	1 050	2 100
RRU Baicells	2	5 000	10 000
BBU Cambium Networks	1	9 000	9 000
Fuente de alimentación	2	1 400	2 800
PC de control	1	1 500	1 500
Rack 19"	1	500	500
Aire acondicionado	1	4 500	4 500
Generador de emergencia	1	2 499	2 499
Depósito exterior	1	1 505	1 505
Toma de tierra	1	500	500
Acometida eléctrica	1	8 000	8 000
Pararrayos	1	4 000	4 000
Terreno	1	22 500	22 500
Instalación de torre	1	16 500	16 500
Acondicionamiento del sitio	1	13 500	13 500
Cimentación	1	7 000	7 000
Arquetas	2	800	1 600
Caseta	1	800	800
Cerramiento	1	6 600	6 600
Baliza	1	300	300
Sobrecostes (+25 %)	—	—	28 926
Coste por estación nueva			144 630

8.2. E-UTRAN — estaciones en alquiler

Tabla 8.2: Presupuesto de una E-UTRAN en alquiler sobre torre existente.

Componente	Unidades	Coste/u (€)	Coste (€)
Antena sectorial Kathrein	2	1 050	2 100
RRU Baicells	2	5 000	10 000
BBU Cambium Networks	1	9 000	9 000
Rack 19"	1	500	500
PC de control	1	1 500	1 500
Instalación del equipo	1	2 000	2 000
Notaría / contratos	1	300	300
Sobrecostes (+15 %)	—	—	3 810
Coste por estación en alquiler			29 210

8.3. Adquisición del espectro

Asumiendo la cesión de la banda 7 por el operador Vodafone, S.A.U., no se ha de incluir un coste adicional para el alquiler del espectro, sino que la cantidad relacionada con el uso del espectro se reduce al pago de la tasa por uso. Como se detalla en el Anexo A. Legislación, este supone un coste de **166,50€**.

8.4. Resumen E-UTRAN total

Tabla 8.3: Coste total de despliegue de la E-UTRAN.

	Unidades	Coste/u (€)	Coste (€)
Uso espectro	1	166,50	166,50
E-UTRAN nueva planta	10	144 630	1 446 300
E-UTRAN alquiler	21	29 210	613 410
Coste TOTAL E-UTRAN			2 059 876,50 €

8.5. EPC y red de transporte

Tabla 8.4: Presupuesto de despliegue del EPC y la red de transporte.

Componente	Unidades	Coste/u (€)	Coste (€)
Zanja para fibra y acometida (m)	74 000	30	2 220 000
OLT Ubiquiti	4	1 590	6 360
ONT Ubiquiti	31	299	9 269
EPC CISCO (MME, S-GW, P-GW, HSS)	1	78 000	78 000
Fibra óptica monomodo (m)	148 000	2,82	417 212
Coste TOTAL EPC			2 730 841 €

8.5.1. Procesado de IA: servicio en la nube frente a infraestructura propia

A lo largo de este proyecto, se ha sostenido que el paradigma de *Edge Computing* resulta indispensable para garantizar la baja latencia y el correcto funcionamiento en tiempo real de la aplicación de seguridad vial. Los resultados obtenidos en la fase de demostración con la NVIDIA Jetson Orin Nano y la evaluación de los tiempos de inferencia mediante la API de Roboflow validan empíricamente que aproximar el procesamiento del modelo IA al origen de los datos es la única vía eficaz para cumplir con las restricciones temporales de aplicación en tiempo real [Rodríguez Cayetano, 2026]. No obstante, el diseño definitivo de la arquitectura se enfrenta a un dilema recurrente en la ingeniería de telecomunicaciones: un *trade-off* entre el coste de la infraestructura, la latencia de red y la Calidad de Experiencia (QoE) percibida por el usuario.

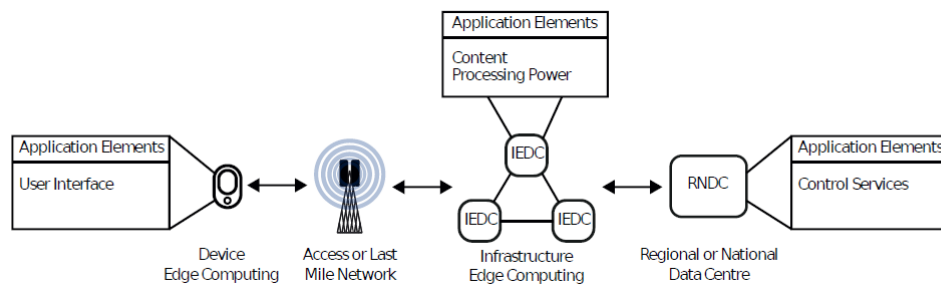


Figura 8.1: Posible arquitectura de *Edge Computing* con alquiler de infraestructura.

Por un lado, la opción ideal desde una perspectiva puramente técnica sería el despliegue de servidores de cómputo dedicados propios (como la opción inicialmente barajada de servidores Dell PowerEdge equipados con GPU NVIDIA RTX A6000¹) situados físicamente en el borde. Sin embargo, asumir la propiedad de estos recursos implicaría una inversión económica inicial (*CAPEX*) inasumible para la empresa y, lo que es más crítico, anularía la escalabilidad del sistema. Una infraestructura rígida y en propiedad carecería de capacidad de escalabilidad frente a incrementos repentinos en la densidad del tráfico de la autovía A-601, y resultaría inviable para absorber el tráfico añadido de futuras actualizaciones, tales como el aumento en la resolución de los flujos de vídeo capturados o la integración de nuevas aplicaciones del ecosistema del vehículo conectado, que proponemos como mejora.

Para resolver este conflicto entre viabilidad financiera y rendimiento, se ha optado por que la estrategia óptima sea la externalización mediante el alquiler o contratación de capacidad de cómputo bajo demanda. Como establece [Rodríguez Cayetano, 2026], la computación en el borde gestionada por terceros está plenamente justificada cuando permite satisfacer las demandas técnicas críticas del servicio (baja latencia y QoE) mitigando las ineficiencias y los riesgos de amortización de una infraestructura propia. De este modo, mediante el despliegue del modelo YOLOv11 en centros de datos del borde de infraestructura (IEDCs), específicamente, el servicio se sustenta sobre el alquiler de capacidad de cómputo en Centros de Datos Micro Modulares (MMDCs) que los operadores de red o proveedores de nube pública (AWS, Azure u Oracle) tienen ubicados junto a las propias estaciones base (eNodeB)[ETSI White Paper, 2023]. Con esta estrategia el equipo maximiza la QoE del servicio sin comprometer los elevados costes. Este modelo transforma un desembolso fijo de capital en un gasto operativo variable (*OPEX*) directamente proporcional al uso real del sistema [Investopedia, 2024], garantizando la escalabilidad futura y trasladando el mantenimiento y la obsolescencia del hardware al proveedor de red.

8.6. Coste de operación anual

Tabla 8.5: Coste de operación anual estimado.

Concepto	Unidades	Coste/u (€)	Coste (€)
E-UTRAN completa (operación)	10	6 862,07	68 620,70
E-UTRAN alquiler (operación)	21	16 841,38	353 668,98
EPC	1	15 086,69	15 086,69
Cómputo de IA en la nube	1	39 420,00	39 420,00
Seguros e imprevistos (+10 %)	—	—	47 679,63
Coste TOTAL anual de operación			524 476,00 €

¹<https://servermall.com/es/sets/servidor-dell-poweredge-r7525/>

El coste de cómputo de IA en la nube se ha estimado de forma conservadora asumiendo tres instancias con GPU operando de forma permanente a un precio medio de 1,5 €/h ($3 \times 1,5 \times 24 \times 365 \approx 39\,420$ € anuales). Este valor es solo orientativo: el modelo de pago por uso de la nube permitiría reducirlo escalando el número de instancias según la demanda real de tráfico (menos instancias en horario nocturno o en tramos de baja IMD).

8.7. Coste total de despliegue

$$\begin{aligned} \text{TOTAL}_{\text{E-UTRAN}} + \text{TOTAL}_{\text{EPC}} &= 2\,059\,876,50 + 2\,730\,841 \\ &= \mathbf{4\,790\,717,50 \text{ €}} \end{aligned} \tag{8.1}$$

A lo que se sumarían **524 476 €/año** de coste de operación. El coste humano del proyecto (sección 3.3) se ha estimado en **10 082 €**, considerando que fuese necesario dedicar las mismas horas que para el proyecto de la asignatura. En caso de necesitar más dedicación se podrá hacer una estimación acorde.

Es necesario indicar que ningún precio incluye I.V.A.

9 Conclusiones y líneas futuras

Este capítulo cierra el informe sintetizando las conclusiones extraídas de los nueve capítulos previos y proyectando las líneas de trabajo futuro que el equipo G2TP2 considera más prometedoras a partir de la experiencia adquirida.

9.1. Conclusiones

El proyecto ha cumplido los objetivos recogidos en el pliego: diseñar una red 4G capaz de soportar un servicio de vehículo conectado sobre una autovía real, valorar económicamente su despliegue, desplegar una demostración funcional en laboratorio y entrenar un modelo de IA con precisión superior al 85 %.

Planificación. El uso de Jira con *sprints* semanales y seguimiento horario por tarea ha proporcionado una trazabilidad efectiva del proyecto y una respuesta ágil ante los obstáculos imprevistos. Su integración con Confluence, GitHub y Drive ha consolidado un ecosistema de gestión completo. El análisis de riesgos elaborado en la semana 3 permitió anticipar los principales contratiempos (daños en la SDR, cuellos de botella en vídeo, incompatibilidades de software), que se materializaron parcialmente pero fueron gestionados sin afectar al cronograma.

Red 4G. Se ha diseñado un despliegue de **31 estaciones base** (10 de nueva planta y 21 en alquiler sobre torres de Vodafone y Telefónica) que garantiza $\text{RSRP} > -92 \text{ dBm}$ en el 100 % de los 104 km del trazado Valladolid–Segovia. La elección del modelo de propagación SUI frente a otras alternativas (Okumura-Hata, COST 231, modelos de difracción) ha resultado clave para obtener predicciones fiables en la banda 7 (2,6 GHz) sobre un entorno interurbano arbolado.

Demo de laboratorio. La decisión técnica más determinante ha sido sustituir Docker Desktop por Docker Engine nativo, sin la cual el SDR BladeRF habría sido inaccesible desde el contenedor. La arquitectura de dos máquinas con `--network host` ha reflejado fielmente el modelo real de una red celular y ha simplificado la comunicación S1AP entre eNodeB y EPC. El despliegue, estructurado en 13 fases, ha permitido avanzar de forma sistemática, resolviendo en cada paso un obstáculo concreto y habilitando el siguiente.

Inteligencia artificial. Tras siete iteraciones, el modelo final YOLOv11 V7 alcanza $\text{mAP@50} = 0,967$, $\text{Precisión} = 0,940$ y $\text{F1} = 0,937$, superando con holgura el objetivo del 85 %. La calidad del *data-set* (especialmente el reetiquetado del corpus en V2) ha tenido más impacto que los cambios de hiperparámetros o arquitectura. La migración de la inferencia a la Jetson Orin Nano ha reducido la latencia de $\sim 190 \text{ ms}$ (nube) a $\sim 85 \text{ ms}$ (local), eliminando la dependencia de conectividad externa.

Viabilidad económica. El coste total de inversión estimado asciende a aproximadamente **5 millones de euros** (E-UTRAN nueva planta: 1 446 300 €; E-UTRAN en alquiler: 613 410 €; EPC y red de transporte: 2 730 841 €) más $\sim 524 476 \text{ €}$ anuales de operación. El modelo de cesión de espectro y el *site sharing* reducen sustancialmente la inversión, pero su magnitud refuerza la necesidad de un despliegue compartido entre operadores y administraciones públicas.

En el siguiente enlace puede consultarse la demostración final del sistema completo (red 4G, inferencia local de señales y conducción autónoma del DeepRacer), que da cumplimiento a la cláusula segunda del pliego:

[Enlace a la presentación de la demostración](#)

9.2. Líneas futuras

Diseño de la red. La línea de mayor impacto es la **migración a 5G** de la parte RAN. La obra civil existente (torres, fibra, OLTs) sería reutilizable, y la latencia inferior a 5 ms de 5G NR

abriría la puerta a niveles de automatización SAE 4. A más corto plazo, un refinamiento del emplazamiento de las estaciones basado en un análisis estadístico de la distribución SNR–distancia podría mejorar la capacidad efectiva sin coste adicional. Complementariamente, un estudio del tráfico real (IMD 2019–2025) con modelos M/M/c permitiría ajustar el dimensionamiento de capacidad por celda.

Laboratorio. Las mejoras prioritarias son la **automatización del despliegue** mediante `Dockerfile` declarativo y `docker-compose` (eliminando la dependencia de imágenes congeladas), el despliegue **multi-eNB** con dos BladeRF para validar el *handover*, y la migración del *core* a Open5GS u OpenAirInterface para disponer de mejor soporte multi-usuario y de 5G. La adición de autenticación al *broker* MQTT y la captura sistemática de métricas con `iperf3` completarían la madurez del demostrador.

Detección de señales. La exportación del modelo a **TensorRT** (`.engine`) reduciría la latencia de inferencia hasta los 20 ms a 50 ms, activando los modos de máximo rendimiento de la Jetson. La ampliación del *dataset* con capturas reales de la A-601 en condiciones variadas, junto con el reentrenamiento balanceando las clases minoritarias (PELIGRO-OPERARIO-OBRAS, LIMITE-90, PELIGRO-NIÑOS), mejoraría la generalización. La incorporación de un algoritmo de seguimiento entre *frames* evitaría acciones duplicadas ante la misma señal.

Escalado del servicio. La arquitectura diseñada es directamente replicable en otras autovías secundarias de Castilla y León. A nivel técnico, las dos extensiones más relevantes son la integración con **C-V2X** (3GPP *Release* 16) para comunicación directa entre vehículos, y el despliegue de **Edge Computing real** sobre nodos MEC de los operadores, llevando la inferencia al pie del eNodeB para minimizar la latencia y reducir el coste de hardware embarcado en el vehículo.

A Documentación legal y permisos

El despliegue de una estación base LTE (eNodeB) para el desarrollo de un servicio de vehículo conectado requiere la realización de una serie de procedimientos administrativos destinados a garantizar el cumplimiento de la normativa vigente en materia de telecomunicaciones, uso del dominio público radioeléctrico y compatibilidad electromagnética.

En el presente proyecto, el sistema desplegado tiene carácter experimental y temporal, utilizándose exclusivamente para la validación tecnológica del servicio de transmisión de vídeo y detección de señales mediante inteligencia artificial. Asimismo, el uso del espectro radioeléctrico no se realiza mediante la obtención de un título habilitante propio, sino a través de una cesión temporal de frecuencias por parte de un operador autorizado, concretamente Vodafone S.A.U.

El marco regulatorio aplicable al despliegue viene definido principalmente por la Ley 11/2022, General de Telecomunicaciones [Jefatura del Estado, 2022], el Real Decreto 123/2017 sobre uso del dominio público radioeléctrico [Ministerio de Energía, Turismo y Agenda Digital, 2017] y el Real Decreto 1066/2001 sobre emisiones radioeléctricas [Ministerio de la Presidencia, 2001]. Estas disposiciones regulan tanto las condiciones de utilización del espectro como las obligaciones técnicas y administrativas asociadas a la instalación y operación de estaciones radioeléctricas.

Con el objetivo de garantizar la legalidad del despliegue y cumplir con la cláusula cuarta del pliego del proyecto, se han considerado los siguientes procedimientos administrativos.

A.1. Cesión de uso del espectro radioeléctrico

Dado que el proyecto se desarrolla sobre frecuencias ya asignadas a un operador autorizado, no resulta necesario solicitar un nuevo título habilitante para el uso del dominio público radioeléctrico. En su lugar, el uso de las bandas de frecuencia se realiza mediante una cesión temporal autorizada por Vodafone S.A.U., titular de los derechos de uso correspondientes.

La cesión incluye las frecuencias pertenecientes a la banda 7 de LTE, concretamente el rango de 2540–2550 MHz para el enlace ascendente (uplink) y 2660–2670 MHz para el enlace descendente (downlink). Estas frecuencias serán utilizadas exclusivamente dentro del entorno experimental definido para el demostrador.

La formalización de este acuerdo resulta esencial para acreditar la disponibilidad legal del espectro radioeléctrico y justificar la legitimidad de las transmisiones realizadas por la estación base desplegada.

UVa – ETSIT – MUIT

Proyecto Vehículo Conectado

G2TP2

ACUERDO DE CESIÓN DE USO DEL ESPECTRO RADIOELÉCTRICO

En Valladolid a 10 de febrero de 2026

REUNIDOS

De una parte:

Vodafone S.A.U., como entidad titular de derechos de uso del dominio público radioeléctrico.

Y de otra:

G2TP2, como entidad responsable del desarrollo del proyecto experimental de vehículo conectado LTE.

EXPONEN

Que Vodafone S.A.U. dispone de derechos de uso sobre la banda de frecuencias correspondiente a la banda 7.

Que la entidad cesionaria desarrolla un proyecto experimental basado en tecnología LTE destinado a la validación de sistemas de vehículo conectado.

ACUERDAN

Primero

Ceder temporalmente el uso de las siguientes frecuencias:

- Enlace ascendente (UL): 2540–2550 MHz
- Enlace descendente (DL): 2660–2670 MHz

Segundo

El uso del espectro se limita exclusivamente a fines experimentales y de validación tecnológica.

Tercero

El ámbito geográfico autorizado corresponde al tramo de la autovía A-601 kilómetro 0 a kilómetro 104.

Cuarto

La duración de la cesión será de 6 meses.

Quinto

La entidad cesionaria se compromete a:

- No causar interferencias perjudiciales
- Cumplir la normativa vigente
- Limitar el uso al ámbito experimental autorizado

Y en prueba de conformidad, se firma el presente documento.

FIRMAN

Por Vodafone S.A.U.

Nombre: _____

Por la entidad solicitante

Nombre: _____

Documentación administrativa

Figura A.1: Contrato de cesión del espectro radioeléctrico.

A.2. Pago de tasas por uso del dominio público radioeléctrico

Aunque el espectro utilizado sea cedido por un operador autorizado, el uso efectivo del dominio público radioeléctrico continúa sujeto a obligaciones administrativas y económicas. En particular, el uso experimental de frecuencias requiere la liquidación de las tasas correspondientes asociadas a la utilización del espectro.

Para ello, se ha considerado la utilización del modelo 790, documento oficial empleado para la autoliquidación de tasas en materia de telecomunicaciones. Este procedimiento permite regularizar administrativamente el uso de frecuencias y garantizar el cumplimiento de las obligaciones económicas derivadas de la operación de la estación radioeléctrica. El pago de la tasa puede realizarse a través de la sede electrónica del Ministerio para la transformación digital y la función pública [Ministerio para la Transformación Digital y de la Función Pública, 2026c].

La inclusión de este trámite resulta especialmente relevante en entornos experimentales, ya que demuestra que incluso los despliegues temporales deben respetar las condiciones administrativas establecidas para la utilización del dominio público radioeléctrico.

**CENTRO GESTOR**

Min. de Ciencia, Innovación y Universidades

TASA: Tasa por Homologación, equivalencia a titulación y a nivel académico, y convalidación de títulos y estudios extranjeros.

CÓDIGO 1 0 7

Modelo
790

Identificación (1)	Espacio reservado para la etiqueta identificativa del sujeto pasivo. Si no dispone de etiquetas, consigne los datos que se solicitan en las líneas inferiores.		Deven- go (2)		Ejercicio..... 2 0 2 6		
	Nº de Justificante: 790 1079658692						
Autoliquidación (3)	1. N.I. 72263457X		2. Primer Apellido LOPEZ		3. Segundo Apellido GARCIA		
	4. Nombre LUCIA		5. Calle/Piza./Avda. PASEO		Nombre de la vía pública. BELEN		
	6. Teléfono		Número 15		Esc. Piso Puerta		
	7. Municipio VALLADOLID		8. Código Postal 47011		9. Provincia VALLADOLID		
10. País ESPAÑA							
DECLARANTE	Firma:		INGRESO Ingreso efectuado a favor del Tesoro Público, cuenta restringida de la A.E.A.T., para la Recaudación de Tasas. Importe Euros: I 166 50 Forma de pago: En efectivo ☒ E.C Adeudo en cuenta ☐ Código IBAN del cliente				
HECHO IMPONIBLE TARIFA a) Solicitud de Homologación de títulos extranjeros de educación superior a títulos oficiales universitarios españoles de Grado o Máster que den acceso a profesión regulada en España. ☐ 166,50 euros b) Solicitud de Equivalencia de título extranjero de educación superior a nivel académico de Grado o Máster universitarios oficiales en ramas de conocimiento y campos específicos. ☒ 166,50 euros NOTA IMPORTANTE: Este justificante de pago no acredita que se haya iniciado ningún expediente de Homologación de título extranjero de educación superior a título oficial universitario español, ni de Equivalencia a nivel académico de Grado o Máster en la Subdirección General de Títulos, del Ministerio de Universidades, por lo que no podrá ser utilizado como justificante de título para la inscripción en los Colegios profesionales o en las Universidades.							

Este documento no será válido sin la certificación mecánica o, en su defecto, firma autorizada

Ejemplar para la Administración

NOTA: ANTES DE CONSIGNAR LOS DATOS, VEA LAS INSTRUCCIONES AL FINAL DEL DOCUMENTO

Figura A.2: Justificante de pago de tasas (Modelo 790)

A.3. Solicitud de autorización de puesta en servicio de la estación radioeléctrica

El principal procedimiento administrativo asociado al despliegue de la estación base corresponde a la solicitud de autorización de puesta en servicio de la estación radioeléctrica. Este trámite permite a la administración verificar que la instalación cumple con los requisitos técnicos exigidos y que su funcionamiento no ocasionará interferencias perjudiciales sobre otros sistemas radioeléctricos [Ministerio para la Transformación Digital y de la Función Pública, 2026a].

La solicitud recoge información detallada sobre la estación, incluyendo:

- ubicación del emplazamiento,
- características técnicas del sistema LTE,
- bandas de frecuencia utilizadas,
- potencia de transmisión,
- finalidad experimental de la instalación.

Asimismo, este procedimiento permite formalizar administrativamente la existencia de la estación base y habilitar legalmente su operación durante el periodo de pruebas definido en el proyecto.

La autorización de puesta en servicio constituye, por tanto, el elemento central del procedimiento administrativo necesario para la operación de la red LTE experimental. Esta puede rellenarse y presentarse en la sede del Ministerio.

Solicitud de autorización para la puesta en servicio de estaciones radioeléctricas

D./D^a LUCIA LOPEZ GARCIA, con NIF/NIE 72263457X, en nombre y representación de G2TP2 - Proyecto Vehículo Conectado LTE, con NIF N/A, y domicilio en Paseo de Belén, 15, con código postal 47011 y con correo electrónico para envío de avisos de notificación lucia.lopez25@estudiantes.uva.es.

EXPONE

1. Que desea poner en servicio:
 - ☒ Estación de telefonía móvil o acceso inalámbrico fijo.
 - ☐ Estación de radiodifusión sonora o de televisión.
 - ☐ Estación de radioafición con distintivo de llamada .
 - ☐ Resto de estaciones radioeléctricas.
2. Que actúa en calidad de:
 - ☐ Titular/cesionario de derecho de uso privativo del dominio público radioeléctrico, cuyo expediente administrativo tiene referencia .
 - ☐ Titular de una autorización individual para el uso especial del dominio público radioeléctrico¹, cuyo expediente administrativo tiene referencia .
 - ☒ Operador de comunicaciones electrónicas², autorizado por los titulares del derecho de uso del dominio público radioeléctrico con expedientes administrativos Cesión temporal de uso experimental de espectro por Vodafone S.A.U.
 - ☐ Gestor del múltiple digital³, autorizado por los titulares del derecho de uso del dominio público radioeléctrico con expedientes administrativos .
 - ☐ Promotor de extensión de cobertura de televisión digital terrestre³(TDT), autorizado por los titulares del derecho de uso del dominio público radioeléctrico con expedientes administrativos .
 - ☐ Otros⁴ (especificar): , autorizado por los titulares del derecho de uso del dominio público radioeléctrico con expedientes administrativos .
3. Que los propietarios de las instalaciones⁵ o, en su caso, los titulares de uso de las instalaciones son: Universidad de Valladolid, con NIF Q4718001C.
4. Que el operador de comunicaciones electrónicas⁶ que realiza las emisiones es: G2TP2 – Proyecto Vehículo Conectado LTE, con NIF N/A.
5. Que previamente ha obtenido la autorización de la Secretaría de Estado para la Sociedad de la Información y la Agenda Digital para realizar las instalaciones, cuyo expediente administrativo tiene referencia EXP-LTE-2026-001 , y que se han ejecutado las instalaciones de acuerdo con los parámetros y características técnicas autorizadas por dicha Secretaría de Estado.

6. Que las instalaciones cumplen con las medidas de seguridad establecidas en la normativa vigente y han sido ejecutadas por:
- ☒ Una empresa instaladora de telecomunicaciones, inscrita en el *Registro de Empresas Instaladoras de Telecomunicación* de la Secretaría de Estado para la Sociedad de la Información y la Agenda Digital para la instalación de estaciones de radiocomunicaciones (tipo D).
 - ☐ El radioaficionado, previamente autorizado por el Jefe Provincial de Inspección de Telecomunicaciones para efectuar dicha instalación.
7. Que, con esta solicitud, se adjunta:
- ☐ Boletín de instalación⁷ y la documentación que lo acompaña, cumplimentado y firmado por la empresa instaladora de telecomunicaciones que ha realizado las instalaciones, para cada una de las estaciones objeto de esta solicitud.
 - ☐ Características técnicas de la instalación⁸, cumplimentado y firmado por el radioaficionado que ha realizado la instalación.
 - ☐ Seguro de responsabilidad civil⁹, establecido en el artículo 20 del Real Decreto 2623/1986, de 21 de noviembre, *por el que se regulan las instalaciones de antenas de estaciones radioeléctricas de aficionado*.
 - ☒ Justificante que acredita el pago de la tasa de telecomunicaciones¹⁰, establecida en el apartado 4 del Anexo I de la Ley 9/2014, de 9 de mayo, *General de Telecomunicaciones*, para cada una de las estaciones objeto de esta solicitud.
 - ☒ Certificado de instalación¹¹, sustitutivo del acto de reconocimiento técnico de las instalaciones, expedido por técnico competente en materia de telecomunicaciones, en conformidad con la Resolución de 4 de mayo de 2017, de la Secretaría de Estado para la Sociedad de la Información y la Agenda Digital, *por la que se determinan los tipos de estaciones radioeléctricas para los que se requiere una certificación sustitutiva del acto de reconocimiento técnico previo a la autorización para la puesta en servicio*, para cada una de las estaciones objeto de esta solicitud.
 - ☐ Certificado de niveles de exposición radioeléctrica¹², expedido por técnico o profesional competente para realizar medidas de niveles de exposición radioeléctrica, de que los niveles existentes, en zonas cercanas a cada una de las estaciones objeto de esta solicitud que lo requieran, cumplen los límites de exposición radioeléctrica establecidos en el Anexo II del *Reglamento que establece las condiciones de protección del dominio público radioeléctrico, restricciones a las emisiones radioeléctricas y medidas de protección sanitaria frente a emisiones radioeléctricas*, aprobado por el Real Decreto 1066/2001, de 28 de septiembre.

SOLICITA a la Secretaría de Estado para la Sociedad de la Información y la Agenda Digital:

- ☒ La autorización para la puesta en servicio de las estaciones radioeléctricas indicadas en la documentación que se adjuntan a esta solicitud.
- ☐ La expedición de la correspondiente licencia de estación de radioafición¹³.

En Valladolid, a 10 de febrero de 2026.

[Firmese la solicitud ^{14 y 15}]

**Signed by LUCIA LOPEZ
GARCIA on 10/02/2026
with certified
electronic signature.**

Figura A.3: Modelo de solicitud APS.

A.4. Cumplimentación técnico y carácter experimental del sistema

Además de los procedimientos administrativos anteriores, el despliegue de la estación base se ha diseñado considerando los requisitos técnicos y de seguridad establecidos por la normativa vigente [Jefatura del Estado, 2022].

En particular, la configuración del sistema LTE se ajusta a las especificaciones técnicas aplicables a redes 4G y se han considerado las limitaciones relativas a compatibilidad radioeléctrica y niveles de emisión electromagnética, de conformidad con lo dispuesto en el RD 1066/2001 [Ministerio de la Presidencia, 2001]. Asimismo, el sistema desplegado tiene carácter exclusivamente experimental, contando con la correspondiente autorización para el uso del dominio público radioeléctrico conforme al RD 123/2017 [Ministerio de Energía, Turismo y Agenda Digital, 2017], sin prestación de servicios comerciales y con una duración limitada al periodo de pruebas definido en el proyecto.

La utilización de una estación base temporal y de un entorno controlado de pruebas permite garantizar que el despliegue se realiza de forma segura y conforme a las condiciones establecidas por la normativa aplicable.

En conjunto, los procedimientos administrativos y técnicos descritos permiten asegurar la viabilidad legal y operativa del demostrador LTE desarrollado para el servicio de vehículo conectado.

B Configuración resumida del laboratorio

B.1. Configuración de red

Tabla B.1: Direccionamiento IP empleado en el laboratorio.

Elemento	IP
Host del core (LAN del laboratorio)	10.0.128.176
Host del eNodeB (LAN del laboratorio)	10.0.128.181
Gateway UE (interfaz <code>srs_spgw_sgi</code>)	172.16.0.1/24
UE asignada (Módem DeepRacer)	172.16.0.2/24
IP interna DeepRacer	192.168.8.105/24
PC laboratorio (RJ45 a Jetson)	192.168.100.101/24
Jetson Orin Nano (RJ45 a PC)	192.168.100.102/24

B.2. Parámetros radio del eNodeB

Tabla B.2: Parámetros de la celda LTE del laboratorio.

Parámetro	Valor
PLMN (MCC + MNC)	901 + 70
TAC	7
APN	srsapn
Algoritmo cifrado	EEA0
Algoritmo integridad	EIA1
EARFCN	3350
DL central	2680 MHz
UL central	2560 MHz
PRB / Ancho de banda	50 / 10 MHz
SDR	BladeRF 2.0 micro xA9
Bitstream FPGA	hostedxA9.rbf v0.15.3
Software base	srsRAN 4G + Docker Engine 27.x

B.3. Resumen del entorno Jetson

Ya recogido en la Tabla 7.7 del cuerpo principal del informe.

C Diario de trabajo del proyecto

En este anexo se recoge el diario completo del proyecto, semana a semana, tal como fue documentado en tiempo real por el equipo G2TP2 en la página **Diario y decisiones tomadas** de Confluence. El diario refleja el desarrollo cronológico del proyecto, las decisiones técnicas tomadas en cada momento y los problemas encontrados, así como el reparto del trabajo entre los miembros del equipo. Se respeta la división por *sprints* semanales empleada en Jira.

C.1. Semana 1 (9/2 – 15/2)

En la reunión inicial del lunes 10 de febrero se estableció una planificación inicial del equipo, repartiendo el responsable principal de cada área (Planificación, Infraestructura 4G e Inteligencia Artificial) e indicando las tareas a realizar durante la primera semana. Las áreas se reparten como sigue: Lucía López coordina el bloque de planificación y documentación; Miguel Barrigón y Rodrigo Seco se reparten la parte de infraestructura 4G; y Víctor Sánchez asume el bloque de IA, con apoyo del resto del equipo.

Bloque de Planificación (PLAN). Se creó el espacio Jira TP2CAR para que de manera conjunta el equipo pudiera tener un entorno donde colocar las tareas a desarrollar, asignar los recursos, mantener todas las notas necesarias para la documentación, un calendario para la visualización temporal del proyecto y algunas estadísticas de rendimiento. En definitiva, un espacio multiusos que permite planificar de la mejor manera posible este proyecto.

Se generaron también los archivos solicitados por A. Bahillo (objetivos, riesgos y tareas) que se mostrarían en su siguiente sesión, y se creó la base del informe técnico-económico.

Bloque de Infraestructura 4G (INF). Se realizó la instalación exitosa de Ubuntu 22.04 LTS en ambos ordenadores destinados a hacer las funciones de *core* y eNodeB. El único problema encontrado fue la configuración del teclado en español, que se resolvió tras varios intentos. Se comenzó la descarga de srsRAN, con previsión de finalizarla la semana siguiente. El sistema Docker dio fallos de compatibilidad iniciales (que más adelante se identificarán como originados por el uso de Docker Desktop en lugar de Docker Engine), por lo que su descarga y configuración inicial se postpone a la semana 2.

Bloque de Inteligencia Artificial (IA). Tras buscar diferentes entornos para el desarrollo de IA, se optó por usar Roboflow, una herramienta *online* para etiquetar imágenes que permite dividir en *train/validation/test* y entrenar modelos para identificar las señales de tráfico del tramo objetivo. Se adquirieron tres *datasets* cedidos por equipos anteriores que habrá que limpiar y verificar la próxima semana.

C.2. Semana 2 (16/2 – 22/2)

Bloque INF. Instalación exitosa de Docker. Tras leer documentación se decidió instalar 4G en lugar de 5G (por madurez y disponibilidad de *open source* estable). Se completó la instalación de srsRAN y se inició el trabajo con BladeRF.

Bloque IA. Se etiquetaron todas las imágenes del *dataset* previo, obteniendo el primer corpus combinado. Se acordó realizar más sesiones fotográficas con la cámara del DeepRacer en las próximas semanas para reforzar las clases con menor representación.

Bloque PLAN. Se comenzó la lectura de la documentación sobre los permisos legales necesarios para la emisión en bandas de RF.

El miércoles 19 de febrero se completó la instalación de la BladeRF en el *core*, y la instalación de srsRAN ya estaba avanzada. Por indicaciones de A. Bahillo se elaboró una previsión exhaustiva de todas las tareas del proyecto desde el principio hasta el final, completando el cronograma en Jira con los hitos principales de cada bloque (Figura 3.1).

C.3. Semana 3 (23/2 – 1/3)

El lunes 24 de febrero se asistió al **primer seminario de Planificación** impartido por Daniel, experto de la empresa Renault, que ayudó al equipo en la planificación del proyecto y en la confección del análisis de riesgos (Tabla 3.1).

Bloque IA. Se finalizó la revisión del etiquetado del corpus inicial y se realizó un análisis de las carencias del *dataset*. Tras analizar los distintos modelos disponibles en la plataforma Roboflow, se eligió YOLOv11 como modelo objetivo. Se preparó la versión v3 2026-02-24 del corpus redimensionada a 640×640 (recomendación de Ultralytics) y se entrenó el primer modelo (V1) con resultados que se documentaron en la página **ENTRENAMIENTO** de Confluence.

Bloque PLAN. Se inició la lectura sobre la normativa vigente de Telecomunicaciones de cara a solicitar un permiso de emisión en bandas de RF. Se identificaron los seis documentos administrativos necesarios para una autorización experimental.

C.4. Semana 4 (2/3 – 8/3)

Bloque INF. Esta semana se logró activar Docker tras muchas dificultades. El problema de raíz era que Docker Desktop ejecutaba los contenedores en una VM con kernel `linuxkit` que impedía el acceso al USB. La solución — que sería confirmada las semanas siguientes — fue migrar a Docker Engine nativo.

Bloque IA. Aunque el *dataset* anterior estaba revisado y se habían identificado las carencias, todavía no se hicieron las nuevas fotos porque se necesitaba acceder al coche y no se disponía de las pantallas necesarias. Se decidió que el procedimiento sería: primero entrenar el modelo en Roboflow (en la nube), después ejecutar la inferencia en local con PyCharm, y finalmente probar en la Jetson.

C.5. Semana 5 (9/3 – 15/3)

Bloque INF. Esta semana se terminó de descargar srsRAN y se obtuvo acceso completo a la BladeRF, también consiguiendo que dentro del contenedor Docker se levantara el *core* (en el ordenador nuevo) y el eNodeB (en el ordenador viejo). Se está en proceso de acceder a la tarjeta SIM para que el coche tenga acceso a la red 4G. Se comenzó también a acceder al coche, ver qué contenidos tiene y cómo comunicarse con él. Se intentó capturar imágenes desde la cámara con código Python.

Bloque IA. Se descargó el entorno PyCharm para hacer funcionar el modelo localmente. Para tener todo preparado cuando el modelo final estuviera disponible, se decidió ir probando con el modelo simulado actual. Se accedió a la cámara del coche con el comando `cheese` y se tomaron **10 fotografías de señales relevantes** (recto, giros, STOP, prohibido, personas) para incluirlas en el modelo.

C.6. Semana 6 (16/3 – 22/3)

Bloque IA. Esta semana se trabajó en dos aspectos: por un lado, se accedió al coche y a la cámara de manera local (mediante cableado), realizando capturas nuevas para incorporar al *dataset*; por otro, se utilizó el modelo previamente entrenado a través de un *script* en Python, obteniendo los primeros resultados de inferencia.

Adicionalmente, se decidió **re-etiquetar el corpus existente** para eliminar las clases que no serían necesarias en la demostración y que estaban introduciendo confusión en el modelo. Se empezó también a leer y utilizar la Jetson para incorporar el modelo de IA dentro del coche y realizar la detección y clasificación localmente.

Bloque INF. Se terminó de configurar el acceso a la tarjeta SIM, de cara a tener acceso al coche desde la misma red que el *core* y el eNodeB, y proporcionar acceso a Internet al coche. El jueves se asistió al **seminario sobre Infraestructura y 4G** impartido por Ángel María Martín, ingeniero de Vodafone.

C.7. Semana 7 (23/3 – 29/3)

Bloque IA. Se ajustó el *dataset* antiguo corrigiendo las clases que no se iban a emplear. Se incluyeron nuevas fotografías para mejorar la detección del modelo y se etiquetaron. Se siguen analizando las carencias del *dataset* actual para programar las próximas sesiones de captura.

Bloque INF. Se siguió intentando la conexión con el coche, en esta ocasión a través de MQTT. La BladeRF presentó dificultades con la conexión a la SIM, por lo que la conexión completa no fue posible esta semana.

Bloque PLAN. Se descargaron los modelos de documentos legales que habrá que rellenar cuando se confirme cuántos son necesarios. El orden propuesto de documentos con su justificación está documentado en la página **LEGISLACIÓN** de Confluence.

C.8. Semana 8 (13/4 – 19/4)

Bloque IA. Se mejoró el modelo con nuevas imágenes de las clases que se confundían (V3), obteniendo un modelo estable que permitía continuar con el proyecto. Se consiguió que el modelo corriera en la Jetson, dando predicciones de las etiquetas encontradas y mostrando por terminal la etiqueta más cercana con confianza superior al 80 %. Quedaba pendiente la conexión con el coche para procesar el vídeo en tiempo real.

Bloque INF. Se obtuvo conexión exitosa al coche con MQTT para enviar comandos a través de la infraestructura. Se ejecutó el fichero `manual_control.py`, obteniendo imagen del coche, y mediante comandos como `w` se consiguió que el coche moviera sus ruedas hacia adelante y girara. Se logró también obtener un fragmento de vídeo. A mitad de semana se perdió la conexión con el coche, posiblemente por interferencias entre los grupos del aula. La semana siguiente se planeaba probar por Wi-Fi para descartar problemas internos del propio coche.

Bloque INF (escalado). Se inició la configuración del estudio de cobertura en Xirio: creación del estudio, definición del tramo de carretera y elección del modelo de antena. Se realizó una primera prueba de cobertura y se añadieron los emplazamientos correspondientes a las antenas existentes de Vodafone que cubrirían el tramo. El siguiente paso, vincular antenas a esos emplazamientos y realizar el cálculo de cobertura, se completaría en la semana 9.

C.9. Semana 9 (20/4 – 26/4)

Bloque INF (escalado). Se completó el estudio de cobertura. Comenzando por las antenas disponibles de Vodafone (empresa con la que se tiene la licencia simulada), se observó que existían zonas de sombra. Se incluyeron antenas de otras empresas (Telefónica) susceptibles de cooperación, obteniendo una mejora pero aún con puntos críticos. Tras añadir 10 nuevas torres en los puntos de sombra detectados, se obtuvo cobertura total del trazado con $\text{RSRP} > -92 \text{ dBm}$. Se especificaron además los parámetros del estudio (frecuencia, potencia, rangos de cobertura). El apartado quedó redactado en una página de Jira para que solo fuera necesario copiar y pegar al informe.

Bloque INF (laboratorio). Se logró la conexión completa con el coche a través de la red 4G, indicando comandos para la conducción y obteniendo el vídeo en tiempo real. Se probó la conexión por Wi-Fi como referencia y, como era de esperar, se obtuvo una mejor calidad.

Con conexión 4G + ejecución del modelo en Jetson se obtuvo detección de señales con correcto funcionamiento del modo manual.

Bloque IA. Al ejecutar el modelo en la Jetson se observaron carencias: confusión derecha/izquierda y entre los límites 30/90 según la posición de la señal. La señal de niños no se detectaba. Se programó nueva sesión de fotos para la semana siguiente.

C.10. Semana 10 (27/4 – 3/5)

Bloque INF. A lo largo de la semana se ejecutó el modelo de IA en la Jetson recibiendo el vídeo en tiempo real del coche. Aunque hubo problemas iniciales de latencia y cortes en el vídeo, se lograron solventar. Con el vídeo en tiempo real se obtuvo la inferencia de las señales del modelo en la Jetson.

Los **problemas encontrados** fueron los siguientes:

- **Problemas de conexión**, en la red 4G y Wi-Fi: alta latencia y cortes del vídeo en tiempo real. Se consultó si había posibilidad de obtener una marca temporal de los *frames* recibidos del vídeo del coche para caracterizar el retardo, pero no había manera de obtener esa referencia temporal con el *script* disponible.
- **Modelo que no reconoce algunas señales y confunde otras.**
- **Conflicto de IPs** al cambiar a la infraestructura 4G: el *core* en 10.0.128.176 y la Jetson en 192.168.172.51 (Wi-Fi del laboratorio) no podían comunicarse directamente. Se dedicó una sesión completa a este problema y se optó por una conexión RJ45 directa entre el *core* y la Jetson para que pudieran verse, configurando IPs estáticas en una subred dedicada.

Bloque IA. Se realizaron nuevas fotos el 27 de abril y se entrenó un nuevo modelo (V4). Este modelo se entrenó con *augmentation* agresiva y resultó tener peores prestaciones que V3. Se entrenó entonces V5 con el mismo *dataset* pero **sin augmentation**: el número de fotogramas efectivos fue menor, llevando a peores resultados que V3 (no reconocía señales que estudios previos sí detectaban).

Con esta lección, se entrenó **V6 con *augmentation* selectiva**, deseleccionando las transformaciones que afectan a la detección de etiquetas direccionales (giros) y de magnitud (límites). Tras probar V6 las prestaciones eran aceptables pero seguía habiendo confusiones residuales. Se entrenó por último **V7** usando el mismo *dataset* pero modificando el preprocesado y la estrategia de *augmentation*: división vertical de la imagen, redimensionado a 640 × 640 y ajuste de contraste con múltiples exposiciones. V7 ofreció prestaciones excelentes (mAP@50=0,967) y se considera la versión definitiva.

C.11. Semana 11 (4/5 – 10/5)

La semana 11 empezó asistiendo a un seminario muy interesante sobre 5G impartido por Ángel Álvés de Telefónica dónde en esta primera sesión se ha hablado de las características principales de la red móvil y de su evolución desde el 2G hasta 5G y dando pinceladas sobre como se esta diseñando el 6G.

- **Bloque INF.** Se ha dedicado a solventar algunos problemas relacionados con la cámara del coche cuando se realizaban pruebas en la alfombra. Se probó a realizar la demo completa a través de la infraestructura de 4G y corriendo la JETSON conectada al host del core. Este día se ha comprobado que el coche es capaz de detectar de manera adecuada todas las señales. Sin embargo no ha sido capaz de reaccionar a estas señales. Por un lado queda pendiente que el coche mejore la cámara ya que se sigue quedando congelada y que al obtener las etiquetas en la JETSON que reaccione a estas señales.

- **Bloque DOC.** También se ha dedicado a iniciar la redacción de la memoria técnico-económica y preparación de la presentación final. Quedaron pendientes algunas mejoras para líneas futuras:

C.12. Semana 12 (11/5 – 17/5)

- **Bloque INF.** A lo largo de esta semana se ha estado realizando pruebas con el reconocimiento de las señales y la reacción del coche para realizar una demostración de cara a la presentación. Las pruebas no han ido del todo mal, el coche reaccionó de manera adecuada a las señales pero se hicieron algunas correcciones para la reacción del coche a la señal de STOP y PROHIBIDO-PASO. Además se hicieron correcciones para el giro y dirección del coche.
- **Bloque DOC.** Por otro lado a lo largo de esta semana se ha continuado escribiendo este informe y sus anexos para realizar la entrega en tiempo y forma. Cada uno de los integrantes se ha encargado de la sección de la cual ha sido responsable durante el proyecto. De manera que nos hemos repartido el informe para avanzar más rápido.

C.13. Semana 13 (18/5 – 24/5)

- **Bloque INF.** Durante esta semana todo el equipo ha estado involucrado en la preparación de la demo, diseñando el circuito, la corrección de errores en el script y la grabación de las demos. A lo largo de la semana nos hemos encontrado con diversos problemas con el funcionamiento de manera correcta del coche, principalmente con las reacciones del coche a las señales o con controles de giro, tiempos o velocidades. A lo largo de la semana se han elaborado varios escenarios, probado y grabado estos de cara a la presentación que ya tiene fecha y será el día 4 de junio de 2026.
- **Bloque DOC.** En cuanto a la documentación, todo el equipo ha continuado con la escritura de este informe para poder tenerlo completo en todos sus aspectos. Además el equipo ha sabido aprovechar ya que el miércoles, que no disponía del vehículo por estarlo compartiendo con los compañeros, se quedó redactando dicho informe final.

C.14. Semana 14 (25/5 – 31/5)

- **Bloque INF.** Durante esta semana hemos continuado preparando y dejado muy atado la demostración que llevaremos a cabo el día 4 de junio a las 09:00h. Para ello, hemos preparado el circuito, probado varias demostraciones y grabado las tomas. Tras haber alcanzado una demostración más genérica donde se incluyeran varias señales: límites de velocidad, giros, stop o prohibido el paso, se ha incluido el *script* definitivo de conducción autónoma en el Anexo D.2. Por otro lado, se ha elaborado un video que podrá mostrarse en caso de cualquier fallo del directo el día de evaluación, este video puede verse en la sección ??.
- **Bloque DOC.** Esta semana se debe realizar la entrega de esta memoria técnico-económica para el proyecto de despliegue de una arquitectura 4G para la detección de señales en Vehículo Autónomo. La entrega está prevista para el viernes 29 de mayo, por lo que, todo el equipo está perfilando el contenido de la memoria. A la par se está preparando la presentación en formato *.ppt* que se proyectará el día de la evaluación. Esta semana se ha consensuado sobre el contenido de dicha presentación, la duración y el reparto de contenido. Para la duración total se nos indicó por parte de los tutores que debía tener un máximo de 20 min.

C.15. Semana 15 (1/6 – 4/6) - final de proyecto

- **Bloque DOC.** Esta última semana, a pesar de que esta memoria ya ha sido entregada, muy seguramente se haya dedicado en su totalidad, y hasta el día 4, a la preparación de la demostración. El equipo, que ya conociéndoles desde el primer día de proyecto estarán nerviosos por la presentación y preocupados por si saldrá la demostración preparada. Viendo todo el esfuerzo dedicado y que queda reflejado en esta memoria, seguramente nos vaya bien.
- Para finalizar este diario, dar las gracias al equipo en su totalidad por su trabajo y dedicación y a los tutores por su apoyo en el proyecto.

C.16. Recuento total de horas trabajadas en el laboratorio

A lo largo de las once semanas de trabajo activo, las horas registradas en Jira por miembro del equipo se resumen como sigue¹:

Tabla C.1: Horas trabajadas registradas en Jira por miembro del equipo en el laboratorio.

Miembro	Horas registradas
Lucía López García	~ 54 h (gestión, planificación, documentación, cobertura)
Miguel Barrigón Gómez	~ 56 h (infraestructura 4G, Jetson)
Rodrigo Seco Dujo	~ 56 h (infraestructura 4G, conexión coche)
Víctor Sánchez Valencia	~ 54 h (IA, entrenamiento, evaluación)
Total registrado	~ 220 h

Como se ha indicado en la sección 3.3, la dedicación estimada del proyecto fue de **192 horas por miembro** incluyendo las horas dedicadas en casa, lo que da un total de **728 horas** y un coste humano aproximado de 10082€.

¹Extracto de horas registradas en las tareas Jira

D Configuración y códigos

En este anexo se recogen los principales ficheros de configuración y *scripts* desarrollados para la demo en laboratorio. Estos ficheros tambien estan disponibles en el repositorio público del equipo: <https://github.com/g2tp2uva-lab/G2TP2>.

D.1. Configuración del DeepRacer (vehicle.conf)

Listing D.1: Configuración del vehículo.

```
[mqtt]
vehicle_id = 3
mqtt_server_ip = 172.16.0.1
mqtt_server_port = 1883

[cloud_autonomous_driving]
cloud_server_ip = 172.16.0.1
cloud_server_port = 20003

[steering_calibration]
max = 1750000
mid = 1370000
min = 1230000
```

Cambio de modo del módem Huawei:

```
sudo usb_modeswitch -v 12d1 -p 1f01 -J
# Verificación de la interfaz de red
lsusb
```

D.2. Script para la conducción autónoma (coche_autonomo.py)

Listing D.2: Script completo para la conducción autonoma.

```
1  """
2  Coche autónomo con seguimiento de línea + reacción a señales
3  =====
4  Incluye 3 opciones de compensación de viraje por rueda defectuosa:
5      - Opción A: Offset fijo
6      - Opción B: Factor proporcional
7      - Opción C: Autocompensación adaptativa
8  """
9
10 import socket
11 import struct
12 import pickle
13 import cv2
14 import time
15 import threading
16 from collections import deque
17 import numpy as np
18 import cmath
19 import math
20
21 import artemis_autonomous_car
22
```

```

23 # CONFIGURACIÓN DE RED

24 SERVER_ADDRESS = ('172.16.0.1', 20003)
25 JETSON_IP      = '192.168.100.102'
26 JETSON_PORT    = 20004
27 RESULTS_PORT   = 20005
28
29 # CONFIGURACIÓN DEL CONTROL AUTÓNOMO

30 PATH           = [2, 2, 2, 2, 2, 2, 2, 2, 2, 2]
31 CONFIDENCE_MIN = 0.70
32 SIGNAL_TIMEOUT = 2.0
33 THROTTLE_DEFAULT = 0.65
34 THROTTLE_LIMITE_30 = 0.60
35 THROTTLE_LIMITE_90 = 0.75
36 THROTTLE_PELIGRO = 0.45
37 STOP_DURATION   = 2.5
38 GIRO_FUERZA      = 0.8
39 GIRO_DURATION    = 1.0
40
41 # COMPENSACIÓN DE VIRAJE (RUEDA DEFECTUOSA)

42 # Activa SOLO UNA de las tres opciones poniéndola a True
43 USAR_OPCION_A = True      # Offset fijo (más simple)
44 USAR_OPCION_B = False     # Factor proporcional
45 USAR_OPCION_C = False     # Autocompensación adaptativa
46
47 # Opción A: Offset constante. Positivo = corrige hacia la izquierda
48 COMPENSACION_OFFSET = 0.23
49
50 # Opción B: Factor proporcional + offset
51 COMPENSACION_FACTOR = 1.15 # Multiplica el giro
52 COMPENSACION_OFFSET_B = 0.10 # Offset adicional
53
54 # Opción C: Autocompensación. Mantiene un histórico y se ajusta solo
55 HISTORICO_TAMANO = 30      # Frames a recordar para promediar
56 COMPENSACION_C_INICIAL = 0.10 # Offset inicial mientras se calibra
57 COMPENSACION_C_MAX = 0.40   # Límite del offset autocalculado
58 LEARNING_RATE = 0.05       # Velocidad de adaptación
59
60 # CONFIGURACIÓN FRENADO PROGRESIVO STOP/PROHIBIDO-PASO
61 IMG_WIDTH = 640
62 IMG_HEIGHT = 480
63 IMG_AREA = IMG_WIDTH * IMG_HEIGHT
64 STOP_AREA_DETENIDO = 1200.0
65 STOP_AREA_INICIO = 500.0
66 THROTTLE_MIN_FRENO = 0.5
67
68
69 # ESTADO GLOBAL

70 ultimo_resultado_jetson = None
71 ultimo_timestamp_jetson = 0
72 ultima_senal_procesada = None
73
74 estado_stop_hasta = 0
75 estado_giro_hasta = 0
76 direccion_giro = 0
77 velocidad_actual = THROTTLE_DEFAULT
78 frenando_por_senal = False
79

```

```

80 # Estado para opción C (autocompensación adaptativa)
81 historico_giros = deque(maxlen=HISTORICO_TAMANO)
82 compensacion_aprendida = COMPENSACION_C_INICIAL
83
84
85 #                SOCKETS UDP
86
87 sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
88 sock.bind(SERVER_ADDRESS)
89 print(f"[CORE] Escuchando en {SERVER_ADDRESS}")
90
91 jetson_send_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
92
93 results_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
94 results_sock.bind(('0.0.0.0', RESULTS_PORT))
95 results_sock.settimeout(0.01)
96
97 #                HILO QUE RECIBE RESULTADOS DE LA JETSON
98
99 def recibir_resultados_jetson():
100     global ultimo_resultado_jetson, ultimo_timestamp_jetson
101     while True:
102         try:
103             data, _ = results_sock.recvfrom(65535)
104             if data[0:1] == b'Y':
105                 resultado = pickle.loads(data[1:])
106                 ultimo_resultado_jetson = resultado
107                 ultimo_timestamp_jetson = time.time()
108         except socket.timeout:
109             pass
110         except Exception as e:
111             print(f"[JETSON] Error: {e}")
112
113 threading.Thread(target=recibir_resultados_jetson, daemon=True).start()
114
115 #                ENVÍO DE CONTROL AL VEHÍCULO
116
117 def send_control(control_giro, control_acelerador, address):
118     sock.sendto(
119         struct.pack('c', bytes('C', 'ascii')) +
120         struct.pack('d', round(control_giro, 3)) +
121         struct.pack('d', round(control_acelerador, 3)),
122         address
123     )
124
125 #                COMPENSACIÓN DE VIRAJE
126
127 def compensar_viraje(giro_original):
128     """
129     Aplica compensación al giro según la opción activa.
130     El coche vira a la derecha, así que sumamos un valor positivo
131     para corregir hacia la izquierda.
132     """
133     global compensacion_aprendida, historico_giros
134
135     if USAR_OPCION_A:
136         # Offset fijo y constante

```

```

136         return giro_original + COMPENSACION_OFFSET
137
138     elif USAR_OPCION_B:
139         # Factor proporcional + offset
140         # Si giro original es 0, suma el offset
141         # Si giro original es negativo (izquierda), lo amplifica
142         # Si giro original es positivo (derecha), lo reduce
143         if giro_original >= 0:
144             return (giro_original / COMPENSACION_FACTOR) + COMPENSACION_OFFSET_B
145         else:
146             return (giro_original * COMPENSACION_FACTOR) + COMPENSACION_OFFSET_B
147
148     elif USAR_OPCION_C:
149         # Autocompensación adaptativa
150         # Si el coche está intentando ir recto (giro original cerca de 0)
151         # y el histórico muestra una tendencia, ajustamos
152         historico_giros.append(giro_original)
153
154         if len(historico_giros) >= HISTORICO_TAMANO:
155             promedio = sum(historico_giros) / len(historico_giros)
156             # Si el promedio es negativo, el coche está corrigiendo a la
157             izquierda
158             # constantemente, lo que significa que necesita más compensación
159             if promedio < -0.05:
160                 compensacion_aprendida += LEARNING_RATE * 0.1
161             elif promedio > 0.05:
162                 compensacion_aprendida -= LEARNING_RATE * 0.1
163
164             # Limitar la compensación a un máximo
165             compensacion_aprendida = max(0, min(COMPENSACION_C_MAX,
166                 compensacion_aprendida))
167
168         return giro_original + compensacion_aprendida
169
170     return giro_original
171
172 # FRENADO PROGRESIVO
173
174 def calcular_velocidad_frenado(area_senal):
175     if area_senal <= STOP_AREA_INICIO:
176         return THROTTLE_DEFAULT
177     if area_senal >= STOP_AREA_DETENIDO:
178         #time.sleep(0.3)
179         return 0.0
180     ratio = (area_senal - STOP_AREA_INICIO) / (STOP_AREA_DETENIDO -
181         STOP_AREA_INICIO)
182     velocidad = THROTTLE_DEFAULT * (1.0 - ratio)
183     velocidad = max(velocidad, THROTTLE_MIN_FRENO) if ratio < 0.9 else max(
184         velocidad, 0.0)
185     return round(velocidad, 3)
186
187 # LÓGICA DE REACCIÓN A SEÑALES
188
189 def aplicar_reaccion_senal(control_giro, control_acelerador):
190     global ultima_senal_procesada
191     global estado_stop_hasta, estado_giro_hasta, direccion_giro
192     global velocidad_actual, frenando_por_senal
193
194     ahora = time.time()

```

```

191
192     if ahora < estado_stop_hasta:
193         frenando_por_senal = False
194         return (COMPENSACION_OFFSET, 0.0)
195
196     # Justo al terminar el stop, restaurar velocidad normal
197     if velocidad_actual == 0.0 and not frenando_por_senal:
198         velocidad_actual = THROTTLE_DEFAULT
199         ultima_senal_procesada = None
200         print(f"[STOP] Reanudando marcha a velocidad {THROTTLE_DEFAULT}")
201
202     if ahora < estado_giro_hasta:
203         frenando_por_senal = False
204         return (direccion_giro * GIRO_FUERZA, velocidad_actual)
205
206     if ultimo_resultado_jetson is None:
207         frenando_por_senal = False
208         return (control_giro, velocidad_actual)
209
210     if ahora - ultimo_timestamp_jetson > SIGNAL_TIMEOUT:
211         frenando_por_senal = False
212         return (control_giro, velocidad_actual)
213
214     cercana = ultimo_resultado_jetson.get('mas_cercana')
215     if cercana is None:
216         frenando_por_senal = False
217         return (control_giro, velocidad_actual)
218
219     if cercana['confidence'] < CONFIDENCE_MIN:
220         frenando_por_senal = False
221         return (control_giro, velocidad_actual)
222
223     clase = cercana['class']
224     confianza = cercana['confidence']
225     area_senal = cercana['width'] * cercana['height']
226
227     if clase in ('STOP', 'PROHIBIDO-PASO'):
228         velocidad_freno = calcular_velocidad_frenado(area_senal)
229         velocidad_actual = velocidad_freno
230         frenando_por_senal = True
231         print(f"[FRENO] {clase} | área={area_senal:.0f}px | velocidad={
                velocidad_freno:.2f}")
232         if velocidad_freno == 0.0:
233             print(f"[PARADO] Detenido ante {clase}")
234             if clase == 'PROHIBIDO-PASO':
235                 estado_stop_hasta = ahora + 999999
236                 return (control_giro, 0.0)
237                 estado_stop_hasta = ahora + STOP_DURATION
238                 return (control_giro, 0.49)
239
240     frenando_por_senal = False
241
242     senal_id = (clase, round(cercana['x']), round(cercana['y']))
243     if senal_id == ultima_senal_procesada:
244         return (control_giro, velocidad_actual)
245
246     ultima_senal_procesada = senal_id
247     print(f"[ACCIÓN] Reaccionando a {clase} ({confianza:.2f}")
248
249     if clase == 'LIMITE-30':
250         velocidad_actual = THROTTLE_LIMITE_30
251         return (control_giro, velocidad_actual)
252     elif clase == 'LIMITE-90':

```

```

253     velocidad_actual = THROTTLE_LIMITE_90
254     return (control_giro, velocidad_actual)
255 elif clase in ('PELIGRO-NINOS', 'PELIGRO-OPERARIO-OBRAS'):
256     velocidad_actual = THROTTLE_PELIGRO
257     return (control_giro, velocidad_actual)
258 elif clase == 'OBLIGATORIO-GIRAR-IZQUIERDA':
259     estado_giro_hasta = ahora + GIRO_DURATION
260     direccion_giro = +1
261     return (direccion_giro * GIRO_FUERZA, velocidad_actual)
262 elif clase == 'OBLIGATORIO-GIRAR-DERECHA':
263     estado_giro_hasta = ahora + GIRO_DURATION
264     direccion_giro = -1
265     return (direccion_giro * GIRO_FUERZA, velocidad_actual)
266 elif clase == 'OBLIGATORIO-RECTO':
267     return (control_giro, velocidad_actual)
268 elif clase in ('CONO', 'OBSTACULO', 'VALLA'):
269     velocidad_actual = THROTTLE_PELIGRO
270     return (control_giro, velocidad_actual)
271
272     return (control_giro, velocidad_actual)
273
274
275 #             INICIALIZAR EL CONTROL AUTÓNOMO
276
277 print("[CORE] Inicializando artemis_autonomous_car...")
278 aac = artemis_autonomous_car.artemis_autonomous_car(PATH)
279 aac.lidar_throttle_control = THROTTLE_DEFAULT
280
281 # Mostrar qué opción de compensación está activa
282 if USAR_OPCION_A:
283     print(f"[COMPENSACIÓN] Opción A: Offset fijo = {COMPENSACION_OFFSET}")
284 elif USAR_OPCION_B:
285     print(f"[COMPENSACIÓN] Opción B: Factor={COMPENSACION_FACTOR}, Offset={COMPENSACION_OFFSET_B}")
286 elif USAR_OPCION_C:
287     print(f"[COMPENSACIÓN] Opción C: Autocompensación adaptativa")
288 else:
289     print("[COMPENSACIÓN] Ninguna opción activa")
290
291 #             BUCLE PRINCIPAL
292
293 print("[CORE] Esperando frames del vehículo...")
294 while True:
295     try:
296         data, address = sock.recvfrom(99999)
297     except KeyboardInterrupt:
298         print("\n[CORE] Cerrando servidor.")
299         break
300
301     data_type = struct.unpack('c', bytes([data[0]]))[0]
302     received_payload = bytes(data[1:])
303     data_decoded = pickle.loads(received_payload, encoding='latin1')
304
305     if data_type == b'I':
306         img = cv2.imdecode(data_decoded, 1)
307         if img is None:
308             continue
309
310     try:
311         jetson_send_sock.sendto(data, (JETSON_IP, JETSON_PORT))

```

```

312     except Exception as e:
313         print(f"[JETSON] Error enviando frame: {e}")
314
315     aac.lidar_throttle_control = velocidad_actual
316     control_giro, control_acelerador, trayectoria_not_found = \
317         aac.proceso_fotograma(img, False, 0)
318
319     if trayectoria_not_found:
320         control_giro = 0.0
321         control_acelerador = velocidad_actual
322
323     giro_final, acelerador_final = aplicar_reaccion_senal(
324         control_giro, control_acelerador)
325
326     # APLICAR COMPENSACIÓN DE VIRAJE
327
328     giro_final = compensar_viraje(giro_final)
329     # Limitar el giro al rango [-1, 1]
330     giro_final = max(-1.0, min(1.0, giro_final))
331
332     send_control(giro_final, acelerador_final, address)
333
334     # MOSTRAR IMAGEN
335
336     display_img = img.copy()
337
338     if ultimo_resultado_jetson is not None and \
339        time.time() - ultimo_timestamp_jetson < SIGNAL_TIMEOUT:
340         cercana = ultimo_resultado_jetson.get('mas_cercana')
341         if cercana and cercana['confidence'] >= CONFIDENCE_MIN:
342             clase = cercana['class']
343             confianza = cercana['confidence']
344             area = cercana['width'] * cercana['height']
345             texto = f"SENAL: {clase} ({confianza:.0%}) area={area:.0f}"
346             overlay = display_img.copy()
347             cv2.rectangle(overlay, (5, 5), (580, 50), (0, 0, 0), -1)
348             cv2.addWeighted(overlay, 0.5, display_img, 0.5, 0, display_img)
349             color = (0, 0, 255) if clase in ('STOP', 'PROHIBIDO-PASO') else
350                 (0, 255, 0)
351             cv2.putText(display_img, texto, (10, 35),
352                 cv2.FONT_HERSHEY_SIMPLEX, 0.8, color, 2)
353         else:
354             cv2.putText(display_img, "Sin senal detectada", (10, 35),
355                 cv2.FONT_HERSHEY_SIMPLEX, 0.7, (200, 200, 200), 1)
356     else:
357         cv2.putText(display_img, "Sin senal detectada", (10, 35),
358             cv2.FONT_HERSHEY_SIMPLEX, 0.7, (200, 200, 200), 1)
359
360     if ultimo_resultado_jetson is not None:
361         todas = ultimo_resultado_jetson.get('detecciones', [])
362         for i, det in enumerate(todas):
363             texto_det = f"{det['class']} {det['confidence']:.0%}"
364             cv2.putText(display_img, texto_det, (10, 70 + i * 25),
365                 cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 200, 255), 1)
366
367     # Mostrar velocidad y compensación
368     cv2.putText(display_img, f"Vel: {velocidad_actual:.2f} | Giro: {
369         giro_final:.2f}",
370         (10, img.shape[0] - 30),
371         cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 2)
372
373     if USAR_OPCION_C:

```

```

369         cv2.putText(display_img, f"Comp.aprendida: {compensacion_aprendida
370                        :.3f}",
371                        (10, img.shape[0] - 10),
372                        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)
373
374     cv2.imshow("Coche ARTEMIS", display_img)
375     cv2.waitKey(1)
376
377     elif data_type == b'L':
378         pass
379     elif data_type == b'D':
380         pass
381
382 cv2.destroyAllWindows()
383 sock.close()
384 jetson_send_sock.close()
385 results_sock.close()

```

D.3. Script de inferencia local con API Roboflow

Listing D.3: Script completo de inferencia local con la API de Roboflow.

```

1  from pathlib import Path
2
3  import cv2
4  import json
5  from inference_sdk import InferenceHTTPClient
6  import supervision as sv
7
8
9  ROOT = Path(__file__).resolve().parent
10 IMAGE_FILE = ROOT / "train" / "fotos27abr" / "2026-04-27-101709.jpg"
11 ANNOTATED_DIR = ROOT / "anotated"
12 ROBOFLOW_API_KEY = "bqpX*****rny"
13 MODEL_ID = "data****0/6"
14 API_URL = "https://serverless.roboflow.com"
15
16
17 def print_detected_classes(results: dict) -> None:
18     predictions = results.get("predictions", [])
19     if not predictions:
20         print("No se detecto ninguna clase.")
21         return
22
23     print("Clases detectadas:")
24     for prediction in predictions:
25         detected_class = prediction.get("class", "desconocida")
26         confidence = prediction.get("confidence")
27         if confidence is None:
28             print(f"- {detected_class}")
29         else:
30             print(f"- {detected_class} ({confidence:.3f})")
31
32
33 def filtrar_deteccion_mas_cercana(detecciones, umbral_confianza=0.85):
34     # Filtrar por confianza mínima
35     filtrados = [d for d in detecciones if d['confidence'] >= umbral_confianza]
36
37     if not filtrados:
38         return None

```

```

39
40     # Encontrar la más "cercana" (la que tenga mayor área: width * height)
41     # Usamos max() con una función lambda que calcula el área
42     mas_cercana = max(filtrados, key=lambda d: d['width'] * d['height'])
43
44     return mas_cercana
45
46
47 def main() -> None:
48     image = cv2.imread(str(IMAGE_FILE))
49     if image is None:
50         raise FileNotFoundError(f"No se pudo cargar la imagen: {IMAGE_FILE}")
51
52     client = InferenceHTTPClient(
53         api_url=API_URL,
54         api_key=ROBOFLOW_API_KEY,
55     )
56
57     results = client.infer(str(IMAGE_FILE), model_id=MODEL_ID)
58     print(results)
59     print_detected_classes(results)
60     detections = sv.Detections.from_inference(results)
61     bounding_box_annotator = sv.BoxAnnotator()
62     label_annotator = sv.LabelAnnotator()
63
64     annotated_image = bounding_box_annotator.annotate(scene=image, detections=
        detections)
65     annotated_image = label_annotator.annotate(scene=annotated_image, detections
        =detections)
66     ANNOTATED_DIR.mkdir(exist_ok=True)
67     output_file = ANNOTATED_DIR / IMAGE_FILE.name
68     cv2.imwrite(str(output_file), annotated_image)
69     print(f"Imagen anotada guardada en: {output_file}")
70
71
72
73     #Obtener la mas cercana
74     predicciones = results.get('predictions', [])
75     cercana = filtrar_deteccion_mas_cercana(predicciones)
76     if cercana:
77         print(f"Detección más cercana: {cercana['class']}")
78         print(f"Confianza: {cercana['confidence']:.2f}")
79         print(f"Dimensiones: {cercana['width']}x{cercana['height']}")
80     else:
81         print("No se encontraron detecciones con confianza suficiente.")
82
83     sv.plot_image(annotated_image)
84
85
86 if __name__ == "__main__":
87     main()

```

D.4. Script de inferencia en la Jetson

Listing D.4: Script completo de inferencia local en la Jetson.

```

1 import socket
2 import struct
3 import pickle
4 import cv2
5 import numpy as np

```

```

6 from ultralytics import YOLO
7 from pathlib import Path
8
9 # --- CONFIGURACIÓN ---
10 CORE_IP = '192.168.100.101' # IP del Core
11 RESULTS_PORT = 20005 # Puerto donde el Core recibe resultados
12 LISTEN_PORT = 20004 # Puerto donde la Jetson escucha frames
13 MODEL_PATH = Path.home() / "weights.pt"
14 CONFIDENCE_THRESHOLD = 0.85
15
16 # --- CARGAR MODELO ---
17 print("Cargando modelo YOLOv11...")
18 model = YOLO(str(MODEL_PATH))
19 print("Modelo listo. Escuchando frames del Core...")
20
21 # --- SOCKETS ---
22 recv_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
23 recv_sock.bind(('0.0.0.0', LISTEN_PORT))
24
25 send_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
26
27 # --- BUCLE PRINCIPAL ---
28 while True:
29     data, addr = recv_sock.recvfrom(99999)
30
31     # Verificar que es un frame de imagen (tipo 'I')
32     data_type = struct.unpack('c', bytes([data[0]]))[0]
33     if data_type != b'I':
34         continue
35
36     # Decodificar imagen (mismo formato que el Core)
37     payload = pickle.loads(bytes(data[1:]), encoding='latin1')
38     buf = np.frombuffer(payload, dtype=np.uint8) if isinstance(payload, bytes)
39     else payload
40     frame = cv2.imdecode(buf, cv2.IMREAD_COLOR)
41
42     if frame is None:
43         continue
44
45     # -- INFERENCIA YOLOV11 EN GPU ---
46     results = model(frame, device=0, conf=0.85, verbose=False)
47
48     # -- EXTRAER DETECCIONES ---
49     detecciones = []
50     names = results[0].names
51     for box in results[0].boxes:
52         x1, y1, x2, y2 = box.xyxy[0].tolist()
53         detecciones.append({
54             'class': names[int(box.cls)],
55             'confidence': float(box.conf),
56             'x': (x1 + x2) / 2,
57             'y': (y1 + y2) / 2,
58             'width': x2 - x1,
59             'height': y2 - y1,
60         })
61
62     # -- FILTRAR Y ENCONTRAR LA MÁS CERCANA ---
63     filtradas = [d for d in detecciones if d['confidence'] >=
64                  CONFIDENCE_THRESHOLD]
65     cercana = max(filtradas, key=lambda d: d['width'] * d['height']) if
66                  filtradas else None
67
68     if cercana:

```

```

66         print(f"Señal: {cercana['class']} ({cercana['confidence']:.2f})")
67
68
69     # -- ENVIAR RESULTADO AL CORE -----
70     resultado = {
71         'detecciones': detecciones,
72         'mas_cercana': cercana
73     }
74     payload_out = b'Y' + pickle.dumps(resultado)
75     #print(f"[DEBUG] Tamaño paquete a enviar: {len(payload_out)} bytes")
76     send_sock.sendto(payload_out, (CORE_IP, RESULTS_PORT))

```

D.5. Script para la obtención de la matriz de confusión

Listing D.5: Script completo empleado para obtener las matrices de confusión de los modelos de IA.

```

1  from ultralytics import YOLO
2  from roboflow import Roboflow
3  import os
4
5  # 1. Descargar el dataset desde Roboflow
6  rf = Roboflow(api_key="bqp*****rny")
7  project = rf.workspace("cocheconectado").project("dat*****0")
8  version = project.version(7)
9  dataset = version.download("yolov11")
10
11 # 2. Cargar tu modelo con los pesos .pt
12 model = YOLO('weights.pt')
13 path_yaml = r"C:\User*****Downloads\Dat****0.v7-20*****on.yolov11\data.yaml"
14
15 # 3. Ejecutar la validación en el set de TEST
16 # El archivo data.yaml descargado de Roboflow ya tiene las rutas configuradas
17 results = model.val(
18     data=path_yaml,
19     split='test',
20     imgsz=640, # usar el mismo tamaño que en el entrenamiento
21     save=True, # Guardar los resultados visuales
22     conf=0.85  # Umbral de confianza
23 )
24
25 print("Validación completada.")
26 print(f"Los resultados, incluyendo la matriz de confusión, están en: {results.
    save_dir}")

```

D.6. Configuración del eNodeB (enb.conf, extracto)

Listing D.6: Parámetros principales del eNodeB.

```

#####
#                               srsENB configuration file
#####

#####
# eNB configuration
#
# enb_id:                20-bit eNB identifier.
# mcc:                   Mobile Country Code

```

```

# mnc:                Mobile Network Code
# mme_addr:           IP address of MME for S1 connection
# gtp_bind_addr:      Local IP address to bind for GTP connection
# gtp_advertise_addr: IP address of eNB to advertise for DL GTP-U Traffic
# s1c_bind_addr:      Local IP address to bind for S1AP connection
# s1c_bind_port:      Source port for S1AP connection (0 means any)
# n_prb:              Number of Physical Resource Blocks (6,15,25,50,75,100)
# tm:                 Transmission mode 1-4 (TM1 default)
# nof_ports:          Number of Tx ports (1 port default, set to 2 for TM2
#                      /3/4)
#
#####
[enb]
enb_id = 0x19B
mcc = 901
mnc = 70
mme_addr = 10.0.128.176
gtp_bind_addr = 10.0.128.181
s1c_bind_addr = 10.0.128.181
s1c_bind_port = 0
n_prb = 50
#tm = 4
#nof_ports = 2

#####
# eNB configuration files
#
# sib_config:  SIB1, SIB2 and SIB3 configuration file
# note: When enabling MBMS, use the sib.conf.mbsfn configuration file which
#        includes SIB13
# rr_config:   Radio Resources configuration file
# rb_config:   SRB/DRB configuration file
#####
[enb_files]
sib_config = sib.conf
rr_config  = rr.conf
rb_config  = rb.conf

#####
# RF configuration
#
# dl_earfcn: EARFCN code for DL (only valid if a single cell is configured in rr
#            .conf)
# tx_gain: Transmit gain (dB).
# rx_gain: Optional receive gain (dB). If disabled, AGC if enabled
#
# Optional parameters:
# dl_freq: Override DL frequency corresponding to dl_earfcn
# ul_freq: Override UL frequency corresponding to dl_earfcn (must be
#          set if dl_freq is set)
# device_name: Device driver family
#              Supported options: "auto" (uses first driver found), "UHD
#              ", "bladeRF", "soapy" or "zmq"
# device_args: Arguments for the device driver. Options are "auto" or any
#              string.
#              Default for UHD: "recv_frame_size=9232,send_frame_size
#              =9232"
#              Default for bladeRF: ""
# time_adv_nsamples: Transmission time advance (in number of samples) to
#                    compensate for RF delay
#                    from antenna to timestamp insertion.
#                    Default "auto". B210 USRP: 100 samples, bladeRF: 27
#####

```

```

[rf]
dl_earfcn = 3300
tx_gain = 80
rx_gain = 40

device_name = bladeRF

# For best performance in 2x2 MIMO and >= 15 MHz use the following device_args
# settings:
#     USRP B210: num_recv_frames=64,num_send_frames=64
#     And for 75 PRBs, also append ",master_clock_rate=15.36e6" to the device
#     args

# For best performance when BW<5 MHz (25 PRB), use the following device_args
# settings:
#     USRP B210: send_frame_size=512,recv_frame_size=512

#device_args = auto
#time_adv_nsamples = auto

# Example for ZMQ-based operation with TCP transport for I/Q samples
#device_name = zmq
#device_args = fail_on_disconnect=true,tx_port=tcp://*:2000,rx_port=tcp://
#     localhost:2001,id=enb,base_srate=23.04e6

#####
# Packet capture configuration
#
# MAC-layer packets are captured to a file in the compact format which can
# be decoded by Wireshark. For decoding, use the UDP dissector and the UDP
# heuristic dissection. Edit the preferences (Edit > Preferences >
# Protocols > DLT_USER) for DLT_USER to add an entry for DLT=149 with
# Protocol=udp. Further, enable the heuristic dissection in UDP under:
# Analyze > Enabled Protocols > MAC-LTE > mac_lte_udp and MAC-NR > mac_nr_udp
# For more information see: https://wiki.wireshark.org/MAC-LTE
# Configuring this Wireshark preferences is needed for decoding the MAC PCAP
# files as well as for the live network capture option.
#
# Please note that this setting will by default only capture MAC
# frames on dedicated channels, and not SIB. You have to build with
# WRITE_SIB_PCAP enabled in srsenb/src/stack/mac/mac.cc if you want
# SIB to be part of the MAC pcap file.
#
# SIAP Packets are captured to a file in the compact format which can
# be decoded by the Wireshark siap dissector with DLT 150.
# To use the dissector, edit the preferences for DLT_USER to
# add an entry with DLT=150, Payload Protocol=siap.
#
# enable:          Enable MAC layer packet captures (true/false)
# filename:        File path to use for LTE MAC packet captures
# nr_filename:     File path to use for NR MAC packet captures
# siap_enable:     Enable or disable the PCAP.
# siap_filename:   File name where to save the PCAP.
#
# mac_net_enable:  Enable MAC layer packet captures sent over the network (true/
#     false default: false)
# bind_ip:         Bind IP address for MAC network trace (default: "0.0.0.0")
# bind_port:       Bind port for MAC network trace (default: 5687)
# client_ip:       Client IP address for MAC network trace (default: "127.0.0.1")
# client_port      Client IP address for MAC network trace (default: 5847)
#####
[pcap]
#enable = false

```

```

#filename = /tmp/enb_mac.pcap
#nr_filename = /tmp/enb_mac_nr.pcap
#slap_enable = false
#slap_filename = /tmp/enb_slap.pcap

#mac_net_enable = false
#bind_ip = 0.0.0.0
#bind_port = 5687
#client_ip = 127.0.0.1
#client_port = 5847

#####
# Log configuration
#
# Log levels can be set for individual layers. "all_level" sets log
# level for all layers unless otherwise configured.
# Format: e.g. phy_level = info
#
# In the same way, packet hex dumps can be limited for each level.
# "all_hex_limit" sets the hex limit for all layers unless otherwise
# configured.
# Format: e.g. phy_hex_limit = 32
#
# Logging layers: rf, phy, phy_lib, mac, rlc, pdcp, rrc, gtpu, slap, stack, all
# Logging levels: debug, info, warning, error, none
#
# filename: File path to use for log output. Can be set to stdout
#           to print logs to standard output
# file_max_size: Maximum file size (in kilobytes). When passed, multiple files
#               are created.
#               If set to negative, a single log file will be created.
#####
[log]
all_level = info
all_hex_limit = 32
filename = /tmp/enb.log
file_max_size = -1

[gui]
enable = false

#####
# Scheduler configuration options
#
# sched_policy:      User MAC scheduling policy (E.g. time_rr, time_pf)
# min_aggr_level:    Optional minimum aggregation level index (l=log2(L) can be
#                   0, 1, 2 or 3)
# max_aggr_level:    Optional maximum aggregation level index (l=log2(L) can be
#                   0, 1, 2 or 3)
# adaptive_aggr_level: Boolean flag to enable/disable adaptive aggregation level
#                   based on target BLER
# pdsch_mcs:         Optional fixed PDSCH MCS (ignores reported CQIs if
#                   specified)
# pdsch_max_mcs:     Optional PDSCH MCS limit
# pusch_mcs:         Optional fixed PUSCH MCS (ignores reported CQIs if
#                   specified)
# pussh_max_mcs:     Optional PUSCH MCS limit
# min_nof_ctrl_symbols: Minimum number of control symbols
# max_nof_ctrl_symbols: Maximum number of control symbols
# pucch_multiplex_enable: Allow PUCCH HARQ to collide with PUSCH and other PUCCH
# pucch_harq_max_rb: Maximum number of RB to be used for PUCCH on the edges of
#                   the grid.

```

```

#           If defined and greater than 0, the scheduler will avoid DL
#           PDCCH allocations if
#           PUCCH HARQ falls outside this region
# target_bler:      Target BLER (in decimal) to achieve via adaptive link
# max_delta_dl_cqi: Maximum shift in CQI for adaptive DL link
# max_delta_ul_snr: Maximum shift in UL SNR for adaptive UL link
# adaptive_dl_mcs_step_size: Step size or learning rate used in adaptive DL MCS
#                       link
# adaptive_ul_mcs_step_size: Step size or learning rate used in adaptive UL MCS
#                       link
# min_tpc_tti_interval: Minimum TTI interval between TPCs different than 1
# ul_snr_avg_alpha: Exponential Average alpha coefficient used in estimation of
#                       UL SNR
# init_ul_snr_value: Initial UL SNR value used for computing MCS in the first UL
#                       grant
# init_dl_cqi:      DL CQI value used before any CQI report is available to the
#                       eNB
# max_sib_coderate: Upper bound on SIB and RAR grants coderate
# pdcch_cqi_offset: CQI offset in derivation of PDCCH aggregation level
# nr_pdsch_mcs:     Optional fixed NR PDSCH MCS (ignores reported CQIs if
#                       specified)
# nr_pusch_mcs:     Optional fixed NR PUSCH MCS (ignores reported CQIs if
#                       specified)
#
#####
[scheduler]
#policy      = time_pf
#policy_args = 2
#min_aggr_level  = 0
#max_aggr_level  = 3
#adaptive_aggr_level = false
#pdsch_mcs      = -1
#pdsch_max_mcs  = -1
#pusch_mcs      = -1
#pusch_max_mcs  = 16
#min_nof_ctrl_symbols = 1
#max_nof_ctrl_symbols = 3
#pucch_multiplex_enable = false
#pucch_harq_max_rb = 0
#target_bler = 0.05
#max_delta_dl_cqi = 5
#max_delta_ul_snr = 5
#adaptive_dl_mcs_step_size = 0.001
#adaptive_ul_mcs_step_size = 0.001
#min_tpc_tti_interval = 1
#ul_snr_avg_alpha=0.05
#init_ul_snr_value=5
#init_dl_cqi=5
#max_sib_coderate=0.3
#pdcch_cqi_offset=0
#nr_pdsch_mcs=28
#nr_pusch_mcs=28

#####
# Slicing configuration
# enable_eMBB      Enables enhanced mobile broadband (eMBB) slice in the
#                       gNodeB
# enable_URLLC      Enables Ultra Reliable Low Latency Communications (URLLC
#                       ) slice in the gNodeB
# enable_MIoT       Enables Massive Internet of Things (MIoT) slice in the
#                       gNodeB
# eMBB_sd           eMBB slice differentiator
# URLLC_sd          URLLC slice differentiator

```

```

# MIoT_sd                MIoT slice differentiator
#####
[slicing]
#enable_eMBB = false
#enable_URLLC = false
#enable_MIoT = false
#eMBB_sd = 1
#URLLC_sd = 1
#MIoT_sd = 1

#####
# eMBMS configuration options
#
# enable:                Enable MBMS transmission in the eNB
# m1u_multiaddr:         Multicast address the M1-U socket will register to
# m1u_if_addr:           Address of the interface the M1-U interface will listen
#                        to for multicast packets
# mcs:                   Modulation and Coding scheme for MBMS traffic
#
#####
[embms]
#enable = false
#m1u_multiaddr = 239.255.0.1
#m1u_if_addr = 127.0.1.201
#mcs = 20

#####
# Channel emulator options:
# enable:                Enable/disable internal Downlink/Uplink channel emulator
#
# -- AWGN Generator
# awgn.enable:           Enable/disable AWGN generator
# awgn.snr:              Target SNR in dB
#
# -- Fading emulator
# fading.enable:         Enable/disable fading simulator
# fading.model:          Fading model + maximum doppler (E.g. none, epa5, eva70,
#                        etu300, etc)
#
# -- Delay Emulator      delay(t) = delay_min + (delay_max - delay_min) * (1 +
#                        sin(2pi*t/period)) / 2
#                        Maximum speed [m/s]: (delay_max - delay_min) * pi * 300
#                        / period
# delay.enable:          Enable/disable delay simulator
# delay.period_s:        Delay period in seconds
# delay.init_time_s:     Delay initial time in seconds
# delay.maximum_us:      Maximum delay in microseconds
# delay.minumum_us:      Minimum delay in microseconds
#
# -- Radio-Link Failure (RLF) Emulator
# rlf.enable:            Enable/disable RLF simulator
# rlf.t_on_ms:           Time for On state of the channel (ms)
# rlf.t_off_ms:          Time for Off state of the channel (ms)
#
# -- High Speed Train Doppler model simulator
# hst.enable:            Enable/disable HST simulator
# hst.period_s:          HST simulation period in seconds
# hst.fd_hz:             Doppler frequency in Hz
# hst.init_time_s:       Initial time in seconds
#####
[channel.dl]

```

```

#enable          = false

[channel.dl.awgn]
#enable          = false
#snr              = 30

[channel.dl.fading]
#enable          = false
#model           = none

[channel.dl.delay]
#enable          = false
#period_s        = 3600
#init_time_s     = 0
#maximum_us      = 100
#minimum_us      = 10

[channel.dl.rlf]
#enable          = false
#t_on_ms         = 10000
#t_off_ms        = 2000

[channel.dl.hst]
#enable          = false
#period_s        = 7.2
#fd_hz           = 750.0
#init_time_s     = 0.0

[channel.ul]
#enable          = false

[channel.ul.awgn]
#enable          = false
#n0              = -30

[channel.ul.fading]
#enable          = false
#model           = none

[channel.ul.delay]
#enable          = false
#period_s        = 3600
#init_time_s     = 0
#maximum_us      = 100
#minimum_us      = 10

[channel.ul.rlf]
#enable          = false
#t_on_ms         = 10000
#t_off_ms        = 2000

[channel.ul.hst]
#enable          = false
#period_s        = 7.2
#fd_hz           = -750.0
#init_time_s     = 0.0

#####
# CFR configuration options
#
# The CFR module provides crest factor reduction for the transmitted signal.
#
# enable:          Enable or disable the CFR. Default: disabled

```

```

#
# mode:                manual:    CFR threshold is set by cfr_manual_thres (default)
#
#                auto_ema: CFR threshold is adaptive based on the signal PAPR
#                . Power avg. with Exponential Moving Average.
#                The time constant of the averaging can be tweaked
#                with the ema_alpha parameter.
#                auto_cma: CFR threshold is adaptive based on the signal PAPR
#                . Power avg. with Cumulative Moving Average.
#                Use with care, as CMA's increasingly slow response
#                may be unsuitable for most use cases.
#
# strength:            Ratio between amplitude-limited vs unprocessed signal (0 to
#                1). Default: 1
# manual_thres:        Fixed manual clipping threshold for CFR manual mode. Default
#                : 0.5
# auto_target_papr:    Signal PAPR target (in dB) in CFR auto modes. output PAPR
#                can be higher due to peak smoothing. Default: 8
# ema_alpha:           Alpha coefficient for the power average in auto_ema mode.
#                Default: 1/7
#
#####
[cfr]
#enable                = false
#mode                  = manual
#manual_thres          = 0.5
#strength              = 1
#auto_target_papr      = 8
#ema_alpha             = 0.0143

# E2 Agent configuration options
#
# ric_ip:              IP address of the RIC controller
# ric_port:            Port of the RIC controller
# ric_bind_ip:         Local IP address to bind for RIC connection
# ric_bind_port:       Local port to bind for RIC connection
# max_ric_setup_retries: Maximum amount of retries to setup the RIC connection.
#                If this value is exceeded, an alarm is written to the log. -1 means infinity
#
# ric_connect_timer:   Connection Retry Timer for RIC connection (seconds)
#####
[e2_agent]
#enable = false
#ric_ip = 127.0.0.1
#ric_port = 36421
#ric_bind_ip = 127.0.0.1
#ric_bind_port = 36425
#max_ric_setup_retries = -1
#ric_connect_timer = 10

#####
# Expert configuration options
#
# pusch_max_its:       Maximum number of turbo decoder iterations (default: 4)
# nr_pusch_max_its:    Maximum number of LDPC iterations for NR (Default 10)
# pusch_8bit_decoder:  Use 8-bit for LLR representation and turbo decoder
#                trellis computation (experimental)
# nof_phy_threads:     Selects the number of PHY threads (maximum: 4, minimum:
#                1, default: 3)
# metrics_period_secs: Sets the period at which metrics are requested from the
#                eNB
# metrics_csv_enable:  Write eNB metrics to CSV file.
# metrics_csv_filename: File path to use for CSV metrics

```

```

# report_json_enable:      Write eNB report to JSON file (default: disabled)
# report_json_filename:    Report JSON filename (default: /tmp/enb_report.json)
# report_json_asn1_oct:    Prints ASN1 messages encoded as an octet string instead
                           of plain text in the JSON report file
# alarms_log_enable:       Enable Alarms logging (default: disabled)
# alarms_filename:         Alarms logging filename (default: /tmp/alarms.log)
# tracing_enable:          Write source code tracing information to a file
# tracing_filename:        File path to use for tracing information
# tracing_buffcapacity:    Maximum capacity in bytes the tracing framework can
                           store
# stdout_ts_enable:        Prints once per second the timestamp into stdout
# tx_amplitude:            Transmit amplitude factor (set 0-1 to reduce PAPR)
# rrc_inactivity_timer     Inactivity timeout used to remove UE context from RRC (
                           in milliseconds)
# max_mac_dl_kos:          Maximum number of consecutive KOs in DL before
                           triggering the UE's release (default: 100)
# max_mac_ul_kos:          Maximum number of consecutive KOs in UL before
                           triggering the UE's release (default: 100)
# max_prach_offset_us:     Maximum allowed RACH offset (in us)
# nof_prealloc_ues:        Number of UE memory resources to preallocate during eNB
                           initialization for faster UE creation (default: 8)
# rlf_release_timer_ms:    Time taken by eNB to release UE context after it detects
                           an RLF
# eea_pref_list:           Ordered preference list for the selection of encryption
                           algorithm (EEA) (default: EEA0, EEA2, EEA1)
# eia_pref_list:           Ordered preference list for the selection of integrity
                           algorithm (EIA) (default: EIA2, EIA1, EIA0)
# gtpu_tunnel_timeout:     Time that GTPU takes to release indirect forwarding
                           tunnel since the last received GTPU PDU (0 for no timer)
# ts1_reloc_prep_timeout:  S1AP TS 36.413 TS1RelocPrep Expiry Timeout value in
                           milliseconds
# ts1_reloc_overall_timeout: S1AP TS 36.413 TS1RelocOverall Expiry Timeout value
                           in milliseconds
# rlf_release_timer_ms:    Time taken by eNB to release UE context after it detects
                           a RLF
# rlf_min_ul_snr_estim:    SNR threshold in dB below which the enb is notified with
                           RLF ko
# s1_setup_max_retries:    Maximum amount of retries to setup the S1AP connection.
                           If this value is exceeded, an alarm is written to the log. -1 means infinity
#
# s1_connect_timer:        Connection Retry Timer for S1 connection (seconds)
# rx_gain_offset:          RX Gain offset to add to rx_gain to calibrate RSRP
                           readings
# use_cedron_f_est_alg:    Whether to use Cedron algorithm for TA estimation or not
                           (Default: false)
#####
[expert]
#pusch_max_its             = 8 # These are half iterations
#nr_pusch_max_its          = 10
#pusch_8bit_decoder        = false
#nof_phy_threads           = 3
#metrics_period_secs       = 1
#metrics_csv_enable        = false
#metrics_csv_filename      = /tmp/enb_metrics.csv
#report_json_enable        = true
#report_json_filename      = /tmp/enb_report.json
#report_json_asn1_oct      = false
#alarms_log_enable         = true
#alarms_filename           = /tmp/enb_alarms.log
#tracing_enable            = true
#tracing_filename          = /tmp/enb_tracing.log
#tracing_buffcapacity      = 1000000
#stdout_ts_enable          = false

```

```

#tx_amplitude           = 0.6
#rrc_inactivity_timer   = 30000
#max_mac_dl_kos         = 100
#max_mac_ul_kos         = 100
#max_prach_offset_us    = 30
#nof_prealloc_ues       = 8
#rlf_release_timer_ms   = 4000
#lcid_padding           = 3
#eea_pref_list = EEA0, EEA2, EEA1
#eia_pref_list = EIA2, EIA1, EIA0
#gtpu_tunnel_timeout    = 0
#extended_cp            = false
#ts1_reloc_prep_timeout = 10000
#ts1_reloc_overall_timeout = 10000
#rlf_release_timer_ms   = 4000
#rlf_min_ul_snr_estim   = -2
#s1_setup_max_retries   = -1
#s1_connect_timer       = 10
#rx_gain_offset         = 62
#mac_prach_bi           = 0
#use_cedron_f_est_alg   = false

```

D.7. Configuración del EPC (epc.conf)

Listing D.7: Parámetros principales del EPC.

```

#####
#                               srsEPC configuration file
#####

#####
# MME configuration
#
# mme_code:           8-bit MME code identifies the MME within a group.
# mme_group:          16-bit MME group identifier.
# tac:                16-bit Tracking Area Code.
# mcc:                Mobile Country Code
# mnc:                Mobile Network Code
# full_net_name       Display Name of the Network
# short_net_name      Short Display Name of the Network
# apn:                Set Access Point Name (APN)
# mme_bind_addr:      IP bind addr to listen for eNB S1-MME connections
# dns_addr:           DNS server address for the UEs
# encryption_algo:    Preferred encryption algorithm for NAS layer
#                     (supported: EEA0 (default), EEA1, EEA2, EEA3)
# integrity_algo:     Preferred integrity protection algorithm for NAS
#                     (supported: EIA0 (rejected by most UEs), EIA1 (default),
#                     EIA2, EIA3)
# paging_timer:       Value of paging timer in seconds (T3413)
# request_imeisv:     Request UE's IMEI-SV in security mode command
# lac:                16-bit Location Area Code.
#
#####
[mme]
mme_code = 0x1a
mme_group = 0x0001
tac = 0x0007
mcc = 901
mnc = 70
mme_bind_addr = 10.0.128.176
apn = srsapn

```

```

dns_addr = 8.8.8.8
encryption_algo = EEA0
integrity_algo = EIA1
paging_timer = 2
request_imeisv = false
lac = 0x0006

#####
# HSS configuration
#
# db_file:          Location of .csv file that stores UEs information.
#
#####
[hss]
db_file = user_db.csv

#####
# SP-GW configuration
#
# gtpu_bind_addr:   GTP-U bind address.
# sgi_if_addr:      SGi TUN interface IP address.
# sgi_if_name:      SGi TUN interface name.
# max_paging_queue: Maximum packets in paging queue (per UE).
#
#####

[spgw]
gtpu_bind_addr   = 10.0.128.176
sgi_if_addr      = 172.16.0.1
sgi_if_name      = srs_spgw_sgi
max_paging_queue = 100

#####
# PCAP configuration
#
# Packets are captured to file in the compact format decoded by
# the Wireshark slap dissector and with DLT 150.
# To use the dissector, edit the preferences for DLT_USER to
# add an entry with DLT=150, Payload Protocol=slap.
#
# enable:           Enable or disable the PCAP.
# filename:         File name where to save the PCAP.
#
#####
[pcap]
enable          = false
filename        = /tmp/epc.pcap

#####
# Log configuration
#
# Log levels can be set for individual layers. "all_level" sets log
# level for all layers unless otherwise configured.
# Format: e.g. slap_level = info
#
# In the same way, packet hex dumps can be limited for each level.
# "all_hex_limit" sets the hex limit for all layers unless otherwise
# configured.
# Format: e.g. slap_hex_limit = 32
#
# Logging layers: nas, slap, mme_gtpc, spgw_gtpc, gtpu, spgw, hss, all
# Logging levels: debug, info, warning, error, none
#

```

```
# filename: File path to use for log output. Can be set to stdout
#           to print logs to standard output
#####
[log]
all_level = info
all_hex_limit = 32
filename = /tmp/epc.log

#nas_level = debug
#s1ap_level = debug
#mme_gtpc_level = debug
#spgw_gtpc_level = debug
#gtpu_level = debug
#spgw_level = debug
#hss_level = debug
```

D.8. Base de datos de usuarios (user_db.csv)

Listing D.8: Fichero del HSS con la SIM del DeepRacer.

```
#
# .csv to store UE's information in HSS
# Kept in the following format: "Name,Auth,IMSI,Key,OP_Type,OP/OPc,AMF,SQN,QCI,
#   IP_alloc"
#
# Name:      Human readable name to help distinguish UE's. Ignored by the HSS
# Auth:      Authentication algorithm used by the UE. Valid algorithms are XOR
#             (xor) and MILENAGE (mil)
# IMSI:      UE's IMSI value
# Key:       UE's key, where other keys are derived from. Stored in hexadecimal
# OP_Type:   Operator's code type, either OP or OPc
# OP/OPc:    Operator Code/Cyphered Operator Code, stored in hexadecimal
# AMF:       Authentication management field, stored in hexadecimal
# SQN:       UE's Sequence number for freshness of the authentication
# QCI:       QoS Class Identifier for the UE's default bearer.
# IP_alloc:  IP allocation stratagy for the SPGW.
#            With 'dynamic' the SPGW will automatically allocate IPs
#            With a valid IPv4 (e.g. '172.16.0.2') the UE will have a statically
#            assigned IP.
#
# Note: Lines starting by '#' are ignored and will be overwritten
ue2,mil,001010123456780,00112233445566778899aabbccddeeff,opc,63
    bfa50ee6523365ff14c1f45f88737d,8000,000000001234,7,dynamic
ue1,xor,001010123456789,00112233445566778899aabbccddeeff,opc,63
    bfa50ee6523365ff14c1f45f88737d,9001,000000001234,7,dynamic
sim1,mil,901700000052128,7c9e0122bd85c05077fe5535c82867bf,opc,
    acce99ce1b0f50eba058a9145f3c6a4d,9001,000000003406,7,dynamic
```

E Guía de instalación y configuración de la red 4G privada

Este anexo documenta, paso a paso, el procedimiento completo de despliegue de la red LTE privada utilizada en la demo del laboratorio. La guía cubre desde la preparación del entorno Docker hasta la puesta en marcha del vehículo DeepRacer sobre la red, y está pensada para ser reproducible por cualquier persona con acceso al mismo *hardware*.

Tabla E.1: Resumen de parámetros de la red 4G privada del laboratorio.

Parámetro	Valor
MCC / MNC	901 / 70
EARFCN	3350
Banda LTE	Banda 7 (FDD)
Frecuencia DL	2680 MHz
Frecuencia UL	2560 MHz
Ancho de banda	10 MHz (50 PRB)
TAC	7
Subred UE	172.16.0.0/24
IP <i>gateway</i> SGI	172.16.0.1
IP <i>host</i> del <i>core</i>	10.0.128.176
IP <i>host</i> del eNodeB	10.0.128.181
IP Jetson Orin Nano	192.168.100.102

E.1. Instalación de Docker Engine nativo

El primer paso consiste en sustituir Docker Desktop por Docker Engine nativo. Docker Desktop ejecuta los contenedores dentro de una máquina virtual con kernel `linuxkit`, lo que impide el acceso USB directo al SDR BladeRF.

E.1.1. Desinstalación de Docker Desktop

```
sudo apt-get remove -y docker-desktop
rm -rf ~/.docker/desktop
```

E.1.2. Limpieza de repositorios conflictivos

```
sudo rm -f /etc/apt/keyrings/docker.gpg
sudo rm -f /etc/apt/keyrings/docker.asc
sudo rm -f /etc/apt/sources.list.d/docker.list
sudo rm -f /etc/apt/sources.list.d/docker.sources
```

E.1.3. Instalación de Docker Engine

```
sudo apt-get update && sudo apt-get install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture) \
signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu jammy stable" \
| sudo tee /etc/apt/sources.list.d/docker.list

sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io
sudo systemctl start docker && sudo systemctl enable docker
sudo usermod -aG docker $USER
sudo chmod 666 /var/run/docker.sock
```

Para verificar que el kernel es el nativo del *host* (no debe aparecer linuxkit):

```
docker info | grep 'Kernel Version'
```

E.2. Construcción de la imagen con soporte BladeRF

Se crea un contenedor Docker sobre Ubuntu 22.04 y se compila la librería BladeRF desde el código fuente para poder comunicarse con el dispositivo SDR.

E.2.1. Creación del contenedor base

```
docker run --privileged -v /dev:/dev --name 4Genode -it ubuntu:22.04 bash
```

E.2.2. Instalación de dependencias y clonación del repositorio

Dentro del contenedor:

```
apt-get update && apt-get install -y \
cmake build-essential pkg-config git \
libusb-1.0-0-dev libtecla-dev libncurses-dev \
libcurl4-openssl-dev libxml2-dev

cd / && git clone https://github.com/Nuand/bladeRF.git
```

E.2.3. Compilación e instalación

```
cd /bladeRF && mkdir -p host/build && cd host/build
cmake -DCMAKE_BUILD_TYPE=Release \
-DMAKE_INSTALL_PREFIX=/usr/local \
-DINSTALL_UDEV_RULES=OFF ../
make -j$(nproc) && make install && ldconfig
```

La opción `INSTALL_UDEV_RULES=OFF` es necesaria porque dentro de un contenedor Docker no existe demonio udev.

E.2.4. Verificación

```
bladeRF-cli --version
bladeRF-cli -p
```

E.2.5. Carga de la FPGA (*bitstream* v0.15.3)

El BladeRF 2.0 micro xA9 requiere cargar un *bitstream* de FPGA en cada sesión. Se carga en RAM (opción -l) para no modificar nada permanente en el dispositivo:

```
wget https://www.nuand.com/fpga/hostedxA9-latest.rbf \
-O /tmp/hostedxA9.rbf
bladeRF-cli -l /tmp/hostedxA9.rbf
```

Verificación dentro del CLI interactivo:

```
bladeRF> version
bladeRF> info
```

E.2.6. Persistencia de la imagen Docker

```
docker commit 4Genode bladerf-listo
docker save bladerf-listo | gzip > ~/bladerf-listo.tar.gz
```

E.3. Instalación de srsRAN 4G

srsRAN 4G es el *software* que implementa la pila de protocolos LTE. Se instala en ambos contenedores: el del *core* (que ejecutará **srsepc**) y el del eNodeB (que ejecutará **srsenb**).

E.3.1. Dependencias

```
apt-get update && apt-get install -y \
cmake build-essential git libfftw3-dev libmbedtls-dev \
libboost-program-options-dev libconfig++-dev libsctp-dev \
libuhd-dev libzmq3-dev python3-dev python3-pip
```

E.3.2. Clonación y compilación con soporte BladeRF

```
cd / && git clone https://github.com/srsRAN/srsRAN_4G.git
cd srsRAN_4G && mkdir build && cd build
cmake ../ -DUSE_BLADERF=ON
make -j$(nproc) && make install && ldconfig
```

La bandera -DUSE_BLADERF=ON es imprescindible: sin ella, el binario **srsenb** no podría instanciar el frontal radio del BladeRF.

E.3.3. Verificación

```
srsenb --version
srsue --version
srsepc --version
```

La salida debe mostrar el *plugin* BladeRF activo:

```
Active RF plugins: libsrslan_rf_uhd.so libsrslan_rf_blade.so libsrslan_rf_zmq.so
```

E.3.4. Generación de los ficheros de configuración

```
srsran_install_configs.sh user
ls ~/.config/srsran/
```

E.4. Configuración de la red 4G

E.4.1. Configuración del EPC (*core*)

Fichero: ~/.config/srsran/epc.conf

```
sed -i 's/mcc = 001/mcc = 901/' ~/.config/srsran/epc.conf
sed -i 's/mnc = 01/mnc = 70/' ~/.config/srsran/epc.conf
sed -i 's/mme_bind_addr = 127.0.1.100/mme_bind_addr = 10.0.128.176/' \
~/.config/srsran/epc.conf
sed -i 's/gtpu_bind_addr = 127.0.1.100/gtpu_bind_addr = 10.0.128.176/' \
~/.config/srsran/epc.conf
```

Los valores MCC 901 y MNC 70 corresponden a un PLMN de pruebas no asignado comercialmente. Deben coincidir obligatoriamente con los tres primeros dígitos del IMSI de la SIM del vehículo.

E.4.2. Registro de la SIM en el HSS

Fichero: ~/.config/srsran/user_db.csv

```
echo 'sim1,mil,9017000000052128,\
7C9E0122BD85C05077FE5535C82867BF,opc,\
ACCE99CE1B0F50EBA058A9145F3C6A4D,\
9001,000000000000,7,dynamic' \
>> ~/.config/srsran/user_db.csv
```

Se emplea autenticación MILENAGE con la clave Ki y el OPc específicos de la SIM programable.

E.4.3. Configuración del eNodeB

Fichero: ~/.config/srsran/enb.conf

```
sed -i 's/mme_addr = 127.0.1.100/mme_addr = 10.0.128.176/' \
~/.config/srsran/enb.conf
sed -i 's/gtp_bind_addr = 127.0.1.1/gtp_bind_addr = 10.0.128.181/' \
~/.config/srsran/enb.conf
sed -i 's/s1c_bind_addr = 127.0.1.1/s1c_bind_addr = 10.0.128.181/' \
~/.config/srsran/enb.conf
sed -i 's/#dl_earfcn = 3350/dl_earfcn = 3350/' \
~/.config/srsran/enb.conf
sed -i 's/#device_name = auto/device_name = bladeRF/' \
~/.config/srsran/enb.conf
```

E.4.4. Configuración del *scheduler* para estabilidad de vídeo

En la sección [scheduler] de enb.conf se ajustan los siguientes parámetros para priorizar la estabilidad del flujo de vídeo sobre la maximización del *throughput*:

Tabla E.2: Parámetros del *scheduler* del eNodeB.

Parámetro	Valor	Justificación
policy	time_pf	<i>Proportional fair</i> temporal, equilibra equidad y rendimiento.
policy_args	2	Ventana de promediado de 2 subtramas.
pusch_mcs	10	MCS inicial conservador (16-QAM baja).
pusch_max_mcs	16	Límite superior del MCS en <i>uplink</i> .
target_bler	0.05	BLER objetivo del 5 %, más estricto que el 10 % por defecto.
max_delta_ul_snr	3	Margen de SNR de 3 dB para el ajuste de potencia.

E.4.5. Configuración NAT para salida a Internet

En la *host* del *core* (fuera del contenedor):

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo iptables -t nat -A POSTROUTING -s 172.16.0.0/24 \
-o eno1 -j MASQUERADE
sudo iptables -A FORWARD -s 172.16.0.0/24 -j ACCEPT
sudo iptables -A FORWARD -d 172.16.0.0/24 -j ACCEPT
```

Sin estas reglas, los UE tendrían conectividad al *gateway* 172.16.0.1 pero no podrían alcanzar Internet, dado que la subred 172.16.0.0/24 no es enrutable públicamente.

E.4.6. Arquitectura de red resultante

La red del laboratorio queda organizada en tres segmentos:

- **Core** (10.0.128.176): ejecuta *srsepc* y el *broker* Mosquitto MQTT dentro del contenedor 4Gcore con `--network host`.
- **eNodeB** (10.0.128.181): ejecuta *srsenb* con BladeRF 2.0 micro dentro del contenedor 4Genode con `--network host`.
- **Subred UE** (172.16.0.0/24): *gateway* en 172.16.0.1 (interfaz virtual *srs_spgw_sgi* creada por *srsepc*).
- **Jetson Orin Nano** (192.168.100.102): conectada al *host* del *core* por cable Ethernet a través de la interfaz *enp1s0* (192.168.100.101).

E.5. Instalación y configuración de Mosquitto MQTT

Se instala Mosquitto como *broker* MQTT dentro del contenedor del *core* para gestionar la comunicación con los vehículos.

E.5.1. Instalación

```
apt-get update && apt-get install -y mosquitto mosquitto-clients
```

E.5.2. Configuración

Fichero: `/etc/mosquitto/conf.d/default.conf`

```
listener 1883 0.0.0.0
allow_anonymous true
```

La escucha en 0.0.0.0 es necesaria porque el vehículo accede al *broker* a través de la red LTE y este debe estar accesible en la IP 172.16.0.1. Se permite acceso anónimo por simplicidad en el entorno de laboratorio; en un despliegue productivo esta decisión debería revisarse.

E.5.3. Arranque del servicio

```
mkdir -p /run/mosquitto && chown mosquitto:mosquitto /run/mosquitto
cd /etc/mosquitto && mosquitto -c mosquitto.conf -d
```

E.5.4. Comandos de control del vehículo por MQTT

Desde el *core* se pueden enviar los siguientes comandos al vehículo (con `vehicle_id=3`):

Tabla E.3: Comandos MQTT del protocolo ARTEMIS.

Comando	Descripción
AM-Cloud	Activa el control remoto por <i>cloud</i> (indicador LED azul).
AM-Local	Activa el control autónomo local (indicador LED verde).
AM-Off	Detiene el vehículo (indicador LED amarillo).
Get-Data	Inicia el envío de telemetría IMU y batería.
Stop-Data	Detiene el envío de telemetría.

Ejemplo de envío:

```
mosquitto_pub -h localhost -t 3/command -m 'AM-Cloud'
```

E.6. Configuración del vehículo DeepRacer

El vehículo AWS DeepRacer incorpora un módem USB Huawei E3372h-320 y el *software* ARTEMIS para la comunicación con el *core*.

E.6.1. Activación del módem USB

El módem Huawei arranca por defecto en modo almacenamiento masivo (*mass storage*). Para conmutarlo a modo módem se ejecuta:

```
sudo usb_modeswitch -v 12d1 -p 1f01 -J
sudo systemctl restart NetworkManager
```

Para automatizar el cambio de modo cada vez que se conecte el dispositivo, se crea una regla udev:

```
echo 'ACTION=="add", SUBSYSTEM=="usb", \
    ATTR{idVendor}=="12d1", ATTR{idProduct}=="1f01", \
    RUN+="/usr/sbin/usb_modeswitch -v 12d1 -p 1f01 -J"' \
    | sudo tee /etc/udev/rules.d/99-huawei-modem.rules
sudo udevadm control --reload-rules
```

E.6.2. Configuración de vehicle.conf

Fichero: /home/deepracer/Desktop/SoftARTEMIS-Original/SoftARTEMIS/vehicle.conf

```
[mqtt]
vehicle_id = 3
mqtt_server_ip = 172.16.0.1
mqtt_server_port = 1883

[cloud_autonomous_driving]
cloud_server_ip = 172.16.0.1
cloud_server_port = 20003

[steering_calibration]
max = 1630000
mid = 1460000
min = 1180000
```

Se utiliza 172.16.0.1 (el *gateway* de la subred UE) como dirección de destino del *broker* y del servidor de control, dado que desde el punto de vista del vehículo el tráfico debe cursar íntegramente a través de la red LTE.

E.6.3. Arranque del servicio ARTEMIS

```
sudo systemctl restart ARTEMIS.service
```

E.6.4. Indicadores de estado LED

Tabla E.4: Indicadores LED del DeepRacer.

Color	Estado
Rojo	Sin conexión, esperando.
Blanco	Intentando conectar.
Amarillo	Conectado al MQTT y al servidor <i>cloud</i> .
Azul	Control <i>cloud</i> activo (AM-Cloud).
Verde	Control autónomo local activo (AM-Local).

E.7. Control del vehículo desde el *core*

El servidor de control recibe datos de cámara y *lidar* del vehículo por UDP y envía comandos de movimiento. Además, reenvía los *frames* a la Jetson Orin Nano para la detección de señales de tráfico con YOLOv11.

E.7.1. Dependencias en el contenedor del *core*

```
apt-get install -y python3-pip python3-opencv
pip3 install pynput numpy
```

E.7.2. Configuración X11 para visualización

En el *host* del *core*:

```
xhost +
```

Dentro del contenedor:

```
export DISPLAY=:1
export LIBGL_ALWAYS_INDIRECT=1
export QT_X11_NO_MITSHM=1
```

La variable QT_X11_NO_MITSHM=1 evita errores `BadAccess` del servidor X cuando se ejecuta OpenCV desde un contenedor Docker.

E.7.3. Parámetros del servidor de control

Tabla E.5: Parámetros de red del servidor de control UDP.

Parámetro	Valor	Función
server_address	('172.16.0.1', 20003)	Escucha conexiones del vehículo por 4G.
JETSON_IP	'192.168.100.102'	IP de la Jetson para reenvío de <i>frames</i> .
JETSON_PORT	20004	Puerto de envío a la Jetson.
RESULTS_PORT	20005	Puerto de recepción de resultados de la Jetson.

E.7.4. Arranque del servidor

```
python3 /coche_autonomo.py
```

E.7.5. Activación del control remoto

```
mosquitto_pub -h localhost -t 3/command -m 'AM-Cloud'
```

E.8. Secuencia completa de arranque del sistema

A continuación se reproduce la secuencia operativa completa para levantar el sistema en cada sesión de laboratorio. El *core* debe arrancarse siempre en primer lugar; el eNodeB no debe iniciarse hasta que la consola del *core* muestre el mensaje `SP-GW Initialized`.

E.8.1. Host del core

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo iptables -t nat -A POSTROUTING -s 172.16.0.0/24 \
-o eno1 -j MASQUERADE
sudo iptables -A FORWARD -s 172.16.0.0/24 -j ACCEPT
sudo iptables -A FORWARD -d 172.16.0.0/24 -j ACCEPT
xhost +
```

E.8.2. Contenedor del core — Terminal 1 (EPC + Mosquitto)

```
docker start 4Gcore && docker exec -it 4Gcore bash
# Dentro del contenedor:
sysctl -w net.ipv4.ip_forward=1
mkdir -p /run/mosquitto && chown mosquitto:mosquitto /run/mosquitto
cd /etc/mosquitto && mosquitto -c mosquitto.conf -d
srsepc ~/.config/srsran/epc.conf
```

E.8.3. Contenedor del *core* — Terminal 2 (servidor de control)

```
docker exec -it 4Gcore bash
export DISPLAY=:1
export LIBGL_ALWAYS_INDIRECT=1
export QT_X11_NO_MITSHM=1
python3 /coche_autonomo.py
```

E.8.4. Contenedor del eNodeB

```
docker start 4Genode && docker exec -it 4Genode bash
# Dentro del contenedor:
bladeRF-cli -l /tmp/hostedxA9.rbf
sysctl -w net.ipv4.ip_forward=1
srsenb ~/.config/srsran/enb.conf
```

E.8.5. Vehículo DeepRacer

```
sudo usb_modeswitch -v 12d1 -p 1f01 -J
sudo systemctl restart ARTEMIS.service
```

E.8.6. Activación del control (Terminal 3 del *core*)

```
docker exec -it 4Gcore bash
mosquitto_pub -h localhost -t 3/command -m 'AM-Cloud'
```

E.8.7. Comandos útiles de operación

Detención de todos los procesos:

```
kill srsepc && kill srsenb && kill mosquitto
kill -f coche_autonomo
fuser -k 20003/udp
```

Copias de seguridad de los contenedores:

```
docker commit 4Gcore bladerf-srsran-core
docker save bladerf-srsran-core | gzip \
> ~/bladerf-srsran-core.tar.gz

docker commit 4Genode bladerf-srsran-enb
docker save bladerf-srsran-enb | gzip \
> ~/bladerf-srsran-enb.tar.gz
```

Bibliografía

- 3GPP. (2014a). *TS 23.401: General Packet Radio Service (GPRS) enhancements for Evolved Universal Terrestrial Radio Access Network (E-UTRAN) access (Release 10)* (inf. téc. N.º V10.13.0) (Publicado por ETSI como TS 123 401. Accedido el 26 de mayo de 2026). 3rd Generation Partnership Project. https://www.etsi.org/deliver/etsi_ts/123400_123499/123401/10.13.00_60/ts_123401v101300p.pdf
- 3GPP. (2014b). *TS 36.101: Evolved Universal Terrestrial Radio Access (E-UTRA); User Equipment (UE) radio transmission and reception (Release 10)* (inf. téc. N.º V10.32.0) (Publicado por ETSI como TS 136 101 V10.32.0. Accedido el 24 de mayo de 2026). 3rd Generation Partnership Project. https://www.etsi.org/deliver/etsi_ts/136100_136199/136101/10.32.00_60/ts_136101v103200p.pdf
- 3GPP. (2014c). *TS 36.211: Evolved Universal Terrestrial Radio Access (E-UTRA); Physical channels and modulation (Release 10)* (inf. téc. N.º V10.7.0) (Publicado por ETSI como TS 136 211. Accedido el 26 de mayo de 2026). 3rd Generation Partnership Project. https://www.etsi.org/deliver/etsi_ts/136200_136299/136211/10.07.00_60/ts_136211v100700p.pdf
- 3GPP. (2014d). *TS 36.300: Evolved Universal Terrestrial Radio Access (E-UTRA) and Evolved Universal Terrestrial Radio Access Network (E-UTRAN); Overall description; Stage 2 (Release 10)* (inf. téc. N.º V10.12.0) (Publicado por ETSI como TS 136 300. Accedido el 26 de mayo de 2026). 3rd Generation Partnership Project. https://www.etsi.org/deliver/etsi_ts/136300_136399/136300/10.12.00_60/ts_136300v101200p.pdf
- 3GPP. (2014e). *TS 36.413: Evolved Universal Terrestrial Radio Access Network (E-UTRAN); S1 Application Protocol (S1AP) (Release 10)* (inf. téc. N.º V10.10.0) (Publicado por ETSI como TS 136 413. Accedido el 26 de mayo de 2026). 3rd Generation Partnership Project. https://www.etsi.org/deliver/etsi_ts/136400_136499/136413/10.10.00_60/ts_136413v101000p.pdf
- Aptica. (2026). XIRIO Online: Herramienta de planificación radioeléctrica [Accedido el 14 de abril de 2026]. <https://www.xirio-online.com/web/home/welcome.aspx>
- Asensio-Pérez, J. I. (2023/2024a). Tema 1: Introducción a la ingeniería de teletráfico [Grado en Ingeniería de Tecnologías de Telecomunicación. Correo: juaase@tel.uva.es]. *Universidad de Valladolid*.
- Asensio-Pérez, J. I. (2023/2024b). Tema 3: Modelos de tráfico [Grado en Ingeniería de Tecnologías de Telecomunicación. Correo: juaase@tel.uva.es]. *Universidad de Valladolid*.
- Asensio-Pérez, J. I. (2023/2024c). Tema 7: Calidad de Servicio en Redes IP [Grado en Ingeniería de Tecnologías de Telecomunicación. Correo: juaase@tel.uva.es]. *Universidad de Valladolid*.
- Atlassian. (2026a). Confluence | AI Workshop for Knowledge Collaboration [Accedido el 14 de abril de 2026]. <https://www.atlassian.com/software/confluence>
- Atlassian. (2026b). Jira | Project Management for the AI Era [Accedido el 14 de abril de 2026]. <https://www.atlassian.com/software/jira>
- Casaseca de la Higuera, P. (2026a). Aprendizaje Automático y Aplicaciones en Visión Artificial [Laboratorio de Procesado de Imagen (LPI)]. <http://www.lpi.tel.uva.es>
- Casaseca de la Higuera, P. (2026b). Tema 6: Codificación y Compresión (Parte 2) [Correo: casaseca@lpi.tel.uva.es]. *Universidad de Valladolid*. <http://www.lpi.tel.uva.es>
- Dirección General de Tráfico. (2026, enero). 2025 finaliza con 1.119 fallecidos en siniestros de tráfico en vías interurbanas [Balance provisional de seguridad vial 2025. Accedido el 24 de mayo de 2026]. <https://www.dgt.es/comunicacion/notas-de-prensa/20260108-2025-finaliza-1119-fallecidos-en-siniestros-de-trafico-en-vias-interurbanas/>
- Durán Barroso, R. J., & de Miguel, I. (2026a). Fundamentos de Aprendizaje Automático para las TIC. Tema 5: El Modelo Lineal: Regresión Logística [Apuntes de la asignatura].

- Durán Barroso, R. J., & de Miguel, I. (2026b). Fundamentos de Aprendizaje Automático para las TIC. Tema 8: Redes Neuronales [Apuntes de la asignatura].
- Eddy Meardon. (2026). What is a Gantt Chart? Key features and benefits with helpful templates [Accedido el 14 de abril de 2026]. <https://community.atlassian.com/forums/App-Central-articles/How-to-Create-Jira-Gantt-Chart-Tips-Alternatives-and-1-Min-Demo/bap/2921394>
- ETSI White Paper. (2023). *MEC Federation: Deployment considerations* (inf. téc. N.º 49) (Accedido el 2 de mayo de 2026). European Telecommunications Standards Institute (ETSI). Sophia Antipolis, Francia. https://www.etsi.org/images/files/ETSIWhitePapers/ETSI_WP_49_MEC-Federation-Deployment-considerations.pdf
- Foschini, G. J., & Gans, M. J. (1998). On Limits of Wireless Communications in a Fading Environment when Using Multiple Antennas. *Wireless Personal Communications*, 6(3), 311-335. <https://doi.org/10.1023/A:1008889222784>
- Galvao, L. G., Abbod, M., Kalganova, T., Palade, V., & Huda, M. N. (2021). Pedestrian and Vehicle Detection in Autonomous Vehicle Perception Systems—A Review. *Sensors*, 21(21), 7267. <https://doi.org/10.3390/s21217267>
- GitHub, Inc. (2026). GitHub · Change is constant [Accedido el 14 de abril de 2026]. <https://github.com/>
- Goldsmith, A. (2005). *Wireless Communications* [isbn:978-0-521-83716-3]. Cambridge University Press.
- Gomez-Miguel, I., Garcia-Saavedra, A., Sutton, P. D., Serrano, P., Cano, C., & Leith, D. J. (2016). srsLTE: An Open-Source Platform for LTE Evolution and Experimentation [Accedido el 24 de mayo de 2026]. *Proceedings of the Tenth ACM International Workshop on Wireless Network Testbeds, Experimental Evaluation, and Characterization (WiNTECH)*, 25-32. <https://doi.org/10.1145/2980159.2980163>
- González, I. R. (2022, septiembre). *Diseño e implementación de un testbed de Edge computing para el soporte de vehículos conectados* [Trabajo Fin de Máster]. Escuela Técnica Superior de Ingenieros de Telecomunicación, Universidad de Valladolid. <https://uvadoc.uva.es/handle/10324/57429>
- Gúeant, V. (2017). iPerf3 and iPerf2 user documentation. <https://iperf.fr/iperf-doc.php>
- Investopedia. (2024). Capital Expenditures (CapEx) vs. Operational Expenditures (OpEx): What's the Difference? [Accedido el 20 de mayo de 2026]. <https://www.investopedia.com/ask/answers/112814/whats-difference-between-capital-expenditures-capex-and-operational-expenditures-opex.asp>
- Ley 11/2022, de 28 de junio, General de Telecomunicaciones (2022). <https://www.boe.es/eli/es/1/2022/06/28/11>
- Jocher, G., & Qiu, J. (2024). *Ultralytics YOLO11* (Ver. 11.0.0) [Accedido el 20 de abril de 2026]. <https://github.com/ultralytics/ultralytics>
- Junta de Castilla y León. (2024). Intensidades medias de tráfico y velocidades. Red regional de carreteras: Datos de la A-601 (2024) [Accedido el 13 de abril de 2026]. <https:// analisis.datosabiertos.jcyl.es/explore/dataset/intensidades-medias-de-trafico-y-velocidades-red-regional-de-carreteras/>
- Real Decreto 123/2017, de 24 de febrero, por el que se aprueba el Reglamento sobre el uso del dominio público radioeléctrico (2017). <https://www.boe.es/eli/es/rd/2017/02/24/123>
- Ministerio de Fomento. (2014). *Norma 8.1-IC. Señalización vertical* (inf. téc.) (Accedido el 24 de abril de 2026). Dirección General de Carreteras. Madrid, España. https://www.transportes.gob.es/recursos_mfom/norma8lic.pdf
- Real Decreto 1066/2001, de 28 de septiembre, por el que se aprueba el Reglamento que establece condiciones de protección del dominio público radioeléctrico, restricciones a las emisiones radioeléctricas y medidas de protección sanitaria frente a emisiones radioeléctricas (2001). <https://www.boe.es/eli/es/rd/2001/09/28/1066>

- Ministerio para la Transformación Digital y de la Función Pública. (2026a). Autorización de uso del dominio público radioeléctrico para pruebas técnicas [Accedido el 18 de mayo de 2026]. <https://sedediatid.digital.gob.es/es-es/procedimientosselectronicos/Paginas/detalle-procedimientos.aspx?IdProcedimiento=34>
- Ministerio para la Transformación Digital y de la Función Pública. (2026b). Geoportal de Cobertura de Banda Ancha y Telefonía Móvil (VCTEL) [Accedido el 27 de marzo de 2026]. <https://geoportal.minetur.gob.es/VCTEL/vcne.do>
- Ministerio para la Transformación Digital y de la Función Pública. (2026c). Pago de tasas de Telecomunicaciones. Modelo 790 [Accedido el 18 de mayo de 2026]. <https://sedediatid.digital.gob.es/es-es/procedimientosselectronicos/Paginas/detalle-procedimientos.aspx?IdProcedimiento=123>
- More, G., Patil, O., More, O., More, M., Suryavanshi, S., & Mali, M. (2024). Comparison of Object Detection Algorithms: CNN, YOLO and SSD [Accedido el 20 de abril de 2026]. *International Journal of Scientific Research and Technology (IJSRT)*, 1(11), 137-144. <https://doi.org/10.5281/zenodo.14186397>
- Myung, H. G., Lim, J., & Goodman, D. J. (2006). Single Carrier FDMA for Uplink Wireless Transmission [Accedido el 26 de mayo de 2026]. *IEEE Vehicular Technology Magazine*, 1(3), 30-38. <https://doi.org/10.1109/MVT.2006.307304>
- NVIDIA Corporation. (2025). *Developing for NVIDIA Jetson Platforms: PyTorch and TensorRT Optimization on ARM aarch64 Architecture* [Technical Resource Guide]. NVIDIA Developer Documentation. <https://developer.nvidia.com/embedded/jetson-linux>
- Reglamento (UE) 2019/2144 del Parlamento Europeo y del Consejo, de 27 de noviembre de 2019, relativo a los requisitos de homologación de tipo de los vehículos de motor y de sus remolques, así como de los sistemas, componentes y unidades técnicas independientes destinados a esos vehículos, en lo que respecta a su seguridad general y a la protección de los ocupantes de los vehículos y de los usuarios vulnerables de la vía pública (2019). <https://eur-lex.europa.eu/eli/reg/2019/2144/oj>
- Roboflow Inc. (2026). Roboflow: Computer vision tools for developers and enterprises [Accedido el 27 de abril de 2026]. <https://roboflow.com/>
- Rodríguez Cayetano, M. (2026). Tema 3. Computación en la nube y virtualización: Introducción a la computación en el borde [Apuntes de la asignatura, versión 1d721ac. Accedido el 25 de mayo de 2026].
- SAE International. (2021). *SAE J3016: Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles* (inf. téc. N.º J3016_202104) (Revisión de 30 de abril de 2021. Accedido el 24 de mayo de 2026). Society of Automotive Engineers. Warrendale, PA, EE.UU. https://www.sae.org/standards/content/j3016_202104
- Sesia, S., Toufik, I., & Baker, M. (2011). *LTE — The UMTS Long Term Evolution: From Theory to Practice* (2.ª ed.) [Texto de referencia para la arquitectura LTE/EPS y la capa física, isbn:978-0-470-66025-6]. John Wiley & Sons.
- Sharkyun. (2023). *Computer Vision: Object Detection — One-Stage vs Two-Stage* [Accedido el 21 de abril de 2026]. <https://sharkyun.medium.com/computer-vision-object-detection-one-stage-vs-two-stage-b05dbff88195>
- Software Radio Systems. (2026). srsRAN documentation: 4G/5G open-source SDR suite [Accedido el 24 de mayo de 2026]. <https://docs.srsran.com/projects/4g/>
- TelecomHall Forum. (2021). Radio Physical Resources. <https://www.telecomhall.net/t/radio-physical-resources/11945>
- Wikipedia. (2026). Autovía de Pinares [Accedido el 12 de mayo de 2026]. https://es.wikipedia.org/wiki/Autov%C3%ADa_de_Pinares